

Exercise 1 :

You are given the following four domains:

- i. A group of people playing poker.
- ii. A person driving a car.
- iii. A machine detecting chocolate bars weighing less than 50g.
- iv. A doctor performing a medical diagnosis.

Classify each of the domains above along with the domain properties that we discussed in the lecture by entering yes/no in the cells of the following table:

	Accessible	Deterministic	Episodic	Static	Discrete	Single agent
Poker						
Car						
Machine						
Doctor						

Solution:

	Accessible	Deterministic	Episodic	Static	Discrete	Single agent
Poker	<i>no</i>	<i>no</i>	<i>no</i>	<i>yes</i>	<i>yes</i>	<i>no</i>
Car	<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>
Machine	<i>yes</i>	<i>yes</i>	<i>yes</i>	<i>yes</i>	<i>yes</i>	<i>yes</i>
Doctor	<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>	<i>yes</i>

Exercise 2 :

For each of the following assertions, say whether it is true or false and give a short explanation in 1 or 2 sentences for each of your answers.

- (a) An agent that senses only partial information about the state cannot be perfectly rational.

Solution: False. Perfect rationality refers to the ability to make good decisions given the sensor information received.

- (b) There exist task environments in which no pure reflex agent can behave rationally.

Solution: True. A pure reflex agent ignores previous percepts, so cannot obtain an optimal state estimate in a partially observable environment. For example, correspondence chess is played by sending moves; if the other player's move is the current percept, a reflex agent could not keep track of the board state and would have to respond to, say, "a4" in the same way regardless of the position in which it was played.

- (c) There exists a task environment in which every agent is rational.

Solution: True. For example, in an environment with a single state, such that all actions have the same reward, it doesn't matter which action is taken. More generally, any environment that is reward-invariant under permutation of the actions will satisfy this property.

- (d) The input to an agent program is the same as the input to the agent function.

Solution: False. The agent function, notionally speaking, takes as input the entire percept sequence up to that point, whereas the agent program takes the current percept only.

- (e) It is possible for a given agent to be perfectly rational in two distinct task environments.

Solution: True. For example, we can arbitrarily modify the parts of the environment that are unreachable by any optimally acting agent as long as they stay unreachable.

- (f) Every agent is rational in an unobservable environment.

Solution: False. Some actions are stupid—and the agent may know this if it has a model of the environment—even if one cannot perceive the environment state.

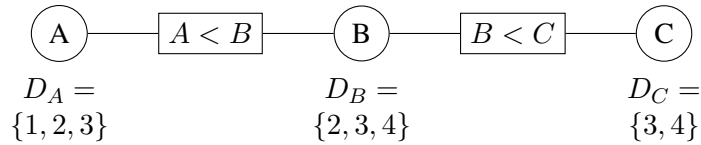
- (g) A perfectly rational poker-playing agent never loses.

Solution: False. Unless it draws the perfect hand, the agent can always lose if an opponent has better cards. This can happen for game after game. The correct statement is that the agent's expected winnings are non-negative.

- (h) Suppose an agent selects its next action uniformly at random from a set of possible actions. There exists a deterministic task environment in which this agent is rational.

Solution: True. Imagine an environment with only one action.

Exercise 1 :



- Which arcs in the above constraint network are arc consistent?
- How can the whole network be made arc consistent, i.e., which changes should be made to the domains of the variables making the network arc-consistent without eliminating potential solutions?

Solution:

- The arc $\langle B, B < C \rangle$ is not consistent, because for $B = 4$ there is no value in D_C that satisfies the constraint. All other arcs are arc consistent.
- First the value 4 has to be removed from D_B . Now the arc $\langle A, A < B \rangle$ no longer is consistent, because for $A = 3$ now no suitable value for B exists. After also removing 3 from D_A the network is arc consistent.

Exercise 2 :

Consider the following constraint network $\gamma = (V, D, C)$:

- Variables: $V = \{a, b, c, d\}$.
- Domains: For all $v \in V$: $D_v = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.
- Constraints: $2|a - c| > 3$; $b^2 - 3d < 9$; $b + 3 < c$.

Run the Generalized Arc Consistency(γ) algorithm, as specified in the lecture. Precisely, for each iteration of the while-loop, give the content of To-do-arcs at the start of the iteration, give the pair (x, c) removed from To-do-arcs, give the domain D_x of x after the call to $\text{Revise}(\gamma, x, c)$, and give the pairs (y, c') added into M .

Note: Initialize To-do-arcs as a lexicographically ordered list (i.e., (a, b) would be before (a, c) , both before (b, a) etc., if any of those exist). Furthermore, use to-do-arcs as a FIFO queue, i.e., always remove the element at the front and add new elements at the back.

Solution: Here is the required information for the iterations of the while-loop as executed. In this case, we always removed the element at the front first, and added new elements to the end.

For brevity, we call To-do-arcs M and use (a, c) to refer to variable a and the constraint C_{ac} .

1. M content: $\{(a, c), (b, c), (b, d), (c, a), (c, b), (d, b)\}$; pair selected: (a, c) ; No change, so nothing added to M .

2. M content: $\{(b, c), (b, d), (c, a), (c, b), (d, b)\}$; pair selected: (b, c) ; $D_b = \{1, 2, 3, 4, 5, 6\}$. Candidate (d, b) already present in M .
3. M content: $\{(b, d), (c, a), (c, b), (d, b)\}$; pair selected: (b, d) ; No change, so nothing added to M .
4. M content: $\{(c, a), (c, b), (d, b)\}$; pair selected: (c, a) ; No change, so nothing added to M .
5. M content: $\{(c, b), (d, b)\}$; pair selected: (c, b) ; $D_c = \{5, 6, 7, 8, 9, 10\}$. Candidate (a, c) added into M .
6. M content: $\{(d, b), (a, c)\}$; pair selected: (d, b) ; No change, so nothing added to M .
7. M content: $\{(a, c)\}$; pair selected: (a, c) ; No change, so nothing added to M .
8. M empty; return modified γ with reduced domains.

Exercise 3 :

Solve exercise 4.14 in PM about variable elimination. (<https://artint.info/3e/html/ArtInt3e.Ch4.S12.html>)

Solution:

- r_1 and r_2 are removed. $r_{11}(B, C)$ is added.
- R_3, r_4 , and r_{11} are removed. $R_{12}(C, D, E)$ is added.

Exercise 4 :

Use Variable Elimination to solve the following CSP given by extensional constraints on Boolean variables A, B, C :

A	B	A	C	B	C
t	f	t	f	t	f
t	t	f	t	f	t
f	t				

Solution:

We eliminate B first. Joining the two tables containing B gives

A	B	B	C	A	B	C
t	f	t	f	t	f	t
t	t	f	t	t	t	f
f	t	f	t	f	t	f

Projecting on A, C then gives

A	C
t	t
t	f
f	f

We eliminate C next (the order in which we perform the elimination is not very important in this example). We now have two tables containing C , which we join:

A	C	$\bowtie$	A	C	$=$	A	C
t	f		t	t		t	f
f	t		t	f			
			f	f			

The result of this join now is the only constraint left. Therefore, the algorithm terminates with the result that the solutions of the constraint satisfaction problem consist of tuples with $A = t$ and $C = f$. To also determine the possible values for B , we have to join this result table with the previous constraint we computed for A, B, C :

A	C	$\bowtie$	A	B	C	$=$	A	B	C
t	f		t	f	t		t	t	f
			t	t	f				
			f	t	t				

This means that there exists exactly one solution to the problem: $A = t, B = t, C = f$.

Exercise 5 :

Consider the cryptarithmic puzzle

$$\begin{array}{r} \text{T} \quad \text{W} \quad \text{O} \\ + \quad \text{T} \quad \text{W} \quad \text{O} \\ \hline \text{F} \quad \text{O} \quad \text{U} \quad \text{R} \end{array}$$

Each letter in a cryptarithmic problem represents a different digit; note that F cannot be 0.

- Construct a constraint network representation for the puzzle. *Hint:* For each of the four columns in the table representation above we have a constraint. E.g. for the first column we have the constraint

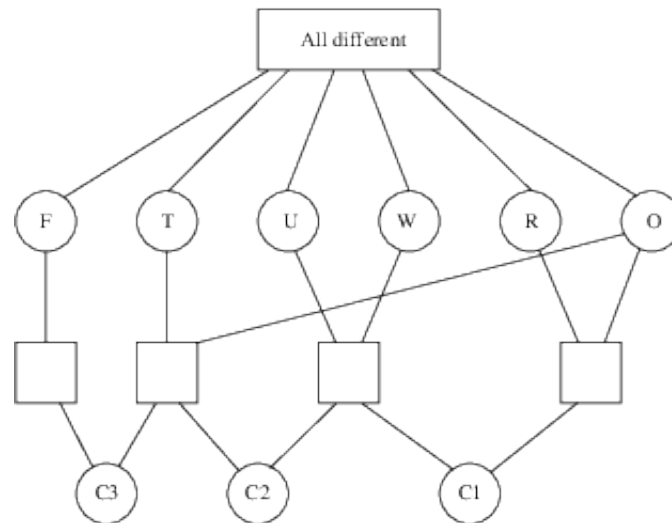
$$O + O = R + 10 \cdot C_1,$$

where C_1 is an auxiliary variable representing what is carried over in the 10 column. Use similar auxiliary variables, say C_2 and C_3 , for encoding what is carried over to the 100 and 1000 column, respectively.

- Perform a BacktrackingWithInference to find a solution to the puzzle. *Hint:* The ordering of the variables has a big impact on the solution size, so a good strategy would be to choose the variable with the smallest number of possible values. E.g. consider starting with $C_3 = 1$.

Solution:

- Here is a constraint network for the cryptarithmic problem



The constraint connecting R , O , and C_1 encodes that $O + O = R + 10 \cdot C_1$. Similar for the other constraints connected to the auxiliary variables.

- There are many possible solutions to the problem. One possibility is to
 - Set $C_3 = 1$.
 - Set $F = 1$ (the only possible value after applying consistency).
 - Set (arbitrarily) $C_2 = 0$.
 - Set (arbitrarily) $C_1 = 0$.
 - By arc consistency, O must be an even number (it is equal to $T + T$) and less than 5 since $C_1 = 0$. Set (arbitrarily) $O = 4$.
 - By arc consistency, R now only has one value, namely 8.
 - By arc consistency, T only has one value, namely 7.
 - By arc consistency, U must be an even number less than 9. Set $U = 6$.
 - By arc consistency, set $W = 3$.

Exercise 6 :

Consider the following mini-Sudoku:

		4	
2			
		1	
3			

The empty fields have to be filled with numbers 1,2,3, or 4, such that each row, each column, and each of the 2×2 sub-squares contain each of these numbers exactly once.

- a. Formalize this problem as a constraint satisfaction problem: define an appropriate set of variables, and a set of constraints that define the solutions of the sudoku.
- Hint 1: instead of using constraints as on the lecture slide, define a smaller set of constraints, each constraint expressing the full condition for one row, one column, or one sub-square. We have not discussed off-the-shelf formal languages for expressing such constraints, so you should try to express them as formally as possible. But don't overthink it, as long as the constraints are not ambiguous, it is fine).
 - Hint 2. To define variables, you can simply label the empty fields with variables like this:

A1	A2	4	A4
2	B2	B3	B4
C1	C2	1	C4
3	D2	D3	D4

- b. Draw the constraint network for this problem. If this gets too large, draw only the part of the network that is sufficient to answer the next question.
- c. Apply the generalized arc consistency algorithm to show that the top left square must contain a 1. Does the operation of the algorithm resemble how you would come to that conclusion yourself?

Solution:

Label the empty fields with variables like this:

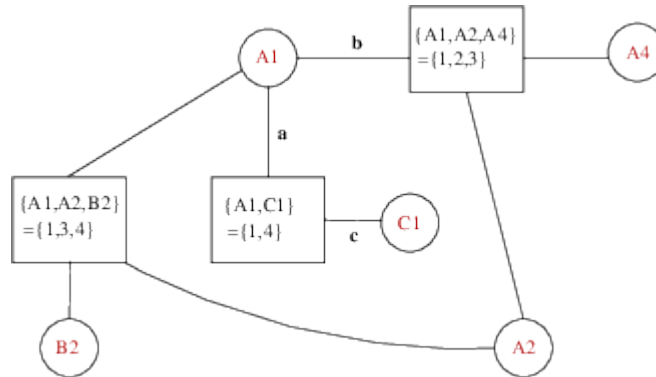
A1	A2	4	A4
2	B2	B3	B4
C1	C2	1	C4
3	D2	D3	D4

We now know that $A1, A2, A4$ combined must contain the values 1,2,3. We write this as a constraint in the form:

$$\{A1, A2, A4\} = \{1, 2, 3\}$$

Similarly, we define one constraint for each remaining row, column, and 2×2 sub-square.

A part of the constraint network now is:

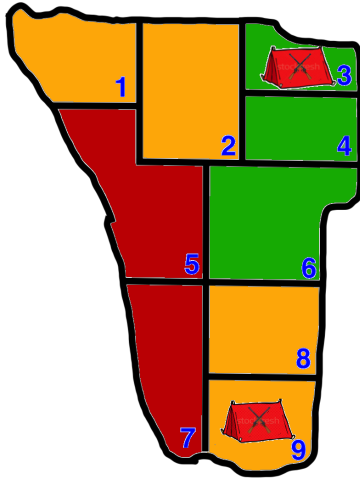


Initially, we have $D_{A1} = D_{A2} = \dots D_{A4} = \{1, 2, 3, 4\}$. We first find that the arc **a** is not arc consistent, because for the values $A1 = 2$ or $A1 = 3$ the constraint $\{A1, C1\} = \{1, 4\}$ can not be satisfied. Thus, we delete 2 and 3 from D_{A1} . Since $D_{C1} = \{1, 2, 3, 4\}$ the arc now is arc consistent. Next inspect arc **b**. For $A1 = 4$ there are no values of $A2$ and $A4$ that could make the constraint satisfied, so 4 also is removed from D_{A1} . With $D_{A2} = D_{A4} = \{1, 2, 3, 4\}$ arc **b** now is arc consistent. At this point we have $D_{A1} = \{1\}$, i.e. we know that in any possible solution of the sudoku $A1$ must have value 1. We can now continue. For example, making arc **c** arc consistent immediately leads to the reduction $D_{C1} = \{4\}$.

Exercise 7 :

(Extra exercise for modelling CSPs)

You are a ranger and own a piece of land, depicted below. You want to make sure that all of your animals survive. This is only possible if no animals that kill each other live in the same or neighboring regions. You own rhinos (R), elephants (E), cheetahs (C), lions (L), jackals (J), springbok (S), zebras (Z), hippos (H) and giraffes (G). Your piece of land contains three different climatic regions: bush (green), desert (red), savannah (orange). Some animals need specific climatic environments. Unfortunately there are also two camps of poachers on the land, who are selling rhino horn and elephant tusk. Poachers treacherously shoot animals in their own and neighboring regions.



- Poachers kill rhinos and elephants
- Jackals eat springbok
- Cheetahs eat springbok and zebras
- Lions eat springbok, zebras and giraffes, and sometimes even kill elephants

- Jackals and cheetahs can live in the desert and in the savannah
- Elephants can only live in the bush
- Lions and zebras can live in the bush and in the savannah
- Springbok and hippos only live in the savannah regions

Formulate the problem as a constraint network. Use the set of variables $V = \{v_1, \dots, v_9\}$, where variable v_i models which animal can live in region i .

- Specify the domain D_{v_i} of each variable $v_i \in V$. That is, for each region type give the set of animals that can live in that region, and state which variables have this domain.
- Specify the binary constraints $C_{v_i v_j} \subseteq D_{v_i} \times D_{v_j}$ for all pairs of variables. That is, for each combination of regions give the set of animal-pairs that the region pair can inhabit, and state which variable pairs have this constraint.
- Specify the unary constraints $C_{v_i} \subseteq D_{v_i}$ for all relevant variables. That is, for each region affected by the poachers, give the set of animals that can inhabit that region.

Hint: You can specifying constraints with those pairs that are not contained:

$$C_{v_i, v_j} = D_{v_i} \times D_{v_j} \setminus \{(d_{v_i, k}, d_{v_j, l}), \dots\}$$

Solution: Solution:

- $D_S = D_{v_1} = D_{v_2} = D_{v_8} = D_{v_9} = \{R, J, C, S, Z, L, G, H\}$
 - $D_B = D_{v_3} = D_{v_4} = D_{v_6} = \{R, E, Z, L, G\}$
 - $D_D = D_{v_5} = D_{v_7} = \{R, J, C, G\}$
- $C_{sav, sav} = C_{v_1 v_2} = C_{v_8 v_9}$
 $= D_S \times D_S \setminus \{(S, J), (J, S), (C, S), (S, C), (C, Z), (Z, C),$
 $(L, S), (L, Z), (L, G), (G, L), (Z, L), (S, L)\}$
 - $C_{des, des} = C_{v_5 v_7}$
 $= D_D \times D_D$

- $C_{bush,bush} = C_{v_3v_4} = C_{v_4v_6}$
 $= D_B \times D_B \setminus \{(L, Z), (L, G), (L, E), (Z, L), (G, L), (E, L)\}$
 - $C_{sav,des} = C_{v_1v_5} = C_{v_2v_5} = C_{v_8v_7} = C_{v_9v_7}$
 $= D_S \times D_D \setminus \{(S, J), (S, C), (Z, C), (L, G)\}$
 - $C_{sav,bush} = C_{v_2v_3} = C_{v_2v_4} = C_{v_8v_6} = C_{v_2v_6}$
 $= D_S \times D_B \setminus \{(C, Z), (L, Z), (L, G), (L, E), (S, L), (Z, L), (G, L)\}$
 - $C_{des,bush} = C_{v_5v_6}$
 $= D_D \times D_B \setminus \{(C, Z), (G, L)\}$
- (iii)
- $D_2 = D_8 = D_9 = D_S \setminus \{R\} = \{J, C, S, Z, L, G, H\}$
 - $D_4 = D_3 = D_B \setminus \{R, E\} = \{Z, L, G\}$
 - $D_7 = D_D \setminus \{R\} = \{J, C, G\}$

Exercise 8 :

(Extra exercise to practice arc consistency (a bit long))

On your piece of land you additionally own a lodge for tourists and a cat sanctuary. You want to show your tourists all your nice animals and everything the region has to offer. You have 5 different time slots: 1, 2, 3, 4, 5, and want to fit all the tasks (see below) in, such that all of the constraints (see below) are satisfied.

Tasks:

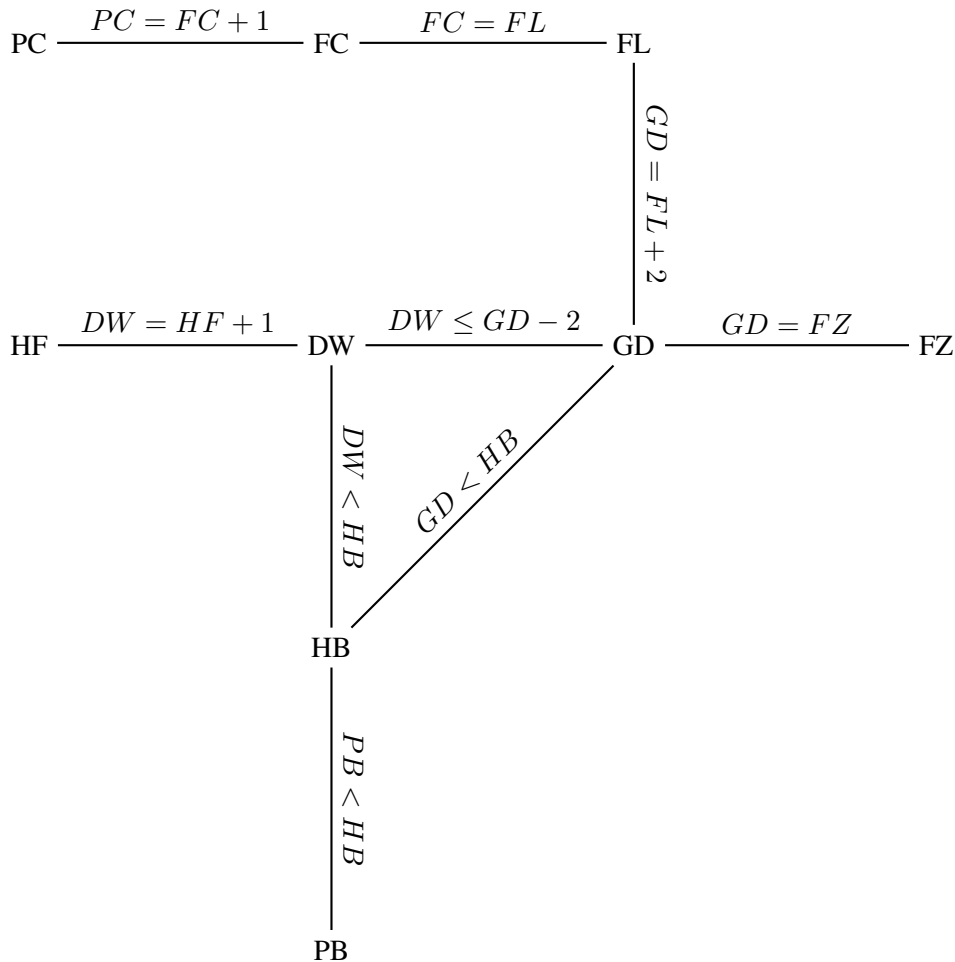
- (GD) go for game drive with tourists
- (DW) go to the desert with tourists
- (HF) make helicopter flight with tourists
- (HB) have braai with tourists
- (PB) prepare braai
- (PC) pet cheetahs
- (FL) feed lions
- (FC) feed cheetahs
- (FZ) feed zebra

Constraints:

- Cheetahs need to be fed immediately before you can pet them ($FC = PC - 1$)
- Lions and cheetahs have to be fed at the same time, otherwise they will get jealous ($FC = FL$)
- The desert walk has to be (at least) two time slots before the game drive, otherwise your tourists will get to exhausted ($DW \leq GD - 2$)
- In order to have braai with your tourists you first have to exhaust them on the game drive and on the desert walk, so they can enjoy their beer ($GD < HB$; $DW < HB$)
- You have to prepare braai before having the braai with your tourists ($PB < HB$)
- You can only find lions on the game drive if they have been fed two time slots earlier ($GD = FL + 2$)

- In order to give your tourists an impression of the area first, the heli flight is done just before the desert walk ($HF = DW - 1$)
 - In order to save time you want to feed the zebra on the game drive ($FZ = GD$)
 - Variables: $V = \{DW, FC, FL, FZ, GD, HB, HF, PC, PB\}$
 - Domains: For all $v \in V$: $D_v = \{1, 2, 3, 4, 5\}$
- (i) Draw the constraint graph of γ . Label each edge with the corresponding constraint.
- (ii) Run Generalized Arc consistency. Initialize To-do-arcs with an alphabetical list. For each step: Give the selected pair, the updated domain, and the pairs that are added to To-do-arcs (give all pairs that would be added and mark those that are not already contained). You do not have to give the updated To-do-arcs list in each iteration.

Solution: Solution:



(ii) $M = \{(DW, GD), (DW, HB), (DW, HF), (FC, FL), (FC, PC), (FL, FC), (FL, GD), (FZ, GD), (GD, DW), (GD, FL), (GD, FZ), (GD, HB), (HB, DW), (HB, GD), (HB, PB), (HF, DW), (PB, HB), (PC, FC)\}$

- (1) **selected:** (DW, GD) ; $D_{DW} = \{1, 2, 3\}$; **add:** $(HB, DW), (HF, DW)$
- (2) **selected:** (DW, HB) ; no changes
- (3) **selected:** (DW, HF) ; $D_{DW} = \{2, 3\}$; **add:** $(GD, DW), (HB, DW)$
- (4) **selected:** (FC, FL) ; no changes
- (5) **selected:** (FC, PC) ; $D_{FC} = \{1, 2, 3, 4\}$; **add:** (FL, FC)
- (6) **selected:** (FL, FC) ; $D_{FL} = \{1, 2, 3, 4\}$; **add:** (GD, FL)
- (7) **selected:** (FL, GD) ; $D_{FL} = \{1, 2, 3\}$; **add:** (FC, FL)
- (8) **selected:** (FZ, GD) ; no changes
- (9) **selected:** (GD, DW) ; $D_{GD} = \{4, 5\}$; **add:** $(FL, GD), (FZ, GD), (HB, GD)$
- (10) **selected:** (GD, FL) ; no changes
- (11) **selected:** (GD, FZ) ; no changes
- (12) **selected:** (GD, HB) ; $D_{GD} = \{4\}$; **add:** $(DW, GD), (FL, GD), (FZ, GD)$
- (13) **selected:** (HB, DW) ; $D_{HB} = \{3, 4, 5\}$; **add:** $(GD, HB), (PB, HB)$
- (14) **selected:** (HB, GD) ; $D_{HB} = \{5\}$; **add:** $(DW, HB), (PB, HB)$
- (15) **selected:** (HB, PB) ; no changes
- (16) **selected:** (HF, DW) ; $D_{HF} = \{1, 2\}$; no adds
- (17) **selected:** (PB, HB) ; $D_{PB} = \{1, 2, 3, 4\}$; no adds
- (18) **selected:** (PC, FC) ; $D_{PC} = \{2, 3, 4, 5\}$; no adds
- (19) **selected:** (FC, FL) ; $D_{FC} = \{1, 2, 3\}$; **add:** (PC, FC)
- (20) **selected:** (FL, GD) ; $D_{FL} = \{2\}$; **add:** (FC, FL)
- (21) **selected:** (FZ, GD) ; $D_{FZ} = \{4\}$; no adds
- (22) **selected:** (DW, GD) ; $D_{DW} = \{2\}$; **add:** $(HB, DW), (HF, DW)$
- (23) **selected:** (GD, HB) ; no changes
- (24) **selected:** (DW, HB) ; no changes
- (25) **selected:** (PC, FC) ; $D_{PC} = \{2, 3, 4\}$; no adds
- (26) **selected:** (FC, FL) ; $D_{FC} = \{2\}$; **add:** (PC, FC)
- (27) **selected:** (HB, DW) ; no changes
- (28) **selected:** (HF, DW) ; $D_{HF} = \{1\}$; no adds
- (29) **selected:** (PC, FC) ; $D_{PC} = \{3\}$; no adds

M empty; return modified γ :

$D_{DW} = \{2\}$

$D_{FC} = \{2\}$

$D_{FL} = \{2\}$

$$D_{FZ} = \{4\}$$

$$D_{GD} = \{4\}$$

$$D_{HB} = \{5\}$$

$$D_{HF} = \{1\}$$

$$D_{PB} = \{1, 2, 3, 4\}$$

$$D_{PC} = \{3\}$$

Exercise 1 :

Calculate $P(A)$, $P(B)$, $P(A|B)$ from the joint probability distribution $P(A, B)$ given in Table 1.

	b_1	b_2	b_3
a_1	0.05	0.10	0.05
a_2	0.15	0.00	0.25
a_3	0.10	0.20	0.10

Table 1: The joint probability distribution for $P(A, B)$.

Solution: In order to find $P(A)$ we need to marginalize (sum) out the variable B . This is done for each state of A . Thus, for $A = a_1$ we get:

$$P(A = a_1) = \sum_B P(A = a_1, B) = 0.05 + 0.10 + 0.05 = 0.20$$

In total we end up with $P(A) = (0.2, 0.4, 0.4)$. Using a similar procedure to find $P(B)$ (now summing out A) we get $P(B) = (0.3, 0.3, 0.4)$.

In order to find $P(A|B)$ we use

$$P(A|B) = \frac{P(A, B)}{P(B)},$$

hence for $B = b_1$ we get

$$P(A|b_1) = \frac{P(A, B = b_1)}{P(B = b_1)} = \frac{(0.05, 0.15, 0.10)}{0.3} = (0.167, 0.5, 0.333).$$

The operations are similar for the other states of B , which gives $P(A|b_2) = (0.333, 0, 0.667)$ and $P(A|b_3) = (0.125, 0.625, 0.25)$.

Last, if we want to compute also $P(B|A)$ (not asked in the exercise), we can calculate these probabilities following the same steps as for $P(A|B)$, resulting in

$$P(B|a_1) = (0.25, 0.5, 0.25)$$

$$P(B|a_2) = (0.375, 0, 0.625)$$

$$P(B|a_3) = (0.25, 0.5, 0.25).$$

Exercise 2 :

Consider the network in Figure 1. Which conditional probability tables must be specified to turn the graph into a Bayesian network? Assuming that all variables are binary, what is the size of those tables (i.e., how many values do we need to specify?)

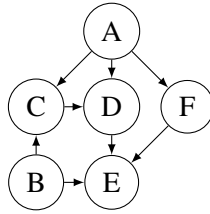
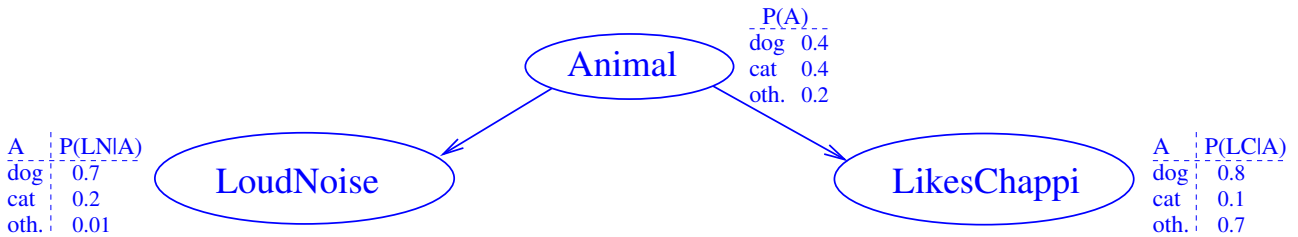


Figure 1: A Bayesian network for Exercise 2.

Solution: The needed tables are $P(A)$, $P(B)$, $P(C | A, B)$, $P(D | A, C)$, $P(E | B, D, F)$, and $P(F | A)$. The sizes are $2 + 2 + 8 + 8 + 16 + 4 = 40$.

Exercise 3 :

Consider the following Bayesian network BN :



Use this network to compute the following probabilities:

- (a) $P(\text{loudnoise}, \text{dog}, \text{likeschappi})$.
- (b) $P(\text{loudnoise}, \text{other}, \text{likeschappi})$.
- (c) $P(\neg \text{loudnoise}, \text{dog}, \text{likeschappi})$.

Include intermediate steps at a level of granularity as in the lecture slides examples. In particular, for each of the probabilities P demanded in (a) – (c), write down *which* probabilities provided in BN can be combined *how* to obtain P .

Solution:

In all cases, we use the method from slides 42 and 43. As the variable ordering consistent with BN , we choose $X_1 = \text{Animal}$, $X_2 = \text{LoudNoise}$, $X_3 = \text{LikesChappi}$.

- (a) $P(\text{loudnoise}, \text{dog}, \text{likeschappi}) = P(\text{likeschappi} | \text{loudnoise}, \text{dog}) * P(\text{loudnoise} | \text{dog}) * P(\text{dog}) = P(\text{likeschappi} | \text{dog}) * P(\text{loudnoise} | \text{dog}) * P(\text{dog}) = 0.8 * 0.7 * 0.4 = 0.224$.
- (b) $P(\text{loudnoise}, \text{other}, \text{likeschappi}) = P(\text{likeschappi} | \text{loudnoise}, \text{other}) * P(\text{loudnoise} | \text{other}) * P(\text{other}) = P(\text{likeschappi} | \text{other}) * P(\text{loudnoise} | \text{other}) * P(\text{other}) = 0.7 * 0.01 * 0.2 = 0.0014$.

(c) $P(\neg \text{loudnoise}, \text{dog}, \text{likeschappi}) = P(\text{likeschappi} \mid \neg \text{loudnoise}, \text{dog}) * P(\neg \text{loudnoise} \mid \text{dog}) * P(\text{dog}) = P(\text{likeschappi} \mid \text{dog}) * P(\neg \text{loudnoise} \mid \text{dog}) * P(\text{dog}) = 0.8 * 0.3 * 0.4 = 0.096$.
 Note here that $P(\neg \text{loudnoise} \mid \text{dog})$ is not explicitly given in the picture, but can be calculated as $1 - P(\text{loudnoise} \mid \text{dog})$; compare Chapter 16 slide 12.

Exercise 4 :

We want to construct a Bayesian network for the following random variables (all with domain *true,false*):

<i>Mexico</i>	Person X has recently travelled to Mexico
<i>Svine_flu_infection</i>	Person X has been infected with the svine flu virus
<i>Vaccination</i>	Person X has been vaccinated against svine flu
<i>Svine_flu_sick</i>	Person X is sick from svine flu
<i>Fever</i>	Person X has fever

- a. Use the above ordering of the variables to determine a Bayesian network structure based on the chain rule and conditional independence relations.
- b. Repeat the construction with the alternative variable ordering:

Vaccination, Mexico, Svine_flu_sick, Fever, Svine_flu_infection

Solution:

- a. We write the joint distribution of the variables using the chain rule:

$$\begin{aligned}
 P(M, S_f_i, V, S_f_s, F) = & \\
 & P(M) \cdot \\
 & P(S_f_i \mid M) \cdot \\
 & P(V \mid M, S_f_i) \cdot \\
 & P(S_f_s \mid M, S_f_i, V) \cdot \\
 & P(F \mid M, S_f_i, V, S_f_s) \cdot
 \end{aligned}$$

We now simplify the conditional distributions by making some conditional independence assumptions (these are reasonable assumptions based on our knowledge of the domain – but not necessarily provably correct).

$$P(V \mid M, S_f_i) = P(V \mid M)$$

Justification: whether or not a person gets a vaccination may depend on whether he plans to travel to Mexico. His decision to get a vaccination (generally) takes place before he the potential time of infection, so the decision is independent of whether he will catch the virus.

$$P(S_f_s \mid M, S_f_i, V) = P(S_f_s \mid S_f_i, V)$$

Justification: Given that we know whether a person has been infected with the virus, and whether or not he has a vaccination, it is no longer relevant to know where he may have contracted the virus, especially

whether or not he traveled to Mexico. Note that this does not mean that $P(S_{f_s} | M) = P(S_{f_s})$: as long as we don't know about S_{f_s} and V , the information of whether the person traveled to Mexico is still very relevant for the probability of S_{f_s} .

$$P(F | M, S_{f_i}, V, S_{f_s}) = P(F | S_{f_s})$$

Justification: the fever is a direct consequence of the sickness. Once we know whether a person is sick, it gives us no additional useful information (regarding the probability of fever) to know whether the person was in Mexico, or got a vaccination.

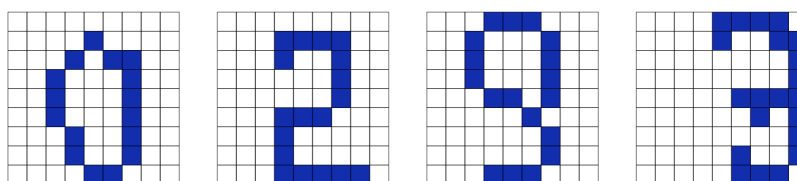
The representation of the joint distribution as the product of the simplified conditional distributions corresponds to the Bayesian network structure

b.

Going through the same procedure for the alternative ordering leads to fewer simplifications, and a more complicated network structure. The basic reason for this is that the second ordering does not respect the cause-effect relationships between the variables (i.e., taking cause variables before their effects).

Exercise 5 :

Design a Bayesian network that can be used to recognize handwritten digits 0,1,2,...,9 from scanned, pixelated images like these:



- What are hypothesis and information variables?
- Could there be any useful mediating variables (consider e.g. the last image above)?
- How could you design a network structure
 - so that the conditional independencies are (approximately) reasonable
 - so that specification and inference complexity remain feasible
- How do you fill in the conditional probability tables?

Solution:

- The hypothesis variable(s) must enable us to make the intended inferences by querying these variables. Here we are interested in predicting which digit is actually represented by the image. Thus, we should have a variable

$$digit \in \{0, 1, \dots, 9\}$$

The information variables must allow us to enter into the Bayesian networks the relevant observations which we make. Here we observe $9 \cdot 9 = 81$ pixels, each of which can be black or white:

$$pixel_i \in \{b, w\} \quad i = 1, \dots, 81$$

- Mediating variables: see below
- Writing a given digit can be seen as a cause for the pixels to become black or white. Just inserting edges for these direct cause-effect relationships gives us the structure. According to this structure, any two pixels are conditionally independent given the digit (this type of structure is also called a *Naive Bayes Model*). This is not entirely realistic: for example consider pixels 27 and 36: According to the network structure:

$$P(pixel_27 = b \mid digit = 3, pixel_36 = b) = P(pixel_27 = b \mid digit = 3),$$

i.e. knowing that the actual digit is a 3, the information on pixel 36 carries no relevant information for pixel 27. However, if in addition to $digit=3$ we know that $pixel_36=b$, this will indicate that the digit is written flush right in the box, and should thereby increase the probability of $pixel_27=b$.

We can improve the model by introducing a mediating variable

$$alignment \in \{l, r, c\}$$

(for left, right, and center alignment) and add it to the network like this: Now two pixels are only independent given both $digit$ and $alignment$ – which still will not make this a perfectly accurate model, but already better than the first one.

Exercise 6 :

For 10000 emails in your inbox you determine the values of the following three boolean variables:

Spam the email is spam
Caps the subject line is in all capital letters
Pills Body of the message contains the word “pills”

You obtain the following counts:

	<i>Caps</i>			
	<i>yes</i>		<i>no</i>	
<i>Spam</i>	<i>Pills</i>		<i>Pills</i>	
	<i>yes</i>	<i>no</i>	<i>yes</i>	<i>no</i>
<i>yes</i>	150	850	600	3400
<i>no</i>	1	99	49	4851

Are

- *Spam* and *Caps* independent?
- *Pills* and *Caps* independent?

- *Pills* and *Caps* independent given *Spam*?
- *Spam* and *Caps* independent given *Pills*?

Solution:

To determine whether *Spam* and *Caps* are independent, we consider the joint and marginal distribution of these two variables:

<i>Spam</i>	<i>Caps</i>		
	<i>yes</i>	<i>no</i>	
<i>yes</i>	0.1	0.4	0.5
<i>no</i>	0.01	0.49	0.5
	0.11	0.89	

The entries are obtained by summing over the *Pills* variable, and normalizing by dividing by 10000. E.g. $0.4 = \frac{600+3400}{10000}$. Since, for example $P(\text{Spam} = \text{no}, \text{Caps} = \text{yes}) = 0.01 \neq P(\text{Spam} = \text{no})P(\text{Caps} = \text{yes}) = 0.5 \cdot 0.11 = 0.055$, we see that the two variables are not independent.

To see whether *Spam* and *Caps* are independent given *Pills*, we first determine the conditional distribution of *Spam* and *Caps* given *Pills*=*yes*:

<i>Spam</i>	<i>Caps</i>		
	<i>yes</i>	<i>no</i>	
<i>yes</i>	0.1875	0.75	0.9375
<i>no</i>	0.00125	0.06125	0.0625
	0.18875	0.81125	

Here, e.g. $0.1875 = \frac{150}{800}$ (800 is the total number of cases with *Pills*=*yes*).

Again, we find that the joint distribution is not the product of the marginals, e.g. $0.0625 \cdot 0.18875 = 0.0118 \neq 0.00125$. Thus, *Spam* and *Caps* are not independent given *Pills*.

For the (conditional) independence of *Pills* and *Caps* one proceeds in the same manner. Now the findings should be: *Pills* and *Caps* are not independent, but *Pills* and *Caps* are independent given *Spam*. For the last result one has to check both the conditional distribution given *Spam*=*yes* and given *Spam*=*no* (above it was enough to consider the conditional distribution of *Spam* and *Caps* given *Pills*=*yes*, because there we already found that *Spam* and *Caps* are not conditionally independent).

Exercise 7 (Optional):

Consider the experiment of flipping a coin, and if it lands heads, rolling a four-sided die, and if it lands tails, rolling a six-sided die.

- Suppose that the coin and both dice are fair. As we are interested only in the number rolled by the die, and the possible worlds $\mathcal{S}_A$ for the experiment could thus be the numbers from 1 to 6. Another set of possible worlds could be $\mathcal{S}_B = \{t1, \dots, t6, h1, \dots, h4\}$, with for example *t2* meaning “tails and a roll of 2” and *h4* meaning “heads and a roll of 4.” Choose either $\mathcal{S}_A$ or $\mathcal{S}_B$ and associate probabilities

with it. According to your chosen set of possible worlds and probability distribution, what is the probability of rolling either 3 or 5.

- (ii) Let $\mathcal{S}_B$ be defined as above, but with a loaded coin and loaded dice. A probability distribution is given in Table 2. What is the probability that the loaded coin lands “tails”? What is the conditional probability of rolling a 4, given that the coin lands tails? Which of the loaded dice has the highest chance of rolling 4 or more?

$t1$	$\frac{5}{18}$	$t6$	$\frac{1}{18}$
$t2$	$\frac{1}{9}$	$h1$	$\frac{1}{24}$
$t3$	$\frac{1}{9}$	$h2$	$\frac{1}{24}$
$t4$	$\frac{1}{18}$	$h3$	$\frac{1}{8}$
$t5$	$\frac{1}{18}$	$h4$	$\frac{1}{8}$

Table 2: Probabilities for $\mathcal{S}_B$.

Solution:

- (i) Probabilities for $\mathcal{S}_A$: $P_A(1) = \dots = P_A(4) = \frac{5}{24}$ and $P_A(5) = P_A(6) = \frac{1}{12}$.

Probabilities for $\mathcal{S}_B$: $P_B(t1) = \dots = P_B(t6) = \frac{1}{12}$ and $P_B(h1) = \dots = P_B(h4) = \frac{1}{8}$.

$$P_A(3) + P_A(5) = \frac{7}{24}.$$

$$P_B(t3) + P_B(t5) + P_B(h3) = \frac{7}{24}.$$

- (ii) $P(t) = \frac{5}{18} + \frac{1}{9} + \frac{1}{9} + \frac{1}{18} + \frac{1}{18} + \frac{1}{18} = \frac{2}{3}$.

$$P(4|t) = P(t4)/P(t) = \frac{1}{12}.$$

$$P(4|t) + P(5|t) + P(6|t) = \frac{1}{4}.$$

$$P(4|h) = P(h4)/P(h) = 1/8 / (\frac{1}{24} + \frac{1}{24} + \frac{1}{8} + \frac{1}{8}) = \frac{3}{8}.$$

The four-sided die thus has a higher probability of rolling 4 or more, than the six-sided die.

Exercise 8 (Optional):

Consider the binary variable A and the ternary variable B . Assume that B has the probability distribution $P(B) = (0.1, 0.5, 0.4)$ and that A has the conditional probability distribution given in Table 3.

	b_1	b_2	b_3
a_1	0.1	0.7	0.6
a_2	0.9	0.3	0.4

Table 3: The conditional probability distribution $P(A|B)$.

Questions:

- (i) Verify that Table 3 specifies a valid conditional probability distribution.
- (ii) Calculate $P(B|A)$. *Hint:* Consider Bayes rule illustrated by the temperature-sensor example that we discussed in the lecture

Solution:

1. We need to check that $\sum_A P(A|B = b_i) = 1$ for each state b_i of B . This is the case in Table 3, hence the we have a valid conditional probability distribution.
2. According to Bayes rule we have

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)} = \frac{P(A|B)P(B)}{\sum_B P(A, B)} = \frac{P(A|B)P(B)}{\sum_B P(A|B)P(B)}$$

Based on the probabilities given in the exercise, we have for the numerator that

$$P(A|B)P(B) = \begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.1 & 0.7 & 0.6 \\ a_2 & 0.9 & 0.3 & 0.4 \end{array} \cdot (0.1, 0.5, 0.4) = \begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.01 & 0.35 & 0.24 \\ a_2 & 0.09 & 0.15 & 0.16 \end{array}$$

Note that

- $P(A|B)P(B)$ is also equal to $P(A, B)$ (an instance of the *chain rule*!)
- the multiplications are done component-wise so that we only multiply entries that match wrt. the state labels.

For the denominator, we can use the intermediate result ($P(A, B)$) that we just calculated above:

$$P(A) = \sum_B P(A, B) = \sum_B \left(\begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.01 & 0.35 & 0.24 \\ a_2 & 0.09 & 0.15 & 0.16 \end{array} \right) = (0.6, 0.4).$$

Plugging these results into Bayes rule we get:

$$\begin{aligned} P(B|A) &= \begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.01 & 0.35 & 0.24 \\ a_2 & 0.09 & 0.15 & 0.16 \end{array} / (0.6, 0.4) \\ &= \begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.01/0.6 & 0.35/0.6 & 0.24/0.6 \\ a_2 & 0.09/0.4 & 0.15/0.4 & 0.16/0.4 \end{array} \\ &= \begin{array}{c|ccc} & b_1 & b_2 & b_3 \\ \hline a_1 & 0.0167 & 0.5833 & 0.4000 \\ a_2 & 0.2250 & 0.3750 & 0.4000 \end{array} \end{aligned}$$

Exercise 9 (Optional):

A routine DNA test is performed on a person (this exercise is set in the not too distant future!). The test T gives a positive result for a rare genetic mutation M linked to Alzheimer's disease. The mutation is present in only 1 in a million people. The test is 99.99% accurate, i.e. it will give a wrong result in 1 out of 10000

tests performed. Should the person be worried, i.e., what is the probability that the person has the mutation given that the test showed a positive result?

Hint: This is partly a modeling exercises and partly a calculation exercise. First you need to formalize the problem:

- What are the relevant variables and what states do they have?
- Based on the description above, what probability distributions can you infer for the variables?

Based on this formalization, you need to find the rules required to answer the question about the probability of a mutation given a positive test result.

Solution: We analyze the problem using the two random variables $Test \in \{positive, negative\}$, $Mutation \in \{present, absent\}$. What our patient should be interested in is the probability of having the mutation, given everything he/she knows, i.e. the fact that there was a positive test result. Thus, we need to compute

$$P(Mutation = present \mid Test = positive).$$

According to Bayes rule, this is equal to

$$P(Test = positive \mid Mutation = present) \frac{P(Mutation = present)}{P(Test = positive)}$$

Of the probabilities on the right, we have $P(Test = positive \mid Mutation = present) = 0.9999$, and $P(Mutation = present) = 0.000001$. We still need $P(Test = positive)$. This can be computed as

$$\begin{aligned} P(Test = positive) &= P(Test = positive, Mutation = present) \\ &\quad + P(Test = positive, Mutation = absent) \\ &= P(Test = positive \mid Mutation = present)P(Mutation = present) \\ &\quad + P(Test = positive \mid Mutation = absent)P(Mutation = absent) \\ &= 0.9999 \cdot 0.000001 + 0.0001 \cdot 0.999999 \sim 0.0001 \end{aligned}$$

We now get

$$P(Mutation = present \mid Test = positive) = 0.9999 \cdot \frac{0.000001}{0.0001} = 0.009999.$$

Thus, there is only about a 1% chance that the patient has the mutation.

Exercise 10 (Optional):

Assume it is your responsibility to monitor volcanic eruptions. You receive data from two different stations (seismometers), S_1 and S_2 . Each S_i is modeled as a Boolean variable where “true” stands for “I detected an eruption” and “false” stands for “I did not detect an eruption”. The seismometers are not fully reliable, however; they may not detect an eruption even though there was one, and they may mistake an earthquake for an eruption of a volcano. We model this situation with two additional Boolean variables: V for volcanic eruption, and E for Earthquake.

Use the algorithm from the lecture to construct a Bayesian network for these 4 variables. Do so for the following two variable orders:

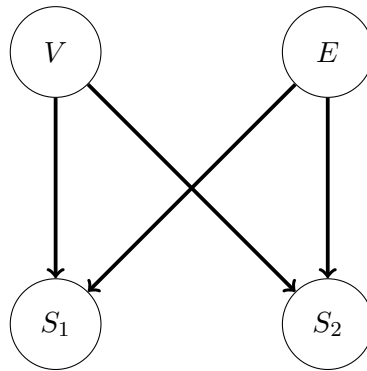
(a) $X_1 = V, X_2 = E, X_3 = S_1, X_4 = S_2$.

(b) $X_1 = S_1, X_2 = S_2, X_3 = E, X_4 = V$.

For each of these orders, draw the resulting Bayesian network. Justify your design, i.e., for each variable X_i added to the network explain why the set of parents you give X_i are needed, and why they are sufficient.

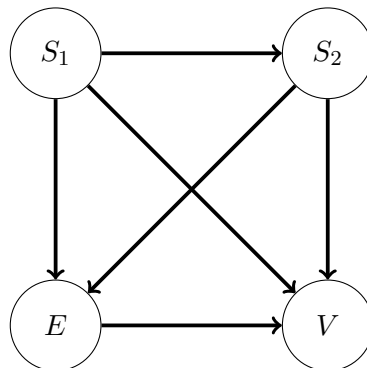
Solution:

(a) With this variable order, we get the following network:



$X_2 = E$ does not need $X_1 = V$ as a parent because Earthquakes are independent from volcano tests. $x_3 = S_1$ needs both $X_1 = V$ and $X_2 = E$ as parents because each of these may influence the measurement; same for $X_4 = S_2$, i.e., here we also need the parents $X_1 = V$ and $X_2 = E$. However, given the values of V and E , the measurements of $X_3 = S_1$ and $x_4 = S_2$ are independent. So $X_4 = S_2$ does not require the parent $X_3 = S_1$.

(b) With this variable order, we get the following network:



$X_2 = S_2$ needs $X_1 = S_1$ as a parent because, if S_1 detects a seismic phenomenon, then chances are higher S_2 will detect one as well. $X_3 = E$ needs each of $X_1 = S_1$ and $X_2 = S_2$ as parents because, if a station detects a seismic phenomenon, then chances are higher there was an earthquake; same for $X_4 = V$, i.e., here we also need the parents $X_1 = S_1$ and $X_2 = S_2$ because measurements

indicate volcano tests as well. Finally, say we already know that S_1 and S_2 are true; then the value of E still has an influence on the value of V : If there was an earthquake, then there is a chance that the seismic measurements were caused by the earthquake rather than a volcano test. Thus V is *not* conditionally independent of E given S_1 and S_2 , and we need $X_3 = E$ as a parent of $X_4 = V$ as well.

Exercise 11 (Optional):

Table 4 describes a test T for an event A . The number 0.01 is the frequency of *false negatives*, and the number 0.001 is the frequency of *false positives*.

- (i) The police can order a blood test on drivers under the suspicion of having consumed too much alcohol. The test has the above characteristics. Experience says that 20% of the drivers under suspicion do in fact drive with too much alcohol in their blood. A suspicious driver has a positive blood test. What is the probability that the driver is guilty of driving under the influence of alcohol?
- (ii) The police block a road, take blood samples of all drivers, and use the same test. It is estimated that one out of 1,000 drivers have too much alcohol in their blood. A driver has a positive test result. What is the probability that the driver is guilty of driving under the influence of alcohol?

	$A = \text{yes}$	$A = \text{no}$
$T = \text{yes}$	0.99	0.001
$T = \text{no}$	0.01	0.999

Table 4: Table for Exercise 11. Conditional probabilities $P(T | A)$ characterizing test T for A .

Solution: We use Bayes rule to calculate the probabilities.

$$\begin{aligned}
 P(A = y | T = p) &= \frac{P(A = y, T = p)}{P(T = p)} \\
 &= \frac{P(T = p | A = y)P(A = y)}{P(T = p)}
 \end{aligned}$$

From the problem specification, we have the numbers to put into the numerator, but we still need to calculate the denominator:

$$\begin{aligned}
 P(T = p) &= P(T = p, A = y) + P(T = p, A = n) \\
 &= P(T = p | A = y)P(A = y) + P(T = p | A = n)P(A = n)
 \end{aligned}$$

By plugging this into the previous expression we get:

$$\begin{aligned}
 P(A = y | T = p) &= \frac{P(T = p | A = y)P(A = y)}{P(T = p | A = y)P(A = y) + P(T = p | A = n)P(A = n)} \\
 &= \frac{0.99 \cdot 0.2}{0.99 \cdot 0.2 + 0.001 \cdot 0.8} \\
 &= 0.996
 \end{aligned}$$

In the second part of the exercise we change the prior probability $P(A)$ for A and update the calculations using the new probabilities. This gives $P(A = y) = 0.498$. Notice how the change in prior distribution affected the posterior distribution for A ; the accuracy of a test should always be considered relative to the frequency of the event which you are trying to predict.

Exercise 12 (Optional):

In Table 5, a joint probability table for the binary variables A , B , and C is given.

- Calculate $P(B, C)$ and $P(B)$.
- Are A and C independent given B ?

	b_1	b_2
a_1	(0.006, 0.054)	(0.048, 0.432)
a_2	(0.014, 0.126)	(0.032, 0.288)

Table 5: $P(A, B, C)$ for Exercise 12.

Solution:

For the first part of the exercise, consider calculating $P(B, c_1)$. To do that we marginalize out A , hence

$$P(B, c_1) = (0.006 + 0.014, 0.048 + 0.032) = (0.02, 0.08).$$

The remaining probabilities are calculated similarly, and we get:

$$(i) \ P(B, c_1) = (0.02, 0.08), P(B, c_2) = (0.18, 0.72), P(B) = (0.2, 0.8)$$

For the second part of the exercise we can check that

$$P(A|C = c_1, B) = P(A|C = c_2, B)$$

to verify that A and C are independent given B . These probabilities can be calculated as

$$P(A|B, C) = \frac{P(A, B, C)}{P(B, C)},$$

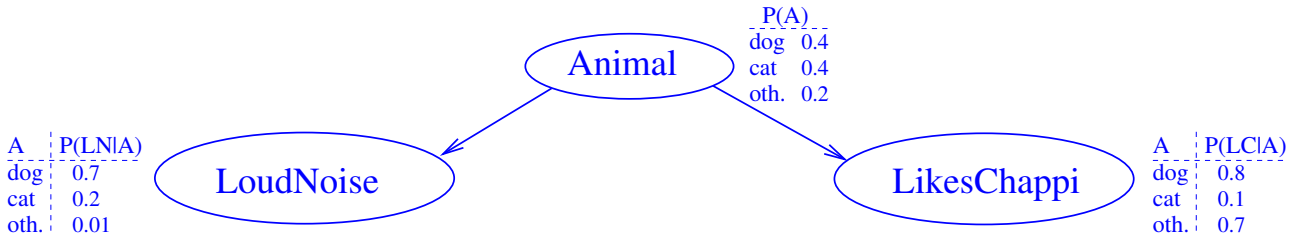
where the denominator was calculated in the first part of the exercise. When doing the calculations we get

$$(ii) \ P(A|b_1, c_1) = (0.3, 0.7) = P(A|b_1, c_2), P(A|b_2, c_1) = (0.6, 0.4) = P(A|b_2, c_2),$$

hence A and C are independent given B .

Exercise 1 :

Consider the following Bayesian network BN :



Use inference by enumeration to compute the following probabilities:

- (a) $P(dog \mid loudnoise, likeschappi)$.
- (b) $P(loudnoise \mid \neg likeschappi)$.

Include intermediate steps at a level of granularity as in the lecture slides examples. In particular, for each of (a) and (b), state what the query variable, evidence, and hidden variables are; and write down *which* probabilities provided in BN can be combined *how* to obtain the demanded probability P .

Solution:

As the variable ordering consistent with BN , we choose $X_1 = Animal$, $X_2 = LoudNoise$, $X_3 = LikesChappi$.

- (a) The query variable X here is $Animal$. The evidence $\mathbf{e}$ is $loudnoise, likeschappi$. There are no hidden variables, $\mathbf{Y} = \emptyset$. Using Normalization+Marginalization $\mathbf{P}(X \mid \mathbf{e}) = \alpha \sum_{\mathbf{y} \in \mathbf{Y}} \mathbf{P}(X, \mathbf{e}, \mathbf{y})$ we get $\mathbf{P}(Animal \mid loudnoise, likeschappi) = \alpha \mathbf{P}(Animal, loudnoise, likeschappi)$. By the Chain rule and exploiting conditional independence, we get $\alpha \mathbf{P}(Animal, loudnoise, likeschappi) = \alpha \mathbf{P}(likeschappi \mid Animal) * \mathbf{P}(loudnoise \mid Animal) * \mathbf{P}(Animal) = \alpha \langle 0.8 * 0.7 * 0.4, 0.1 * 0.2 * 0.4, 0.7 * 0.01 * 0.2 \rangle \approx \langle 0.96, 0.03, 0.01 \rangle$. Thus $P(dog \mid loudnoise, likeschappi) \approx 0.96$. (“If your animal likes Chappi and makes loud noise, chances are good it’s a dog.”)
- (b) The query variable X here is $LoudNoise$. The evidence $\mathbf{e}$ is $\neg likeschappi$. The hidden variables $\mathbf{Y}$ are $\{Animal\}$. Using Normalization+Marginalization $\mathbf{P}(X \mid \mathbf{e}) = \alpha \sum_{\mathbf{y} \in \mathbf{Y}} \mathbf{P}(X, \mathbf{e}, \mathbf{y})$ we get $\mathbf{P}(LoudNoise \mid \neg likeschappi) = \alpha \sum_{\mathbf{y} \in \{Animal\}} \mathbf{P}(\mathbf{y}, LoudNoise, \neg likeschappi)$. By the Chain rule and exploiting conditional independence, we get $\alpha \sum_{\mathbf{y} \in \{Animal\}} \mathbf{P}(\mathbf{y}, LoudNoise, \neg likeschappi) = \alpha \sum_{\mathbf{y} \in \{Animal\}} P(\neg likeschappi \mid \mathbf{y}) * \mathbf{P}(LoudNoise \mid \mathbf{y}) * P(\mathbf{y})$. That is, for each truth value v of $LoudNoise$, we need to sum over each possible animal $\mathbf{y}$ the product of the likelihood of $\mathbf{y}$ not liking Chappi multiplied with the likelihood of $\mathbf{y}$ having value v for $LoudNoise$ multiplied with the likelihood of $\mathbf{y}$. This gives us $\mathbf{P}(LoudNoise \mid \neg likeschappi) = \alpha \langle 0.2 * 0.7 * 0.4 + 0.9 * 0.2 * 0.4 + 0.3 * 0.01 * 0.2, 0.2 * 0.3 * 0.4 + 0.9 * 0.8 * 0.4 + 0.3 * 0.99 * 0.2 \rangle = \alpha \langle 0.1286, 0.3714 \rangle = \langle 0.2572, 0.7428 \rangle$. Thus $P(loudnoise \mid \neg likeschappi) = 0.2572$. (“If your animal does not like Chappi, chances are good it is quiet.”)

Exercise 2 :

In the graphs in Figures 1 and 2, determine which variables are d-separated from A . Note that it is sufficient to find a single open path along which evidence can be transmitted; if such a path exists then the variables are *not* d-separated (instead they are said to be d-connected).

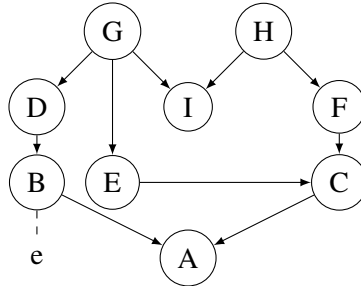


Figure 1: Figure for Exercise 2.

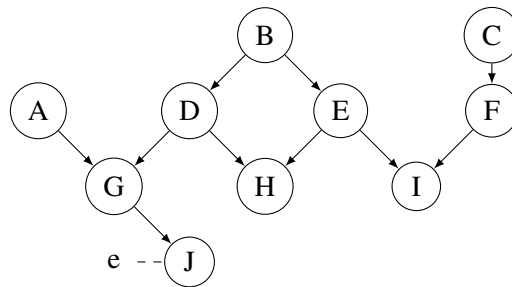


Figure 2: Figure for Exercise 2.

Solution:

In (a), all variables are d-connected to A . We can see that by checking whether there *exists* an open path between each pair of variables. Take A and D for instance. The path $A \leftarrow C \leftarrow F \leftarrow H \rightarrow I \leftarrow G \rightarrow D$ is closed, since it contains the connection $(H \rightarrow I \leftarrow G)$, which is converging and there is no evidence on I (nor on any of its descendants for which it has none). However, the path $A \leftarrow C \leftarrow E \leftarrow G \rightarrow D$ is open since each triplet of nodes along the path (i.e., $(A \leftarrow C \leftarrow E)$, $(C \leftarrow E \leftarrow G)$, $(E \leftarrow G \rightarrow D)$) defines an open connection according to the d-separation rules. With this path we have established the existence of an open path, and we therefore have that A and D are not d-separated given the evidence provided.

In (b), all variables except C and F are d-connected to A .

Exercise 3 :

Consider the network in Figure 3.

- What are the minimal set(s) of variables that we should have evidence on in order to d-separate C and E (that is, sets of variables for which no proper subset d-separates C and E)?

- What are the minimal set(s) of variables we should have evidence on in order to d-separate A and B ?
- What are the maximal set(s) of variables that we can have evidence on and still d-separate C and E (that is, sets of variables for which no proper superset d-separates C and E)?
- What are the maximal set(s) of variables that we can have evidence on and still d-separate A and B ?

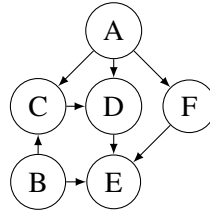


Figure 3: A causal network for Exercise ??.

Solution:

Minimal sets d-separating C and E : $\{B, D, F\}$ and $\{A, B, D\}$.

Minimal sets d-separating A and B : $\emptyset$.

Maximal set d-separating C and E : $\{A, B, D, F\}$.

Maximal set d-separating A and B : $\{F\}$.

Exercise 4 :

Consider the network defined by the two binary variables A and B , where A is the parent of B . Assume that the conditional probability tables are given as $P(A) = (0.1, 0.9)$ and

	A	
	a_1	a_2
b_1	0.05	0.2
b_2	0.95	0.8

- Assume that you want to estimate $P(b_1)$ using sampling. How many samples would be required if you only accept an error larger than 0.15 in 10% of the cases?
- Generate two random samples, using the following list of random numbers (any time that you use a random number pick one from the list): 0.5, 0.3, 0.01, 0.8.
- Suppose that after sampling 10 states, you got $(A = a_2, B = b_2)$ 3 times, and $(A = a_1, B = b_2)$ 6 times, and $(A = a_1, B = b_1)$ once. Estimate $P(B = b_1)$
- (Optional) Implement the network above in Hugin and use Hugin to sample the number of cases that you calculated in step (i); use the function 'Simulate cases' under 'File'. Use the sampled cases to estimate $P(b_1)$ and compare the result with Hugin. Feel free to use a spreadsheet for the counting.
- Assume that you want to use rejection sampling to estimate $P(A|B = b_1)$. How many samples do you expect you would have to generate in order to end up (after rejection) with a sample set of 1000 cases for estimating the probability.

Solution:

i) Hoeffding's inequality gives us

$$P(|s - p| > 0.15) \leq 2e^{-2n0.15^2} < 0.1,$$

which we can rewrite to find that $n > 66.57$.

ii) As A is parent of B , we always sample A first.

First state: $A = a_2, B = b_2$: $0.5 > 0.1$, so $A = a_2$, then $0.3 > 0.2$ so $B = b_2$. Second state: $A = a_1, B = b_2$: $0.01 \leq 0.1$, so $A = a_1$, then $0.8 > 0.05$ so $B = b_2$.

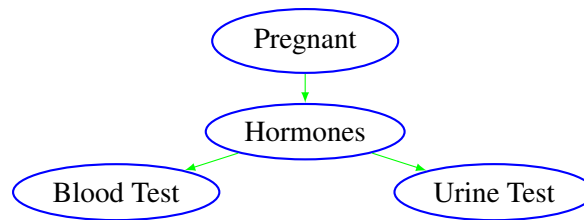
iii) $P(B = b_1) = 0.1$

iv) (Must be done in Hugin)

v) The probability of $B = b_1$ is $P(B = b_1) = 0.185$, hence only 18.5% of all the generated samples would not be rejected. We therefore need to generate approximately 5400 cases to end up with a sample set of 1000 cases.

Exercise 5 :

Consider the insemination example from Section 3.1.13 in BNDG:



Let the probabilities be as in Table 1 ($Ho = y$ means that hormonal changes have taken place) $P(Pr=y) = 0.87$ and $P(Pr=n) = 0.13$.

	$Pr = y$	$Pr = n$		$Ho = y$	$Ho = n$
$Ho = y$	0.9	0.01	$BT = y$	0.7	0.1
$Ho = n$	0.1	0.99	$BT = n$	0.3	0.9

	$Ho = y$	$Ho = n$
$UT = y$	0.8	0.1
$UT = n$	0.2	0.9

Table 1: Tables for Exercise 5.

i) What is $P(Pr | BT = n, UT = n)$?

ii) (Optional) Verify your solution modelling the BN in Hugin.

Solution: Part 1:

$$P(Pr|Bt = n, Ut = n) = \frac{P(Pr, Bt = n, Ut = n)}{P(Bt = n, Ut = n)}$$

Next, we should calculate $P(Pr, Bt = n, Ut = n)$ and $P(Bt = n, Ut = n)$:

$$P(Pr, Bt = n, Ut = n) = \sum_{Ho} P(Pr, Bt = n, Ut = n, Ho)$$

$$P(Bt = n, Ut = n) = \sum_{Pr} P(Pr, Bt = n, Ut = n)$$

Thus, we only need to calculate $P(Pr, Bt = n, Ut = n, Ho)$ and this can be done using the chain rule:

$$P(Pr, Bt = n, Ut = n, Ho) = P(Ut = n|Ho)P(Bt = n|Ho)P(Ho|Pr)P(Pr)$$

The final value is $P(Pr|Bt = n, Ut = n) = (0.53, 0.47)$. See also the [Hugin network](#).

Exercise 6 :

Complete Exercise 9.4(a-b) in PM.

Solution:

We have two variables in the network: *Company* (C) with states *green* (g) and *blue* (b) and *Witness* (W) with states *green* (g) and *blue* (b). The structure of the network incorporating one witness is illustrated in Figure 4(a).

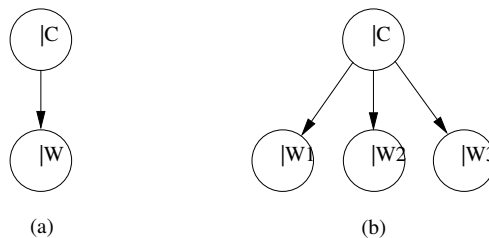


Figure 4: The witness model: Model (a) includes one witness and model (b) includes three witnesses.

The conditional probability tables for the model are given by:

$\frac{P(C = g)}{0.85}$	$\frac{P(C = b)}{0.15}$		$C = g$	$C = b$
		$W = g$	0.8	0.2
		$W = b$	0.2	0.8
		$P(W C)$		

For calculating $P(C = b | W = b)$ we use Bayes rule:

$$\begin{aligned}
 P(C = b | W = b) &= \frac{P(W = b | C = b)P(C = b)}{P(W = b)} \\
 &= \frac{P(W = b | C = b)P(C = b)}{P(C = b, W = b) + P(C = g, W = b)} \\
 &= \frac{0.8 \cdot 0.15}{0.8 \cdot 0.15 + 0.2 \cdot 0.85} \\
 &= \frac{0.12}{0.12 + 0.17} \approx 0.41
 \end{aligned}$$

With three independent witnesses we have the network in Figure 4(b), where we assign the same conditional probability distribution to each of the three witnesses. Thus, we have a naive Bayes model with three information variables.

We can perform probability updating in this model as shown on the slides in the previous lecture. E.g.

$$\begin{aligned}
 P(C = b | W_1 = W_2 = W_3 = b) &= \frac{P(W_1 = b | C = b)P(W_2 = b | C = b)P(W_3 = b | C = b)P(C = b)}{P(W_1 = b, W_2 = b, W_3 = b)} \\
 &= \frac{0.8 \cdot 0.8 \cdot 0.8 \cdot 0.15}{0.8 \cdot 0.8 \cdot 0.8 \cdot 0.15 + 0.2 \cdot 0.2 \cdot 0.2 \cdot 0.85} = 0.92
 \end{aligned}$$

Exercise 7 :

Consider the running example of the lecture with the following probability tables:

Burglary	
t	f
.1	.9

Earthquake	
t	f
.1	.9

Burglary	Earthquake	Alarm	
		t	f
t	t	.9	.1
t	f	.8	.2
f	t	.5	.5
f	f	.1	.9

Alarm	JohnCalls	
	t	f
t	.8	.2
f	.1	.9

Alarm	MaryCalls	
	t	f
t	.7	.3
f	.1	.9

Determine the conditional probability $P(MC | B = t)$. For each step indicate the operations that are performed.

Solution:

Note: Below you have the solution using Variable Elimination, but the exercise can also be solved by Naive Enumeration and the result should be the same.

$$\begin{aligned}
 & \sum_{a \in \{t, f\}} \sum_{eq \in \{t, f\}} \sum_{jc \in \{t, f\}} P(B = t)P(EQ = eq)P(A = a \mid B = t, EQ = eq)P(JC = jc \mid A = a)P(MC \mid A = a) = \\
 & \sum_{a \in \{t, f\}} \sum_{eq \in \{t, f\}} P(B = t)P(EQ = eq)P(A = a \mid B = t, EQ = eq)P(MC \mid A = a)F_1(a) = \\
 & \sum_{a \in \{t, f\}} P(B = t)P(MC \mid A = a)F_1(a)F_2(a) = \\
 & P(B = t)F_3(MC)
 \end{aligned}$$

With the concrete tables we have specified for the conditional distributions, we can now compute: to eliminate the variable JohnCalls, we multiply all tables that contain the variable JohnCalls, and sum out the JohnCalls variable. The result is the table, or factor, F_1 . Since there is only one table containing JohnCalls, the result is very simple:

	$F_1(Alarm)$	
	t	f
	1	1

Next we eliminate *Earthquake*. There are two tables containing Earthquake: the table of the Earthquake node, and the conditional distribution of Alarm. The latter table is restricted to the cases that are consistent with the observed evidence $B = t$, which gives a table only containing Earthquake and Alarm:

Earthquake	Alarm	
	t	f
t	.9	.1
f	.8	.2

Multiplying this with the Earthquake table gives

Earthquake	Alarm	
	t	f
t	.09	.01
f	.72	.18

Summing out the Earthquake variable then gives the factor F_2 :

	$F_2(Alarm)$	
	t	f
	0.81	0.19

Finally, we eliminate *Alarm*. Multiplying F_1, F_2 , and the table for *MaryCalls* gives

Alarm	MaryCalls	
	t	f
t	$.7 \cdot 0.81 = 0.567$	$.3 \cdot 0.81 = 0.243$
f	$.1 \cdot 0.19 = 0.019$	$.9 \cdot 0.19 = 0.171$

Summing out *Alarm* gives

$F_3(\text{MaryCalls})$	
t	f
0.586	0.414

This multiplied with $0.1 = P(B = t)$ gives

$P(B = t)F_3(\text{MaryCalls})$	
t	f
0.0586	0.0414

which is now the table containing the function $P(B = t, MC)$. To obtain the conditional distribution $P(MC \mid B = t)$ this has only to be normalized by dividing with $0.0586 + 0.0414$, which gives

$P(\text{MaryCalls} \mid B = t)$	
t	f
0.586	0.414

Note that by inspecting the semantics of the potentials calculated above, we could have stopped after summing out *Alarm*, since the resulting potential is the conditional distribution $P(MC \mid B = t)$.

Exercise 1 :

Consider that we have the following data-points, which we want to fit using linear regression:

x_1	x_2	y
1	1	2
0	0	0
1	0	3
2	1	2

What is the Mean Squared Error (MSE) error for the following sets of parameters:

- i) $w_0 = 0, w_1 = 1, w_2 = -1$
- ii) $w_0 = 1, w_1 = 1, w_2 = 0$

Which model is preferable according to the MSE error function?

Solution:

	x_1	x_2	y	$\hat{y}$
i) First, we compute the value of $\hat{y}$	1	1	2	0
	0	0	0	0
	1	0	3	1
	2	1	2	1

The error is then: $SSE = 2^2 + 0^2 + 2^2 + 1^2 = 9$ and $MSE = \frac{9}{4}$

	x_1	x_2	y	$\hat{y}$
ii)	1	1	2	2
	0	0	0	1
	1	0	3	2
	2	1	2	3

The error is then: $SSE = 0^2 + 1^2 + 1^2 + 1^2 = 3$ and $MSE = \frac{3}{4}$

Therefore, it is preferable the second model, as it has less error

Exercise 2 :

Consider a database of cars represented by the five training examples below. The target attribute *Acceptable*, which can have values *yes* and *no*, is to be predicted based on the other attributes of the car in question. These attributes indicate a) the age of the car (*Age* having values < 5 years and ≥ 5 years), b) the make of the car (*Make* having states *Toyota* and *Mazda*), c) the number of previous owners (*#Owners* having values 1, 2 and 3), d) the number of kilometers (*#Kilometers* having values $> 150k$ and $\leq 150k$) and e) the number of doors (*#Doors* having values 3 and 5).

	Attributes					Target
	Age	Make	#Owners	#Kilometers	#Doors	Acceptable
1	< 5	Mazda	1	> 150k	3	yes
2	≥ 5	Mazda	3	> 150k	3	no
3	≥ 5	Toyota	1	≤ 150k	3	no
4	≥ 5	Mazda	3	> 150k	5	yes
5	≥ 5	Toyota	2	≤ 150k	5	yes

- Calculate the entropy for the attribute #Owners.¹²
- Show the decision/classification tree that would be learned by the learning algorithm assuming that it is given the training examples in the database.
- Show the value of the information gain for each candidate attribute at each step in the construction of the tree.

Solution: For question (a):

$$ENT(\#Owners) = -\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{1}{5} \log_2 \left(\frac{1}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) = 1.521 \quad (1)$$

For question (b) we need to calculate the information gain for each of the features. That is, for the generic feature X we should calculate

$$Gain(X) = Ent(\text{Accept}) - ExpectedEntropy(\text{Accept}|X),$$

where $ExpectedEntropy(\text{Accept}|X)$ is the expected entropy of *Accept* wrt. X .

For $Ent(\text{Accept})$ we get

$$Ent(\text{Accept}) = -\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) = 0.971$$

Considering now the features in sequence from left to right, we have that the expected entropy of *Age* is

$$\begin{aligned} ExpectedEntropy(\text{Accept}|Age) &= P(\text{Age} < 5)Ent(\text{Accept}|Age < 5) + P(\text{Age} \geq 5)Ent(\text{Accept}|Age \geq 5) \\ &= \frac{1}{5}Ent(\text{Accept}|Age < 5) + \frac{4}{5}Ent(\text{Accept}|Age \geq 5). \end{aligned}$$

Here $Ent(\text{Accept}|Age < 5)$ and $Ent(\text{Accept}|Age \geq 5)$ denote the entropy of *Accept* when only considering the instances restricted to $Age < 5$ and $Age \geq 5$, respectively. Thus we get

$$\begin{aligned} Ent(\text{Accept}|Age < 5) &= -\frac{1}{1} \log_2 \left(\frac{1}{1} \right) - \frac{0}{1} \log_2 \left(\frac{0}{1} \right) = 0 \\ Ent(\text{Accept}|Age \geq 5) &= -\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) = 1 \end{aligned}$$

¹Note that $\log_2(x) = \frac{\log_{10}(x)}{\log_{10}(2)}$.

²Note that part a) asks for Entropy(Owners), which is not the same as Entropy (Target | Owners) that is used by the learning algorithm.

Plugging these values into the expression for the expected entropy above, we get

$$ExpectedEntropy(Accept|Age) = \frac{1}{5} \cdot 0 + \frac{4}{5} \cdot 1 = \frac{4}{5}$$

and the information gain therefore becomes

$$Gain(Age) = 0.971 - \frac{4}{5} = 0.171.$$

Doing the same calculations for the remaining attributes we end up with

$$Gain(Make) = 0.0202$$

$$Gain(\#Owners) = 0.171$$

$$Gain(\#Kilo) = 0.0202$$

$$Gain(\#Doors) = 0.4202$$

Since #Doors has the highest information gain we put that feature at the top of the tree.

For the branch of the tree with #Doors = 5 there is nothing more to do, since both of the training examples with this feature value has the same value for *Accept*, namely *yes*. For #Doors = 3, we have two instances with *Accept* being *yes* and one instance with *Accept* being *no*. Thus, we need to make another feature test for instances where #Doors = 3. To do that we proceed along the same lines as above, expect that we now only consider the instances consistent with #Doors = 3. For this restricted data set we find the entropy for *Accept*:

$$Ent(Accept|\#Doors = 3) = -\frac{1}{3} \log_2 \left(\frac{1}{3} \right) - \frac{2}{3} \log_2 \left(\frac{2}{3} \right) = 0.918$$

Based on this value we can calculate the information gain for each of the remaining features (this involves calculating the expected entropy for these features using the same calculation scheme as above):

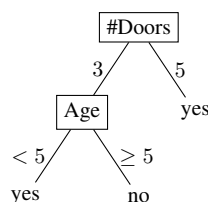
$$Gain(Age) = 0.918 - 0$$

$$Gain(Make) = 0.918 - 2/3$$

$$Gain(\#Owners) = 0.918 - 2/3$$

$$Gain(\#Kilo) = 0.918 - 2/3$$

Hence, for #Doors= 3 we pick *Age* as the next node. The classification tree can now directly be constructed based on the results above:



Exercise 3 :

Solve Exercise 7.4 (except sub-question f) in PM.

Solution:

1. (a) The optimal decision tree with one node predicts Likes = false. It has 5 errors.
2. (b) The optimal prediction is to predict likes with probability $5/12$. It has sum-of- squares error $5 \cdot (7/12)^2 + 7 \cdot (5/12)^2 = 2.92$.
3. (c) The optimal (with respect to sum of absolute errors) decision tree of depth 2 is: *if lawyers then Likes=true else Likes=false*. It has 3 errors. At the root are all of the examples $(e_1, \dots, e_{13})$. Filtered to the *lawyers = true* node are $e_2, e_3, e_4, e_8, e_9, e_{10}$. Filtered to the *lawyers = false* node are $e_1, e_5, e_6, e_7, e_{11}, e_{12}$.
4. (d) The optimal (with respect to sum-of-squares error) decision tree of depth 2 is: *if lawyers then likes with probability 2/3 else likes with probability 1/6* The error is $4 \cdot (1/3)^2 + 2 \cdot (2/3)^2 + 5 \cdot (1/6)^2 + 1 \cdot (5/6)^2 = 2.16$.
5. (e) The smallest tree that correctly classifies all training examples is: *if guns then lawyers else comedy* The information gain split gives a more complicated tree that represents the same function. To construct the tree follow the same procedure as for the preceding exercise.
6. (g) It is not linearly separable. The examples e_3 and e_{10} must be on the same side of a hyperplane (as they are both true on Likes). Therefore any linear interpolation also must be on the same side, but e_4 is between these in the input categories, but has a different classification.

Exercise 4 :

Given below is a trainings data set about esoteric programming languages. We want to predict whether people are willing to learn the languages depending on Form, Usefulness, and Torment.

ID	Usefulness	Form	Torment	Appealing
1	useful	Other	pleasant	No
2	useless	Text	torture	No
3	useless	Text	torture	No
4	useless	Images	pleasant	Yes
5	useful	Images	pleasant	Yes
6	useful	Other	torture	No
7	useless	Images	torture	Yes
8	useless	Text	pleasant	Yes
9	useless	Text	pleasant	Yes
10	useful	Other	torture	No

- (a) Compute the information gain associated with choosing the attribute Form as root.
- (b) Draw a decision tree with Form as root.

(c) Use your decision tree to predict the following records:

Usefulness	Form	Torment
useful	Other	pleasant
useless	Text	torture

Solution: Solution 0:

(a)

$$\text{Gain}(A) = B\left(\frac{p}{p+n}\right) - \text{Remainder}(A)$$

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p+n} B\left(\frac{p_k}{p_k + n_k}\right)$$

$$B(q) = -q \log_2 q - (1-q) \log_2 (1-q)$$

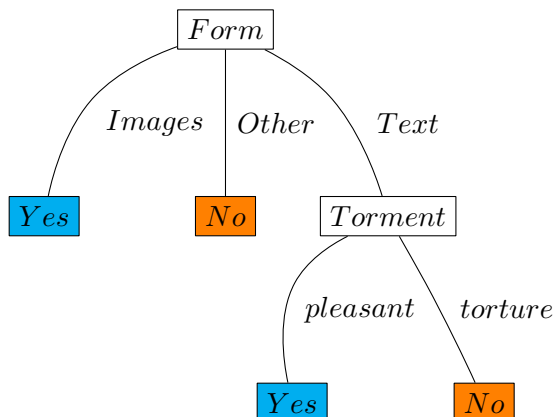
We know for Form:

Form	#Yes	#No	Σ
Other	0	3	3
Text	2	2	4
Images	3	0	3

$$\begin{aligned} \text{Remainder}(\text{Form}) &= \frac{3}{10} B\left(\frac{0}{3}\right) + \frac{4}{10} B\left(\frac{2}{4}\right) + \frac{3}{10} B\left(\frac{3}{3}\right) \\ &= \frac{2}{5} \end{aligned}$$

$$\begin{aligned} \text{Gain}(\text{Form}) &= B\left(\frac{5}{10}\right) - \text{Remainder}(\text{Form}) \\ &= \frac{3}{5} \end{aligned}$$

(b)



(c)

Usefulness	Form	Torment	Appealing
useful	Other	pleasant	No
useless	Text	torture	No

Exercise 5 :

Consider a fair six-sided die and another loaded die such that the probability of obtaining an outcome of 6 is 50%, the probability of obtaining 5 is 25% and the remaining outcomes are equally likely.

Calculate the entropy for both dice. How do the results differ? Explain!

Solution: Fair die:

$$P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = \frac{1}{6}$$

$$H_{fair} = \sum_1^6 -\frac{1}{6} \cdot \log_2 \left(\frac{1}{6} \right) \approx 2.58$$

Loaded die:

$$P(6) = \frac{1}{2}, \quad P(5) = \frac{1}{4}, \quad P(1) = P(2) = P(3) = P(4) = \frac{1}{16}$$

$$H_{loaded} = -\frac{1}{2} \cdot \log_2 \left(\frac{1}{2} \right) - \frac{1}{4} \cdot \log_2 \left(\frac{1}{4} \right) - 4 \cdot \left(\frac{1}{16} \cdot \log_2 \left(\frac{1}{16} \right) \right) = 2$$

Entropy is a measure of disorder. Since the outcome 6 or 5 is more certain for the loaded die the entropy is lower than for the fair die.

Exercise 6 :

Harry wants to invite his friends to a Pizza-Party. He does not want to ask them which pizza preferences they have in order to keep the party secret. He asks other people to get an impression what pizza preferences most people have.

1. Use the Decision-Tree-Learning algorithm from the lecture to build up a decision tree for his observations depicted in the table below. Make sure to show your calculations. For each step, begin with the calculation of the entropy, and then after that, calculate the gain of each ingredient, calculating them in the same order as they are presented in the table (*Pineapple* $\rightarrow$ *Mushrooms* $\rightarrow$ *Ham* $\rightarrow$ *Sweetcorn*). Explain which variable you take in each step and why. Draw the resulting Decision-Tree
2. Harry gets bored of all the calculations and decides to just pick variables at random. Why is this a bad idea. Specify what the resulting problem could be, and what the Decision-Tree-Learning algorithm tries to do.

Pineapple	Mushrooms	Ham	Sweetcorn	Rating
1	1	1	1	true
1	0	1	0	false
1	1	0	1	false
1	1	1	0	false
0	1	0	1	true
0	0	1	1	true
0	1	1	1	true
0	0	0	1	false
0	1	1	0	true
0	1	0	1	true

Solution: Solution:

1. We use the following abbreviations for the ingredients: *P* is Pineapple, *M* is Mushrooms, *H* is Ham, *S* is Sweetcorn.
Using Decision-Tree-Learning algorithm from the lecture notes.

Step 1

$$p = 6, n = 4 \Rightarrow B\left(\frac{6}{6+4}\right) = 0.97 = B(pos)$$

Pineapple

$$E_1 : \quad p = 1, n = 3 \\ \Rightarrow B\left(\frac{1}{1+3}\right) = 0.81$$

$$E_0 : \quad p = 5, n = 1 \\ \Rightarrow B\left(\frac{5}{6}\right) = 0.65$$

$$\Rightarrow R(P) = \frac{4}{10} * 0.81 + \frac{6}{10} * 0.65 = 0.71$$

$$\Rightarrow G(P) = B(Pos) - R(P) = 0.26$$

Mushrooms

$$E_1 : \quad p = 5, n = 2$$

$$\Rightarrow B\left(\frac{5}{7}\right) = 0.86$$

$$E_0 : \quad p = 1, n = 2$$

$$\Rightarrow B\left(\frac{1}{3}\right) = 0.92$$

$$\Rightarrow R(M) = \frac{7}{10} * 0.86 + \frac{3}{10} * 0.92 = 0.88$$

$$\Rightarrow G(M) = B(Pos) - R(M) = 0.09$$

Ham

$$E_1 : \quad p = 4, n = 2$$

$$\Rightarrow B\left(\frac{4}{6}\right) = 0.92$$

$$E_0 : \quad p = 2, n = 2$$

$$\Rightarrow B\left(\frac{2}{4}\right) = 1$$

$$\Rightarrow R(H) = \frac{6}{10} * 0.92 + \frac{4}{10} * 1 = 0.95$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 0.02$$

Sweetcorn

$$E_1 : \quad p = 5, n = 2$$

$$\Rightarrow B\left(\frac{5}{7}\right) = 0.86$$

$$E_0 : \quad p = 1, n = 2$$

$$\Rightarrow B\left(\frac{1}{3}\right) = 0.92$$

$$\Rightarrow R(S) = \frac{7}{10} * 0.86 + \frac{3}{10} * 0.92 = 0.88$$

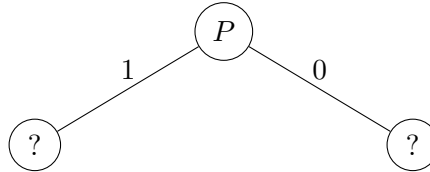
$$\Rightarrow G(S) = B(Pos) - R(S) = 0.09$$

Result of step 1

$$G(P) > G(M) = G(S) > G(H)$$

$$0.26 > 0.09 = 0.09 > 0.02$$

Choose the ingredient with the highest information. As a result, we have Pineapple as our first node in the decision tree:



Step 2 - Investigation of the right side

$$p = 5, n = 1 \Rightarrow B\left(\frac{5}{5+1}\right) = 0.65 = B(pos)$$

Mushrooms

$$E_1 : \quad p = 4, n = 0 \\ \Rightarrow B\left(\frac{4}{4}\right) = 0$$

$$E_0 : \quad p = 1, n = 1 \\ \Rightarrow B\left(\frac{1}{2}\right) = 1$$

$$\Rightarrow R(M) = \frac{4}{6} * 0 + \frac{2}{6} * 1 = 0.33$$

$$\Rightarrow G(M) = B(Pos) - R(M) = 0.32$$

Ham

$$E_1 : \quad p = 3, n = 0 \\ \Rightarrow B\left(\frac{3}{3}\right) = 0$$

$$E_0 : \quad p = 2, n = 1 \\ \Rightarrow B\left(\frac{2}{3}\right) = 0.92$$

$$\Rightarrow R(H) = \frac{3}{6} * 0 + \frac{3}{6} * 0.92 = 0.46$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 0.19$$

Sweetcorn

$$E_1 : \quad p = 4, n = 1$$

$$\Rightarrow B\left(\frac{4}{5}\right) = 0.72$$

$$E_0 : \quad p = 1, n = 0$$

$$\Rightarrow B\left(\frac{1}{1}\right) = 0$$

$$\Rightarrow R(S) = \frac{5}{6} * 0.72 + \frac{1}{6} * 0 = 0.6$$

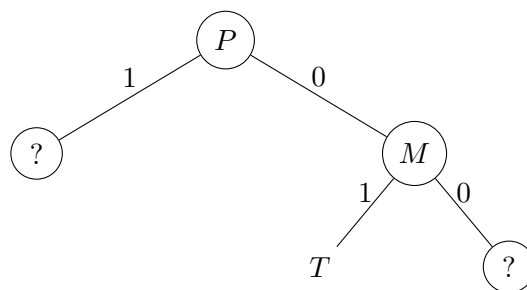
$$\Rightarrow G(S) = B(Pos) - R(S) = 0.05$$

Result of step 2

$$G(M) > G(H) > G(S)$$

$$0.32 > 0.19 > 0.05$$

Choose the ingredient with the highest information. As a result, we have Mushrooms as next Node:

**Step 3**

$$p = 1, n = 1 \Rightarrow B\left(\frac{1}{1+1}\right) = 1 = B(pos)$$

Only Ham and Sweetcorn remain as attributes.

Ham

$$E_1 : \quad p = 1, n = 0$$

$$\Rightarrow B\left(\frac{1}{1}\right) = 0$$

$$E_0 : \quad p = 0, n = 1$$

$$\Rightarrow B\left(\frac{0}{1}\right) = 0$$

$$\Rightarrow R(H) = \frac{1}{2} * 0 + \frac{1}{2} * 0 = 0$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 1$$

Sweetcorn

$$E_1 : \quad p = 1, n = 1$$

$$\Rightarrow B\left(\frac{1}{2}\right) = 1$$

$$E_0 : \quad p = 0, n = 0$$

$$\Rightarrow B\left(\frac{0}{0}\right) = 0$$

$$\Rightarrow R(S) = \frac{2}{2} * 1 + \frac{0}{2} * 0 = 1$$

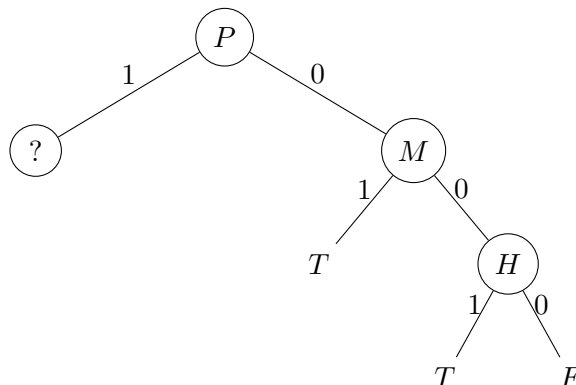
$$\Rightarrow G(S) = B(Pos) - R(S) = 0$$

Result of step 3

$$G(H) > G(S)$$

$$1 > 0$$

Choose the ingredient with the highest information. As a result, we have Ham as next Node:



Step 4 - Investigation of the left side

Only Mushroom, Ham and Sweetcorn remain as attributes.

$$p = 1, n = 3 \Rightarrow B\left(\frac{1}{1+3}\right) = 0.81 = B(pos)$$

Mushrooms

$$\begin{aligned} E_1 : \quad p = 1, n = 2 \\ \Rightarrow B\left(\frac{1}{3}\right) = 0.91 \end{aligned}$$

$$\begin{aligned} E_0 : \quad p = 0, n = 1 \\ \Rightarrow B\left(\frac{0}{1}\right) = 0 \end{aligned}$$

$$\Rightarrow R(H) = \frac{3}{4} * 0.91 + \frac{1}{4} * 0 = 0.68$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 0.13$$

Ham

$$\begin{aligned} E_1 : \quad p = 1, n = 2 \\ \Rightarrow B\left(\frac{1}{3}\right) = 0.91 \end{aligned}$$

$$\begin{aligned} E_0 : \quad p = 0, n = 1 \\ \Rightarrow B\left(\frac{0}{1}\right) = 0 \end{aligned}$$

$$\Rightarrow R(H) = \frac{3}{4} * 0.91 + \frac{1}{4} * 0 = 0.68$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 0.13$$

Sweetcorn

$$E_1 : \quad p = 1, n = 1$$

$$\Rightarrow B\left(\frac{1}{2}\right) = 1$$

$$E_0 : \quad p = 0, n = 2$$

$$\Rightarrow B\left(\frac{0}{2}\right) = 0$$

$$\Rightarrow R(S) = \frac{2}{4} * 1 + \frac{2}{4} * 0 = 0.5$$

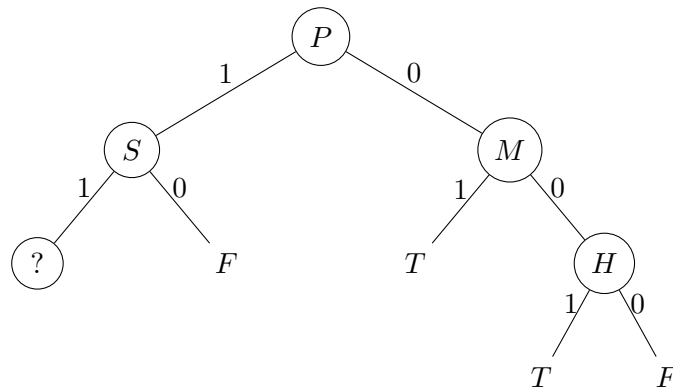
$$\Rightarrow G(S) = B(Pos) - R(S) = 0.31$$

Result of step 4

$$G(S) > G(M) = G(H)$$

$$0.31 > 0.13 = 0.13$$

Choose the ingredient with the highest information. As a result, we have Sweetcorn as next Node:



Step 5

Only Ham and Mushrooms remain as attributes.

$$p = 1, n = 1 \Rightarrow B\left(\frac{1}{1+1}\right) = 1 = B(pos)$$

Mushrooms

$$E_1 : \quad p = 1, n = 1$$

$$\Rightarrow B\left(\frac{1}{2}\right) = 1$$

$$E_0 : \quad p = 0, n = 0$$

$$\Rightarrow B\left(\frac{0}{0}\right) = 0$$

$$\Rightarrow R(H) = \frac{2}{2} * 1 + \frac{0}{2} * 0 = 1$$

$$\Rightarrow G(H) = B(Pos) - R(H) = 0$$

Ham

$$E_1 : \quad p = 1, n = 0$$

$$\Rightarrow B\left(\frac{1}{1}\right) = 0$$

$$E_0 : \quad p = 0, n = 1$$

$$\Rightarrow B\left(\frac{0}{1}\right) = 0$$

$$\Rightarrow R(H) = \frac{1}{2} * 0 + \frac{1}{2} * 0 = 0$$

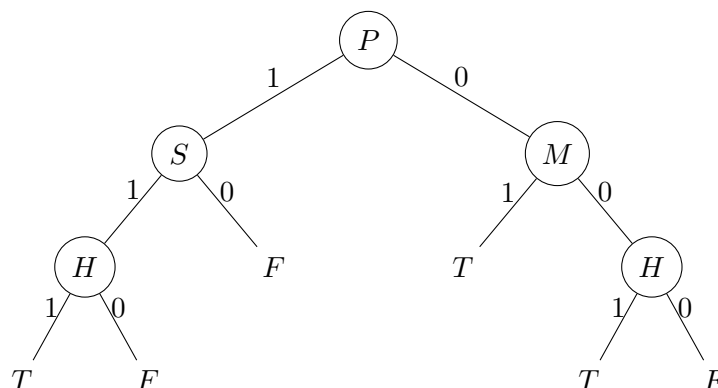
$$\Rightarrow G(H) = B(Pos) - R(H) = 1$$

Result of step 5

$$G(H) > G(M)$$

$$1 > 0$$

Choose the ingredient with the highest information. As a result, we have Ham as next Node:

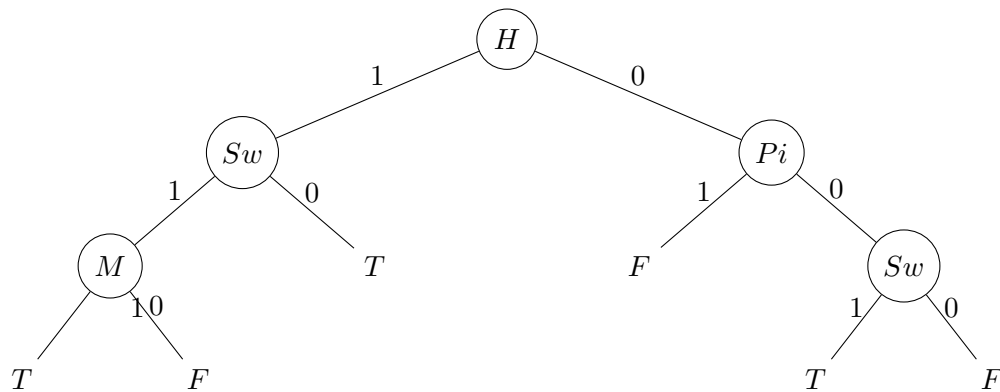


2. The Decision-Tree-algorithm tries to generate "smallish" trees. If Harry just picks random variables, the resulting tree could be much bigger, and therefore harder to use.

Exercise 7 :

In the table below the favorite pizzas of Harry's friends are listed. For each of the pizzas, use the following tree to say if Harry orders this pizza or not. (Note that this is not necessarily the solution to Exercise 6). Do not forget to declare your steps. We use the following abbreviations for the ingredients: *Pi* is Pineapple, *M* is Mushrooms, *H* is Ham, *Sw* is Sweetcorn.

Hermione	Mushroom, Ham, Sweetcorn and Onion
Neville	Pineapple, Sweetcorn and Mushrooms
Ron	Salami and Sweetcorn
Ginny	Ham
Luna	Salami, Bolognese sauce and Pepper



Solution: Solution:

	<i>Ordered?</i>
<i>Hermione</i>	<i>Yes</i>
<i>Neville</i>	<i>No</i>
<i>Ron</i>	<i>Yes</i>
<i>Ginny</i>	<i>Yes</i>
<i>Luna</i>	<i>No</i>

Exercise 1 :

Suppose you want to use a neural network for predicting user preferences based on the data set in Table 1.

Example	Author	Thread	Length	WhereRead	UserAction
e_1	known	new	long	home	skips
e_2	unknown	new	short	work	reads
e_3	unknown	follow Up	long	work	skips
e_4	known	follow Up	long	home	skips
e_5	known	new	short	home	reads
e_6	known	follow Up	long	work	skips
e_7	unknown	follow Up	short	work	skips
e_8	unknown	new	short	work	reads
e_9	known	follow Up	long	home	skips
e_{10}	known	new	long	work	skips
e_{11}	unknown	follow Up	short	home	skips
e_{12}	known	new	long	work	skips
e_{13}	known	follow Up	short	home	reads
e_{14}	known	new	short	work	reads
e_{15}	known	new	short	home	reads
e_{16}	known	follow Up	short	work	reads
e_{17}	known	new	short	home	reads
e_{18}	unknown	new	short	work	reads
e_{19}	unknown	new	long	work	?
e_{20}	unknown	follow Up	long	home	?
e_{21}	unknown	follow Up	short	home	?

Table 1: The user preference data to be used in Exercise 1.

What neural network structure could you use, especially: what would be the input and output units?

Solution:

This is a problem of finding a numerical encoding for discrete input and output attributes. There are several possibilities:

Assuming that all attributes are binary (i.e., only can have the two values that appear in the table, and, e.g., 'Length' can not also have the value 'medium'), one can use a neural network with one input node for each input attribute, and, for each attribute, encode one of the possible values as 0, and the other as 1. For example, assuming that 'unknown', 'follow Up', 'short' and 'home' are the values encoded as 0, one would have that the input in the first example is given as (1,1,1,0). In the same way, the output attribute 'User Action' would be encoded.

Alternatively (and this also works for attributes with more than 2 values), one can use one input node for each attribute-value combination. Here we would have the 8 different attributes Author_is_known, Author_is_unknown, ..., WhereRead_is_home, WhereRead_is_work. Then we encode the actual inputs

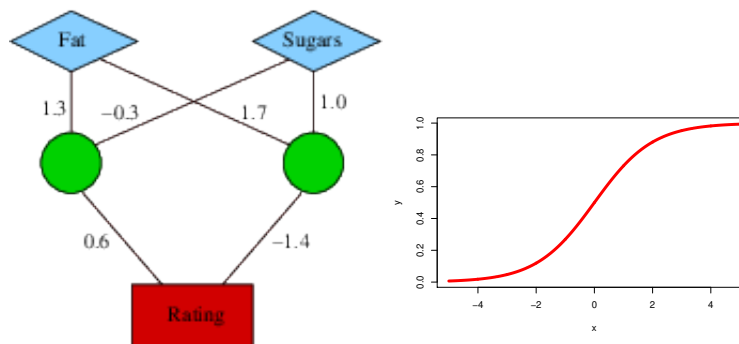
as 0/1 values of these new attributes. In the first example, the inputs then are set to `Author_is_known=1`, `Author_is_unknown=0`, ..., `WhereRead_is_home=1`, `WhereRead_is_work=0`.

Exercise 2 :

Compute for the neural network below the *Rating* output computed for the two inputs

Fat	Sugars
1	5
0	14

The two hidden units have the sigmoid activation function. Values for this function can be either computed precisely according to the definition $\sigma(x) = 1/(1 + e^{-x})$, or you can read approximate function values off the plot on the right below. The output unit has the identity activation function, i.e. the output is just the weighted sum of the inputs.

**Solution:**

The computation in the neural network proceeds top-to-bottom, where each node computes its output from the input it receives from the nodes in the preceding layer.

The input nodes don't perform any computations. Their output is just the input, i.e. in the first case, the *Fat* input node outputs a 1, and the *Sugars* input node outputs a 5.

Next, the two nodes in the hidden layer perform their computations. Each node first computes the weighted sum of its input, and then applies the activation function to compute the final output.

The weighted sum of inputs is:

$$\text{left hidden node: } 1.3 \cdot 1 + (-0.3) \cdot 5 = -0.2$$

$$\text{right hidden node: } 1.7 \cdot 1 + 1.0 \cdot 5 = 6.7$$

Now the activation function is applied to these numbers. The function value can be approximately read off the plot, or computed precisely:

$$\text{output left hidden node: } 1/(1 + e^{0.2}) = 0.45 \quad \text{output right hidden node: } 1/(1 + e^{-6.7}) = 0.998$$

Next, the 'Rating' output node can compute its output. The weighted input is

$$0.6 \cdot 0.45 + (-1.4) \cdot 0.998 = -1.1272.$$

Since the output node has the identity activation function, this is also already the output of the 'Rating' node.

Exercise 3 :

Assume that we have the following training examples:

X_1	X_2	T
1	1	1
-1	1	-1
1	-1	1
-1	-1	-1

That is, with input $X_1 = 1$ and $X_2 = -1$ we want the output 1.

Consider a perceptron (single neuron) with threshold input 1 and with initial weights $w_0 = 0, w_1 = 0$ and $w_2 = 0$.

- Show the first two iterations of gradient descent when learning a perceptron (having the sign function as activation function) using learning rate $\alpha = 0.25$. Indicate the intermediate steps that need to be computed (e.g. the value of forward propagation, error term and how the weights are adjusted).
- Show the first two iterations of stochastic gradient descent when learning a perceptron (having the sign function as activation function) using learning rate $\alpha = 0.25$. Indicate the intermediate steps that need to be computed (e.g. the value of forward propagation, error term and how the weights are adjusted). Take examples one by one, in the same order as they are in the table.

Note: For this exercise, we assume that $\text{sign}(0) = -1$

Solution:

i) First iteration:

Cases:	(1, 1, 1)	(1, -1, 1)	(1, 1, -1)	(1, -1, -1)
y	1	-1	1	-1
o	-1	-1	-1	-1
$\delta_o = y - o$	<u>2</u>	0	<u>2</u>	0

$$\bar{w} := (0, 0, 0) + \frac{1}{4} \cdot 2 \cdot (1, 1, 1) + \frac{1}{4} \cdot 2 \cdot (1, 1, -1) = (1, 1, 0)$$

(note that we can focus on elements where there is some error $\delta_o \neq 0$ because the others cancel out)

Second iteration:

Cases:	(1, 1, 1)	(1, -1, 1)	(1, 1, -1)	(1, -1, -1)
t	1	-1	1	-1
o	1	-1	1	-1
$\delta_o = y - o$	0	0	0	0

$$\bar{w} := (1, 1, 0)$$

(there is no error, therefore gradient descent has already found the minimum value)

ii) First iteration:

Cases:	(1, 1, 1)
y	1
o	-1
$\delta_o = y - o$	2

$$\bar{w} := (0, 0, 0) + \frac{1}{4} \cdot 2 \cdot (1, 1, 1) = (0.5, 0.5, 0.5)$$

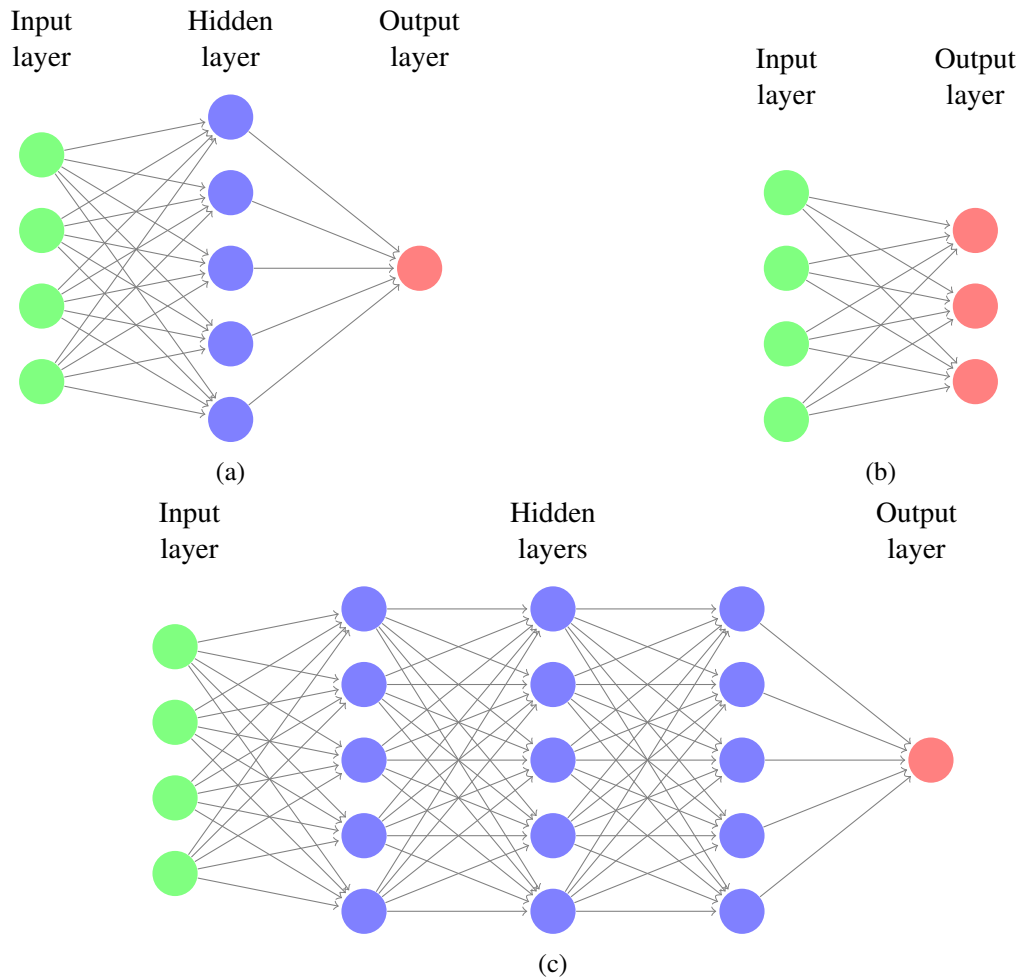
Second iteration:

Cases:	(1, -1, 1)
t	-1
o	1
$\delta_o = y - o$	<u>-2</u>

$$\bar{w} := (0.5, 0.5, 0.5) - \frac{1}{4} \cdot 2 \cdot (1, -1, 1) = (0, 1, 0)$$

Exercise 4 :

For each of the following neural networks:



indicate:

- i) How many parameters need to be adjusted by the learning algorithm?
- ii) How many functions are represented by the network?
- iii) Can the network represent the Boolean function $x_1 \text{ xor } x_2$? Justify your answer.

Solution:

1. The parameters are the weights which are trained for (for each neuron that is the number of input neurons plus one due to the weight that is multiplied by the constant 1): so (a) has $(4 + 1) \cdot 5 + (5 + 1) = 31$, (b) has $(4 + 1) \cdot 3 = 15$, and (c) has $(4 + 1) \cdot 5 + 2 \cdot (5 + 1) \cdot 5 + 5 + 1 = 91$.
2. The number of represented functions are the number of output neurons, so 1, 3, and 1.

3. (b) cannot represent it, as it has no hidden layers.

Exercise 5 :

Experiment with the tensorflow playground

<https://playground.tensorflow.org>

- Use different data sets and neural network structures (varying the number of layers and neurons). How does the different network structures affect the learning ability? Can you derive any useful insights by considering the features learned at the hidden units.
- What is the effect of varying the learning rate?

Optional: You can also take a look at the questions in this course: <https://developers.google.com/machine-learning/crash-course/introduction-to-neural-networks/playground-exer>

Exercise 6 (Optional):

Complete one more iteration of the back propagation algorithm for the example on slide 08.29.

Exercise 1 :

Consider a poker game consisting of two rounds, and where each player is initially dealt three cards. During the first round all three cards can be changed (*FC*), but during the second round at most two cards can be changed (*SC*). When deciding on whether to call or fold you can taken into account the number of cards changed by your opponent as well as your current hand (*MH*). After playing 20 games we have the results in Table 1, where *BH* shows who has the best hand.

Case number:	<i>BH</i>	<i>MH</i>	<i>FC</i>	<i>SC</i>
1	op	no	3	1
2	op	1a	2	1
3	draw	2 v	1	1
4	me	2 a	1	1
5	draw	fl	1	1
6	me	st	3	2
7	me	3 v	1	1
8	me	sfl	1	0
9	op	no	0	0
10	op	1 a	3	2
11	draw	2 v	2	1
12	me	2 v	3	2
13	op	2 v	1	1
14	op	2 v	3	0
15	me	2 v	3	2
16	draw	no	3	2
17	draw	2 v	1	1
18	op	fl	1	1
19	op	no	3	2
20	me	1 a	3	2

Table 1: Training data for constructing a poker classifier.

- Construct a naive Bayes classifier for the poker domain.
- Use the data cases to learn the parameters in the model; if you feel comfortable with the estimation procedure, you only need to estimate the probabilities required for solving the exercise below.
- What class does your classifier assign to a case with $MH=1a$, $FC=1$, and $SC=1$?

Solution:

See the Hugin network .

BH	MH	op	me	draw
no	0.375	0	0	0.2
1a	0.25	0.142857	0	0
2v	0.25	0.285714	0	0.6
2a	0	0.142857	0	0
fl	0.125	0	0	0.2
st	0	0.142857	0	0
3v	0	0.142857	0	0
sfl	0	0.142857	0	0

The probabilities in the model have been calculated using simple frequency counting. For example, for $P(FC = 1|BH = op)$ we get

$$P(FC = 1|BH = op) = \frac{N(FC = 1, BH = op)}{N(BH = op)} = \frac{2}{8} = \frac{1}{4}.$$

By estimating the remaining entries in the conditional probability tables and inserting the evidence $MH = 1a, FC = 1, SC = 1$, we get the posterior probability $P(BH|MH = 1a, FC = 1, SC = 1) = (0.671_{op}, 0.329_{me}, 0_{dr})$.

Observe that if you are *only* interested in calculating the probability $P(BH|MH = 1a, FC = 1, SC = 1)$, then you need not estimate all the probabilities required to get a fully specified naive Bayesian network (this would require estimating $P(BH), P(MH|BH), P(FC|BH), P(SC|BH)$). Instead you only need to estimate probabilities for the configurations that are consistent with the configuration that you are conditioning on. Specifically,

$$P(BH|MH = 1a, FC = 1, SC = 1) = \frac{P(BH)P(MH = 1a|BH)P(FC = 1|BH)P(SC = 1|BH)}{\sum_{BH} P(BH)P(MH = 1a|BH)P(FC = 1|BH)P(SC = 1|BH)},$$

so you need not estimate probabilities that are not used in the calculations above (e.g. $P(MH = 2v|BH)$).

Exercise 2 :

You want to predict whether a person will pay back a loan based on the features *Income*, *Houseowner* and *Marital Status* of the person. Domains and distance functions on the domains of these features are defined as follows:

<i>Income</i>	<i>low</i>	<i>medium</i>	<i>high</i>
<i>low</i>	0	1	2
<i>medium</i>	1	0	1
<i>high</i>	2	1	0

<i>Houseowner</i>	<i>yes</i>	<i>no</i>
<i>yes</i>	0	1
<i>no</i>	1	0

<i>Marital Status</i>	<i>unmarried</i>	<i>married</i>	<i>divorced</i>
<i>unmarried</i>	0	1	1
<i>married</i>	1	0	1
<i>divorced</i>	1	1	0

Define the distance between two examples by the sum of the distances for the three features.

Your training examples are:

	Income	Houseowner	Marital Status	Pay back
1	high	yes	married	yes
2	high	yes	unmarried	yes
3	medium	no	divorced	no
4	low	yes	married	no
5	low	no	unmarried	no

Now you want to predict 'Pay back' for a new case with Income = high, Houseowner = no, Marital Status = divorced.

- What is the prediction obtained by the 1-nearest-neighbor rule?
- What is the prediction obtained by the 3-nearest-neighbor rule?
- What could be a sensible modification of the distance function such that you would get a different result from the 1-nearest-neighbor rule?

Solution:

The distance of the new case to the 5 training examples is:

$$\begin{array}{ll}
 1 & 0 + 1 + 1 = 2 \\
 2 & 0 + 1 + 1 = 2 \\
 3 & 1 + 0 + 0 = 1 \\
 4 & 2 + 1 + 1 = 4 \\
 5 & 2 + 0 + 1 = 3
 \end{array}$$

Thus:

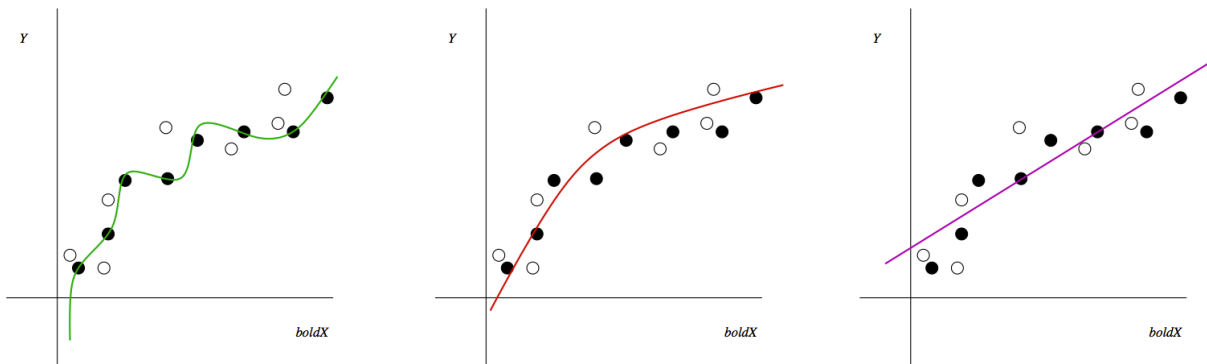
- The prediction from the 1-nearest-neighbor rule is Pay back = no, because training example 3 has smallest distance to the new case.
- The prediction from the 3-nearest-neighbor rule is Pay back = yes, because 2 out of the 3 nearest neighbors (training cases 1,2,3) have Pay back = yes.
- A sensible modification of the distance function could be to give lower weight to the distance contributed by the marital status attribute, as by the income attribute. For example, if we give a weight of 10 to the income attribute, a weight of 5 to the houseowner attribute, and a weight of 1 to the marital status attribute, then the distances are

- 1 $10 \cdot 0 + 5 \cdot 1 + 1 = 6$
- 2 $10 \cdot 0 + 5 \cdot 1 + 1 = 6$
- 3 $10 \cdot 1 + 5 \cdot 0 + 0 = 10$
- 4 $10 \cdot 2 + 5 \cdot 1 + 1 = 26$
- 5 $10 \cdot 2 + 5 \cdot 0 + 1 = 21$

Now there is a tie between training cases 1 and 2 for being the nearest neighbor, but both have Pay back = yes, so now the 1-nearest neighbor rule would predict Pay back = yes.

Exercise 3 :

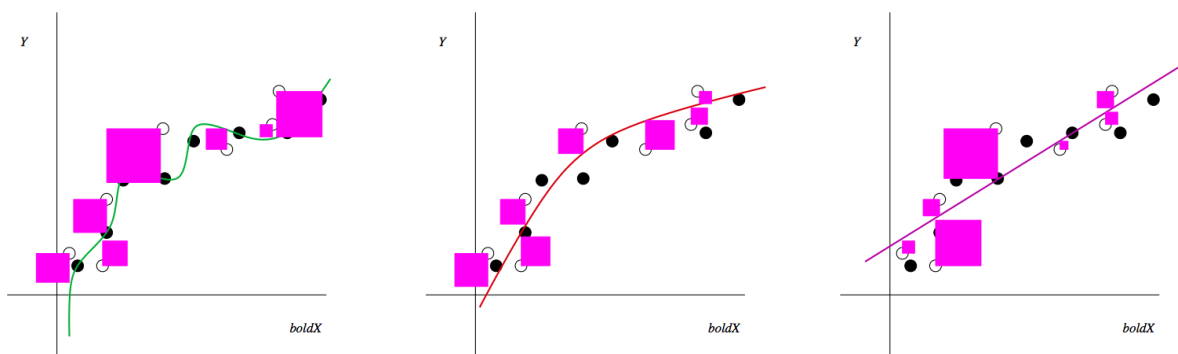
The following graphs show three regression models learned from training examples (filled dots):



The open dots represent a set of future observations (or a validation set). Which of the three models has the smallest SSE on these future observations? No exact computations required – try to read it (approximately) off the graphs!

Solution:

Indicating the squared errors of each validation example by a colored square (cf. slide 09.22) gives:



From this it appears (subject to exact verification by measurement!) that the SSE of the leftmost model is much higher than that of the other two, and that the one of the middle model is a little lower than that of the right (linear) model.

Exercise 4 :

Suppose you are working on a spam detection system. You formulated the problem as a classification task where “Spam” is the positive class and “not Spam” is the negative class. Your training set contains $m = 1000$ emails.

	Predicted Spam	Predicted not Spam
Actual Spam	8	2
Actual not Spam	16	974

Calculate the Accuracy, Recall, Specificity and Precision in this example.

How do your answers change if we now consider “Spam” the negative class and “not Spam” the positive class?

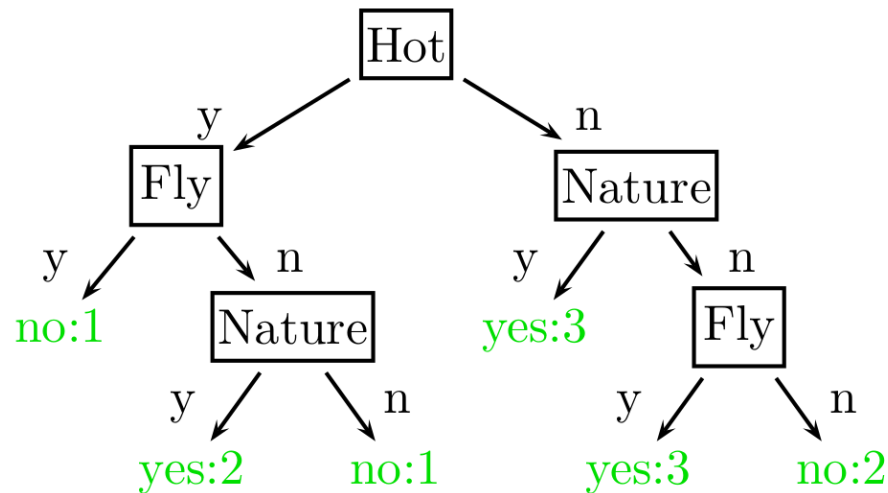
Solution:

- Accuracy $\frac{8+974}{1000} = 0.982$
- Recall $\frac{8}{8+2} = 0.80$
- Specificity $\frac{974}{974+16} = 0.9838$
- Precision $\frac{8}{8+16} = 0.3333$

If we consider “Spam” the negative class and “not Spam” the positive class then the original Specificity becomes Recall and vice versa, so new Recall is $\frac{974}{974+16} = 0.9838$ and new Specificity is $\frac{8}{8+2} = 0.80$. Accuracy remains the same, Accuracy $\frac{974+8}{1000} = 0.982$. Precision becomes $\frac{974}{976} = 0.9980$ (In tutorial note the difference in Precision values).

Exercise 5 :

Based on 12 training examples the following decision tree was learned:



Here, for example, the label “yes:2” at the end of the branch Hot=y, Fly=n, Nature=y means that there were two examples in the training set with Hot=y, Fly=n, Nature=y, and both examples had class label Likes=yes. Thus, the decision tree has 100% accuracy on the training data (all leaves are class pure).

Now suppose you have the following 5 examples (not used in the construction of the tree), which you want to use as a validation set:

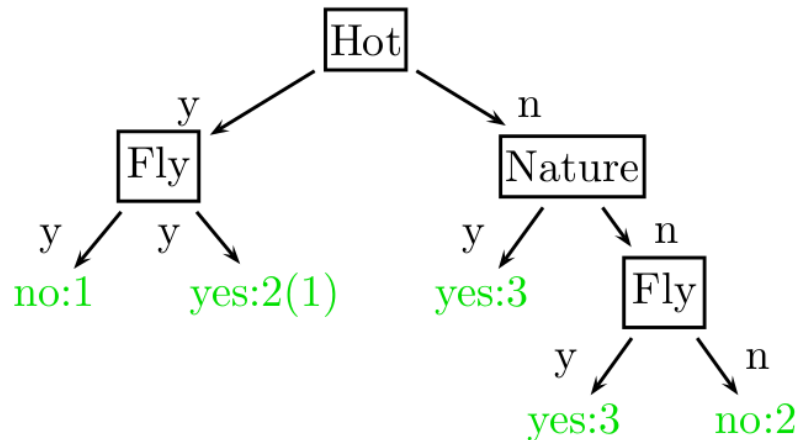
Culture	Fly	Hot	Music	Nature	Likes
no	no	yes	yes	yes	no
no	no	yes	no	no	yes
yes	yes	no	no	no	yes
yes	no	yes	no	yes	yes
no	no	no	no	no	no

Based on these validation examples, we perform post-pruning of the decision tree:

- First check whether the 'Nature' node reached by Hot=y, Fly=n should be pruned (eliminated)
- If yes, what does the new tree look like after pruning?
- Continue the pruning process by checking for other nodes whether they should be pruned (in a bottom-up order; the next candidate for pruning could be the 'Fly' node reached by Hot=n, Nature=n).

Solution:

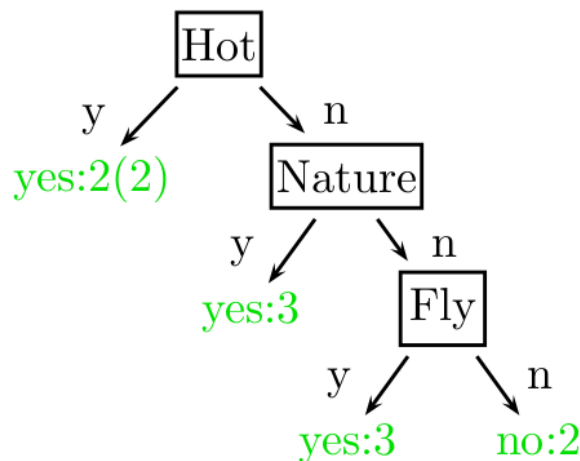
3 of the validation examples (1,2, and 4) go into the Hot=y, Fly=n branch. 2 of these (1,2) will be misclassified based on the 'Nature' feature. If the 'Nature' node is pruned, then it will be replaced by a leaf labeled with 'yes' (because the majority of the *original training examples* are 'yes' cases). With the resulting tree, then only validation example 1 will be misclassified. Thus, the performance on the validation set improves, we would perform the pruning to obtain:



The 'yes:2(1)' label means: cases ending in this leaf are predicted 'yes', and this is correct for 2 training examples, but there is also one 'no' training example.

Next we check whether the rightmost 'Fly' node should also be pruned. Two validation examples (3,5) reach this node. Both of them are correctly classified based on the 'Fly' feature. If that node was eliminated, then one of the two would have to be mis-classified. Therefore, the accuracy on the validation set is higher when the 'Fly' node is kept.

Finally, we can check whether pruning the 'Fly' node reached by Hot=y should be pruned. This node is still only reached by the three examples 1,2,4, two out of which are correctly classified. If the node was pruned, then the resulting leaf could be labeled either 'yes' or 'no' (because there are then 2 training examples for each of these labels). Assuming that we would label the resulting leaf 'yes', we would still obtain 2 out of 3 correct classifications for these three validation examples. Thus, by pruning, we obtain a tree with the same accuracy, but one that is a bit smaller than the previous. This is a borderline situation, and depending on how the pruning algorithm exactly resolve this, one might decide not to prune 'Fly', or prune this node also and end up with the tree:



Exercise 6 :

Look at this example, introduced in the lecture. Assuming the points are measurements from some experiment and the functions are fitting functions to predict further measurements. The left one is a linear function and the right one is a polynomial function. According to Ockham's razor, which one of these functions is the better pick. Justify your answer briefly (in 2-3 sentences), highlighting why Ockham's razor prefers one of the functions and what the problem of overfitting would be in this case.

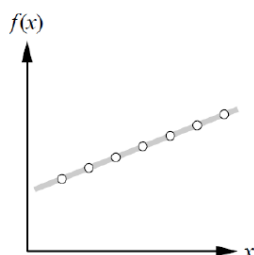


Figure 1: The linear function

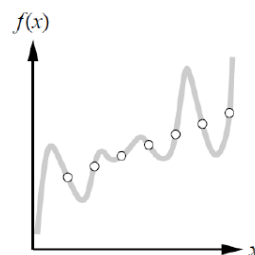


Figure 2: The polynomial function

Solution: Ockham's razor would pick the Linear Function. It prefers the simpler solution, and in this case, the polynomial function is more complex than the linear function. The polynomial function overfits the past measurements, being correct for the current data points, but not following the general curve, so it will likely deliver worse predictions for future measurements.

Exercise 7 :

Solve Exercise 10.1 in PM.

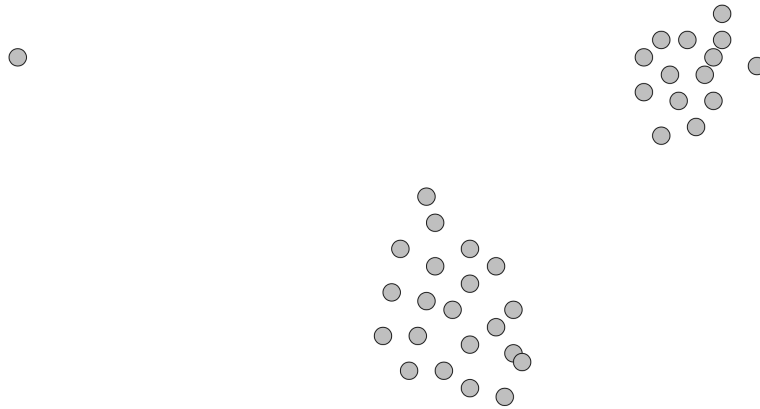
Solution: The posterior distribution for C given $A = a$ and $B = b$ is

$$P(C | a, b) = \frac{P(C, a, b)}{P(C = 0)P(a|C = 0)P(b|C = 0) + P(C = 1)P(a|C = 1)P(b|C = 1)}.$$

Both terms in the denominator are zero due to $P(A | C = 0)$ and $P(B | C = 1)$, respectively.

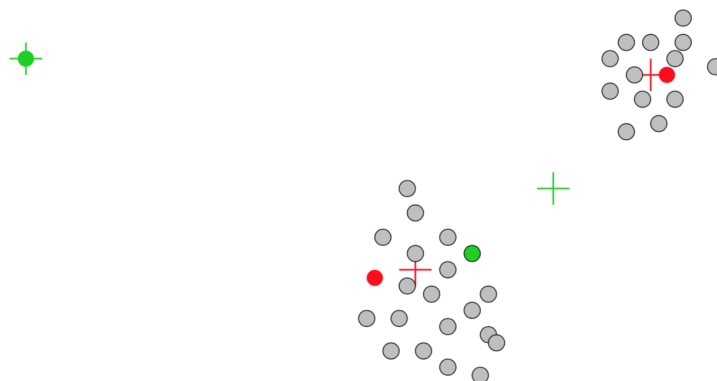
Exercise 1 :

For the following data set:



- identify two initial cluster centers such that 2-means clustering initialized with these cluster centers will terminate with the single outlier forming one cluster, and all other points the second cluster
- identify two initial cluster centers such that 2-means clustering initialized with these cluster centers will terminate with the two big groups of points forming different clusters (and the outlier belonging to one of those)

In both cases indicate the (approximate) position of the final cluster centers.

Solution:

- The green filled instances are possible initial cluster centers; the green crosses are the (approximate) final cluster centers.
- Same with red ...

Exercise 2 :

Consider the data points plotted in Figure 1.

- Perform two iterations of the k -means algorithm using
 - the data points $(2, 6)$ and $(3, 5)$ as the initial cluster centers.
 - the Euclidean distance as distance metric
- Calculate the sum of squared errors using the initial cluster centers and the cluster centers that you found above.

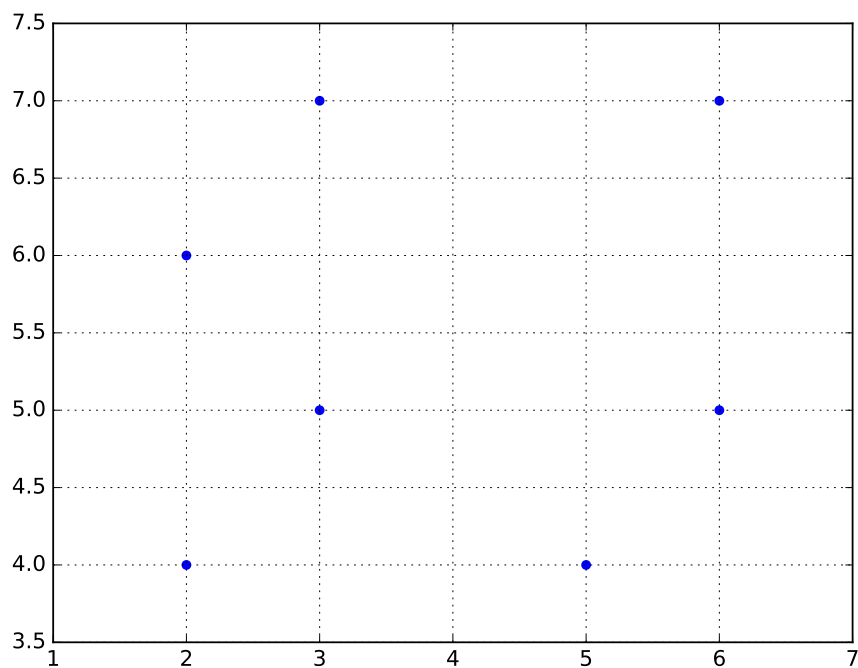


Figure 1: Data to be clustered in Exercise 2.

Solution:

The result of the first two iterations are illustrated in Figure 2. The cluster centers are located at:

- 1. iteration: $(2.5, 6.5)$ and $(4.4, 5)$.
- 2. iteration: $(2.33, 5.67)$ and $(5, 5.25)$.

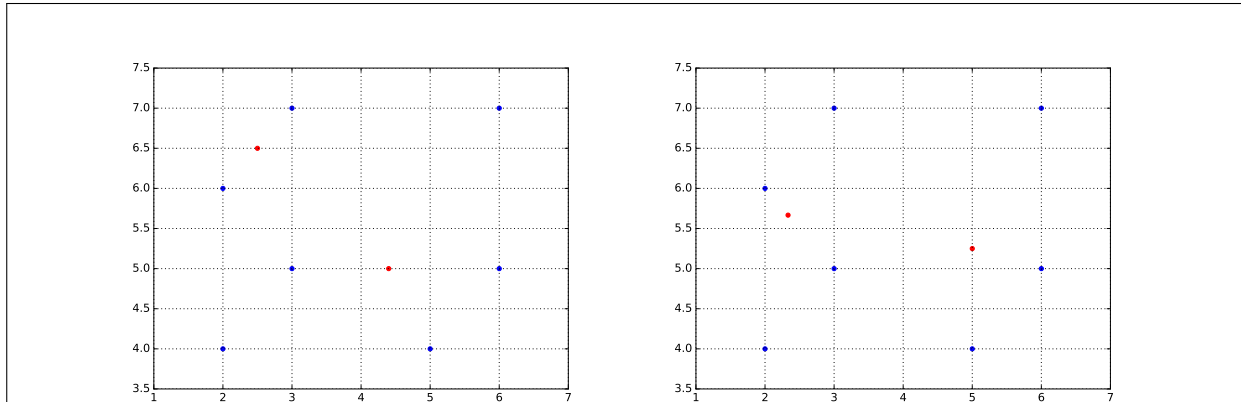


Figure 2: The cluster centers (red dots) after the first and second iterations, respectively.

The sum of squared errors are 31.0 and 12.91, respectively.

Exercise 3 :

Perform one more iteration of the EM algorithm for the example on Slide 10.33. Note that you will first need to complete the last two maximization calculations for the 2nd iteration, which is left unfinished on the slides.

Solution:

First we start by completing the second iteration of the EM-algorithm. This consists of updating the distributions for $P_2(F_2|C)$ and $P_2(F_3|C)$; note that the subscript 2 refers to the iteration number.

$$\begin{aligned}
 P_2(F_2|C) &= \frac{\sum_{F_1, F_3} A(F_1, F_2, F_3, C)}{\sum_{F_1, F_2, F_3} A(F_1, F_2, F_3, C)} \\
 &= \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_2 = t & 0.88 + 0 + 0 + 0 & 0.12 + 0 + 0 + 0 \\ F_2 = f & 0 + 0.66 + 0.48 + 0.47 & 0 + 0.34 + 0.52 + 0.53 \end{array}}{(0.88 + 0.66 + 0.48 + 0.47, 0.12 + 0.34 + 0.52 + 0.53)} \\
 &= \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_2 = t & 0.88 & 0.12 \\ F_2 = f & 1.61 & 1.39 \end{array}}{(2.49, 1.51)} \\
 &= \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_2 = t & 0.35 & 0.08 \\ F_2 = f & 0.65 & 0.92 \end{array}}{}
 \end{aligned}$$

The tables above are structured in the same way as on the slides. Thus, the columns correspond to the two states of C and the rows correspond to $F_2 = t$ and $F_2 = f$, respectively. For $P_2(F_3|C)$ we end up

with (the intermediate calculations follow the same steps as above):

$$P_2(F_3|C) = \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_3=t & 2.01 & 0.99 \\ F_3=f & 0.48 & 0.52 \end{array}}{(2.49, 1.51)}$$

$$= \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_3=t & 0.81 & 0.66 \\ F_3=f & 0.19 & 0.34 \end{array}}{(1.0, 1.0)}$$

Continuing with the third iteration of the EM-algorithm, we start by calculating the expected counts that should go into our count table $A(F_1, F_2, F_3, C)$. This should be done using the conditional probability tables estimated during the last iteration (i.e., $P_2(C)$, $P_2(F_1|C)$, $P_2(F_2|C)$, and $P_2(F_3|C)$) and reduces to calculating $P_2(C|F_1, F_2, F_3)$ for the four different configurations of F_1 , F_2 , and F_3 observed in the data. In total we end up with:

F_1	F_2	F_3	$P(C F_1, F_2, F_3)$
t	t	t	(0.92, 0.08)
t	f	t	(0.64, 0.36)
t	f	f	(0.44, 0.56)
f	f	t	(0.43, 0.57)

Based on the updated count table, we recalculate the conditional probabilities of the model, i.e., $P_3(C)$, $P_3(F_1|C)$, $P_3(F_2|C)$, and $P_3(F_3|C)$. The calculation procedures are the same as for the previous two iterations and results in the following tables:

$$P_3(C) = (0.61, 0.39)$$

$$P_3(F_1|C) = \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_1=t & 0.82 & 0.64 \\ F_1=f & 0.18 & 0.36 \end{array}}{(1.0, 1.0)}$$

$$P_3(F_2|C) = \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_2=t & 0.39 & 0.05 \\ F_2=f & 0.61 & 0.95 \end{array}}{(1.0, 1.0)}$$

$$P_3(F_3|C) = \frac{\begin{array}{c|cc} & C=1 & C=2 \\ \hline F_3=t & 0.82 & 0.64 \\ F_3=f & 0.18 & 0.36 \end{array}}{(1.0, 1.0)}$$

Exercise 4 :

Consider the following five data points living in $\mathbb{R}^2$:

d_1	(7, 2)
d_2	(52, 3)
d_3	(70, 10)
d_4	(85, 11)
d_5	(90, 8)

Using the Euclidean distance to measure distances:

- (i) Find the data point closest to d_2 in the data set, i.e., (52, 3).
- (ii) Normalize the data using Z-score normalization. Find the data point closest to d_2 using the transformed data set.
- (iii) Let the data points (7, 2) and (70, 10) be initial cluster centers for the k -means algorithm. Perform one more k -means iteration by updating these cluster centers using the non-normalized data set above.
- (iv) Do you see any potential problems in directly using k -means clustering with the Euclidean distance based the data set above? What could you do to mitigate such problems?

Solution:

- (i) The data point $d_3 = (70, 10)$ is the one closest to $d_2 = (52, 3)$ with a distance of 19.31.

d1	0.00	45.01	63.51	78.52	83.22
d2	45.01	0.00	19.31	33.96	38.33
d3	63.51	19.31	0.00	15.03	20.10
d4	78.52	33.96	15.03	0.00	5.83
d5	83.22	38.33	20.10	5.83	0.00

Distances

- (ii) The normalized data set is given by:

$$(-1.79, -1.31), (-0.29, -1.03), (0.3, 0.87), (0.81, 1.15), (0.97, 0.33)$$

The data point closest to $(-0.29, -1.03)$ (corresponding to $d_2 = (52, 3)$) is $(-1.79, -1.31)$ (corresponding to $d_1 = (7, 2)$).

d1	0.00	1.53	3.03	3.58	3.22
d2	1.53	0.00	2.01	2.45	1.86
d3	3.03	2.01	0.00	0.57	0.86
d4	3.58	2.45	0.57	0.00	0.84
d5	3.22	1.86	0.86	0.84	0.00

Normalized distances

- (iii) First we find the points that belong to the two clusters defined by the initially chosen cluster centers:

$dist$	d_2	d_4	d_5
d_1	45.01	78.51	83.22
d_3	19.31	15.03	20.10

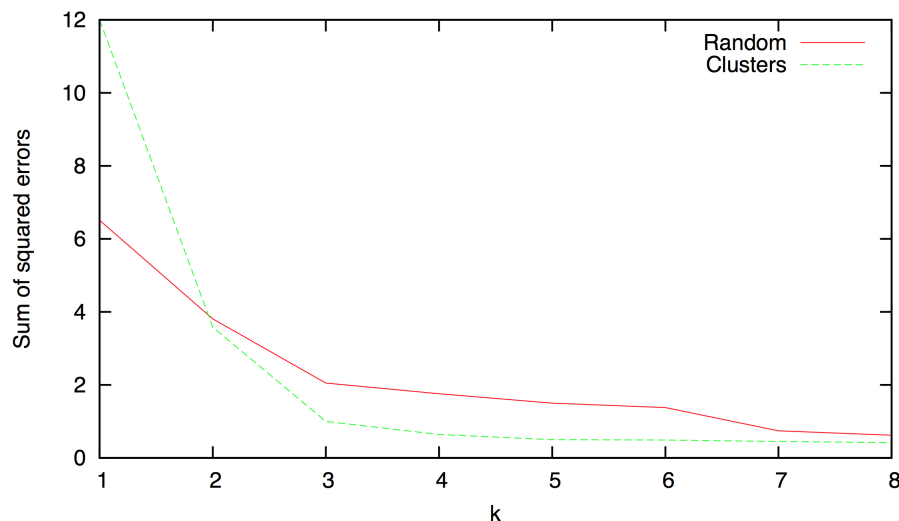
Thus, the data points d_2 , d_4 , and d_5 all move to the cluster defined by d_3 ; d_1 now defines a cluster with a single element.

Since no points are assigned to the cluster defined by d_1 we keep that data point as a cluster center. The cluster center defined by the mean of the four remaining points becomes $(74.25, 8)$.

Exercise 5 :

1.

We perform k -means clustering for $k = 1, 2, 3, 4, 5, 6, 7, 8$ on two datasets (Random and Clusters). sets. For each of the cases, we compute the “Within cluster sum of squared errors” (which corresponds to the sum of squared errors, the distance for each example to the centroid of its cluster). How can plots like these be used to determine the “right” number of clusters?



- Suppose that, for a given value of k , we run k -means multiple times with different random seeds. How could we compare the results obtained in the different runs to determine if the dataset has k “real” clusters?

Solution:

- The plot indicates that for clustering clusters a particularly sharp drop in the error value (compared to the drop for a random dataset) up to $k = 3$, where the curve then levels off. The random data shows a more uniform decrease in error value over the whole k -range. “Knees” in the SSE error function are a (heuristic) indicator for the “right” number of clusters.
- We could determine whether the clusters are stable, i.e. the same clusters are returned independent of the random seed (which determines the random initial cluster centers of the algorithm).

If the final result is completely different for different settings of the seed, this could indicate that the computed clusters are not well-defined clusters in the data.

Exercise 1 :

Consider the state space from Figure 1.

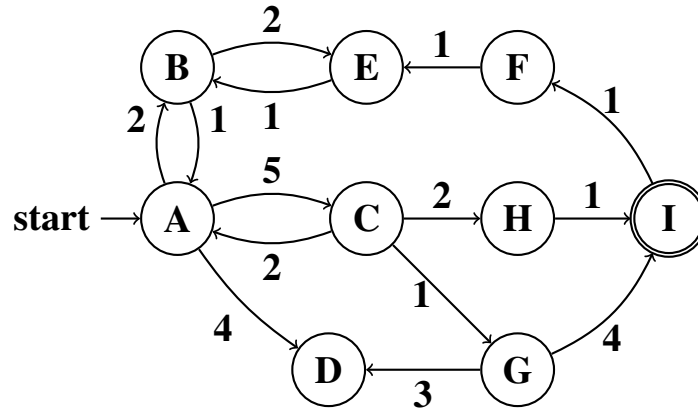


Figure 1: State space.

As a heuristic estimate for a state s , use the minimal number of edges that are needed to reach a goal state from s (or ∞ if s is not solvable, e.g., $h(C) = 2$).

As tie-breaking criteria, if the choice of the next state to be expanded is not unique, expand the lexicographically smallest state first (so A would be expanded before B if both have the same f -value).

- (i) Run Uniform-cost-search on this problem. Draw the search graph and **annotate each node with the g value as well as the order of expansion**. Draw duplicate nodes as well, and mark them accordingly by crossing them out. Give the solution found. Is this solution optimal? Justify your answer.
- (ii) Run A^* search on this problem. Draw the search graph and **annotate each node with the g and h value as well as the order of expansion**. Draw duplicate nodes as well, and mark them accordingly by crossing them out. Give the solution found by A^* search. Is this solution optimal? Justify your answer.
- (iii) What is the minimum number of states that must be expanded by uniform-cost-search regardless of tie-breaking?
- (iv) What is the lower bound on the minimum number of states that must be expanded by A^* search regardless of tie-breaking (ignoring all states with f equal to the optimal solution cost).
- (v) In terms of runtime, which algorithm will find the optimal solution faster. Justify your answer.

Solution:

- (i) The solution is A, C, H, I . Uniform-cost search is guaranteed to return an optimal solution if there are no negative costs.

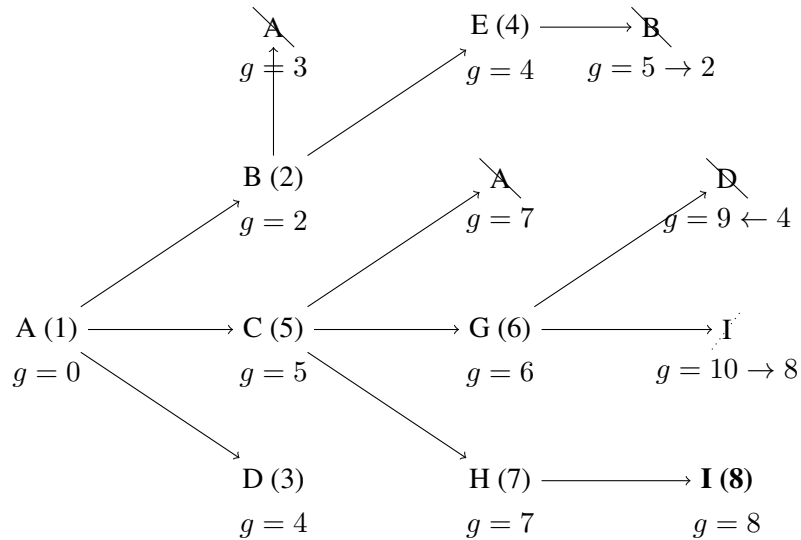


Figure 2: Solution to part (i). The algorithm stops when the goal node I is selected for expansion.

- (ii) The solution is A, C, H, I . A^* is guaranteed to return an optimal solution if the heuristic that is being used is consistent. Thus, it remains to show that the described heuristic h is consistent, i.e., that for every transition $s \xrightarrow{a} s'$, it holds that $h(s) - h(s') \leq c(a)$. By definition of our heuristic the following holds: $\forall s, s' \in S \setminus \{D\} : s \xrightarrow{a} s' \implies |h(s) - h(s')| = 1$. That means that the difference in heuristic values between any state s and any successor state s' of s is maximally 1, except for state D . In the special case of state D being the successor state, $h(s) - h(s') = -\infty$, which is less than the cost of any action (since D is a dead end it does not have any successors). Otherwise the minimal cost for any transaction is 1, so the heuristic is consistent.

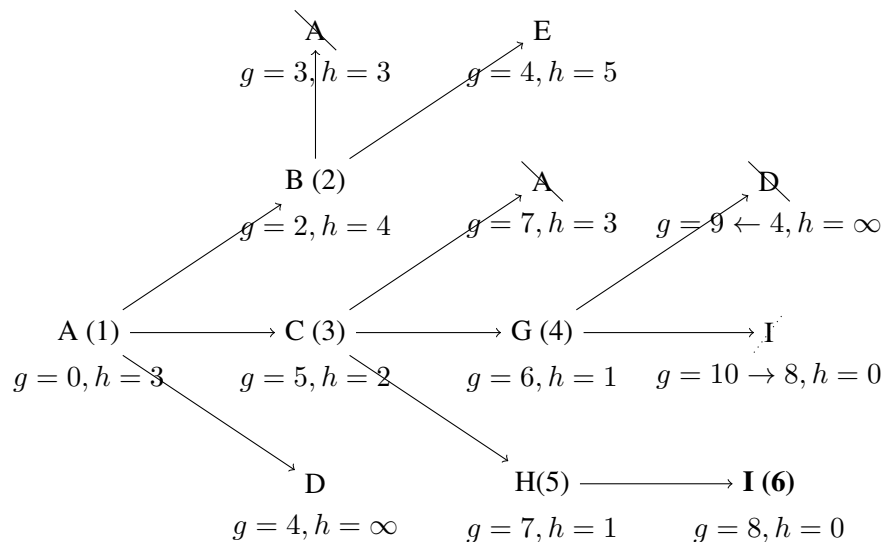


Figure 3: Solution to part (ii). The algorithm stops when the goal node I is selected for expansion.

- (iii) Uniform-cost search will explore all states with a g -value lower than the cost of the optimal solution. In this example, the optimal solution cost is 8 and there are 7 states with an optimal g -value lower than 8.
- (iv) A* search (and any other algorithm that guarantees the optimal solution and does not have access to any additional source of information) will explore all states with an f -value lower than the cost of the optimal solution. In this example, the optimal solution cost is 8 and there are 4 states with an optimal f -value lower than 8. Note that in this case to find a solution H must also be expanded, so that's why the question asks for the lower-bound ignoring those states with an f -value equal to the optimal solution (as it is difficult in the general case to know whose of those must be expanded necessarily).
- (v) It depends. A* expands fewer states, but it requires computing the heuristic per state. So which algorithm is faster depends on the runtime of expanding a state and generating the successors and the runtime of computing the heuristic. Also, if the heuristic is inconsistent, A* may expand nodes with the same state more than once.

Exercise 2 :

Formalize the following problems as state-space problems: define a suitable state space, a start state, set of actions, the action function, and a goal test.

- a. In the movie *Die hard: With a vengeance*, Bruce Willis and Samuel L. Jackson are given a water jug riddle, which they need to solve in order to disarm a bomb: There is a water fountain and two jugs that can hold 3 and 5 liters of water, respectively. How do you measure up 4 liters of water (to disarm the bomb) using only the two jugs?
- b. (From Russel & Norvig, Exercise 3.9): The *missionaries and cannibals problem* is usually stated as follows: Three missionaries and three cannibals are on one side of a river, along with a boat that can hold either one or two people. Find a way to get everyone to the other side of the river without ever leaving a group of missionaries in one place outnumbered by the cannibals in that place.

Note: This exercise is for you to reflect in the problem definition. Do not spend too much time “formalizing” the problem if you find it difficult, just provide an intuition of how these problems can be encoded as state space search problems.

Solution:

- a. States are the possible filling amounts (in full liters) of the two jugs. For example, (2, 0) is the state corresponding to the case where the first jug contains 2 liters and the second jug contains 0 liters. The start state is (0, 0). The 'fill second jug' action leads to the state (0, 5) from which the state (3, 2) can be reached by the action 'fill first jug from second jug'. A goal state is any state where the second jug contains 4 liters.

To formalize the actions and transition relation, consider the set of actions to be fill-x, fill-y (representing filling each of the two jars), empty-x, empty-y (representing emptying each of

the two jars), and transfer-x, transfer-y (representing pouring one jar into the other). Then, the transition relation is: $T = T_{fillx} \cup T_{filly} \cup T_{emptyx} \cup T_{emptyy} \cup T_{transferx} \cup T_{transfery}$, where:

- $T_{fillx} = \{((x, y), \text{fill-x}, (3, y)) \text{ for all } x \in [0, 3], y \in [0, 5]\}$
- $T_{filly} = \{((x, y), \text{fill-y}, (x, 5)) \text{ for all } x \in [0, 3], y \in [0, 5]\}$
- $T_{emptyx} = \{((x, y), \text{empty-x}, (0, y)) \text{ for all } x \in [0, 3], y \in [0, 5]\}$
- $T_{emptyy} = \{((x, y), \text{empty-y}, (x, 0)) \text{ for all } x \in [0, 3], y \in [0, 5]\}$
- $T_{transferx} = \{((x, y), \text{transfer-x}, (\max(0, 5-(x+y)), \min(x+y, 5))) \text{ for all } x \in [0, 3], y \in [0, 5]\}$
- $T_{transfery} = \{((x, y), \text{transfer-y}, (\min(x+y, 3), \max(0, 3-(x+y)))) \text{ for all } x \in [0, 3], y \in [0, 5]\}$

- b. Here is one possible representation: A state is a six-tuple of integers listing the number of missionaries, cannibals, and boats on the first side, and then the second side of the river. The goal is a state with 3 missionaries and 3 cannibals on the second side. The cost function is one per action, and the successors of a state are all the states that move 1 or 2 people and 1 boat from one side to another.

Exercise 3 :

Consider a variant of the Vacuum Cleaner problem from the lecture where a robot has to clean a 3×3 square room (see Figure 4). There are four possible actions: up, left, right, and down. There is no suck action since the robot automatically cleans any dirty spot it stands on. Hence, its task is moving around the room and visit all dirty spots. Throughout the exercise, we use Manhattan distance.

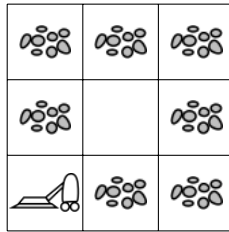


Figure 4: Illustration of the Vacuum Cleaner problem

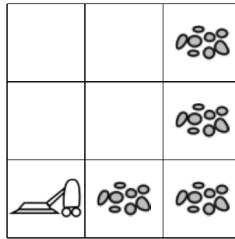
Which of the following heuristics are admissible? Why / Why not? Give clear proof arguments. (Formal proofs are not needed.)

- (i) h_1 = Number of dirty spots.
- (ii) h_2 = Number of clean spots.
- (iii) h_3 = Sum of the distances from the robot to all the dirty spots.
- (iv) $h_4 = h_1 + h_2$.
- (v) h_5 = Minimum distance from the robot to any dirty spot.

Note: To prove a heuristic admissible, you need to show that it is admissible for every state of the illustrated 3×3 example. To show that a heuristic is not admissible, a counter example is sufficient.

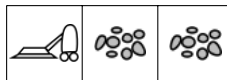
Solution:

- (i) h_1 : Admissible, because to clean one dirty spot it takes at least one action and one action can clean at most one spot. the robot needs to reach at least one dirty spot.
- (ii) h_2 : Not admissible (Not even a heuristic, since $h_2(G) \neq 0$). Consider the following counter example:



Here, $h_2 = 5$, while $h^* = 4$, which makes it not admissible.

- (iii) h_3 : Not admissible. Consider the following counterexample, where $h_3 = 1 + 2 = 3 > 2 = h^*$ (assume that the rest of the board has already been cleaned):



- (iv) h_4 : Not admissible. Consider again the counter example from h_3 . It is $h_5 = h_1 + h_2 = 2 + 1 = 3$, but $h^* = 2$. (Since it incorporates h_2 which is not a heuristic, the combination is also not admissible).
- (v) h_5 : Admissible, because the robot needs to reach at least one dirty spot.

Exercise 4 :

Consider a robot that can move in the grid below. The cost of moving from one cell to another is equal to the number of steps required for the move, where by a single step the robot can move horizontally or vertically to an adjacent cell (no diagonal steps allowed).

				S			
		X_1				X_5	
	X_2				X_4		
			X_3				

The cells marked X_1 , X_2 , X_3 , X_4 , and X_5 are *cells of interest*, each holding one or two of the symbols a , b , and c :

- X_1 : (a, c)
- X_2 : (b)
- X_3 : (b, c)
- X_4 : (c)
- X_5 : (a)

That is, X_1 holds the symbols a and c . When visiting a cell the robot collects the symbols located at that cell.

Starting in cell S , the robot's task is to visit cells of interest and collect exactly one copy of each of the three symbols (a, b, c) before returning to cell S . The robot is *not* allowed to move to a cell containing, e.g., an a if the robot has previously visited a cell holding an a .

Determine which of the cells of interest to visit and in which order these cells should be visited by the robot, so that the cost of the path traveled (when starting and ending in cell S) is minimized.

1. Define a suitable state and action representation for this problem.
2. Define a heuristic function that at any given state provides an underestimate of the cost of reaching a goal state (i.e., a state where the robot is located in cell S with the symbols (a, b, c)).
3. Show the search tree for this problem as it would be expanded using A* search.

Solution:

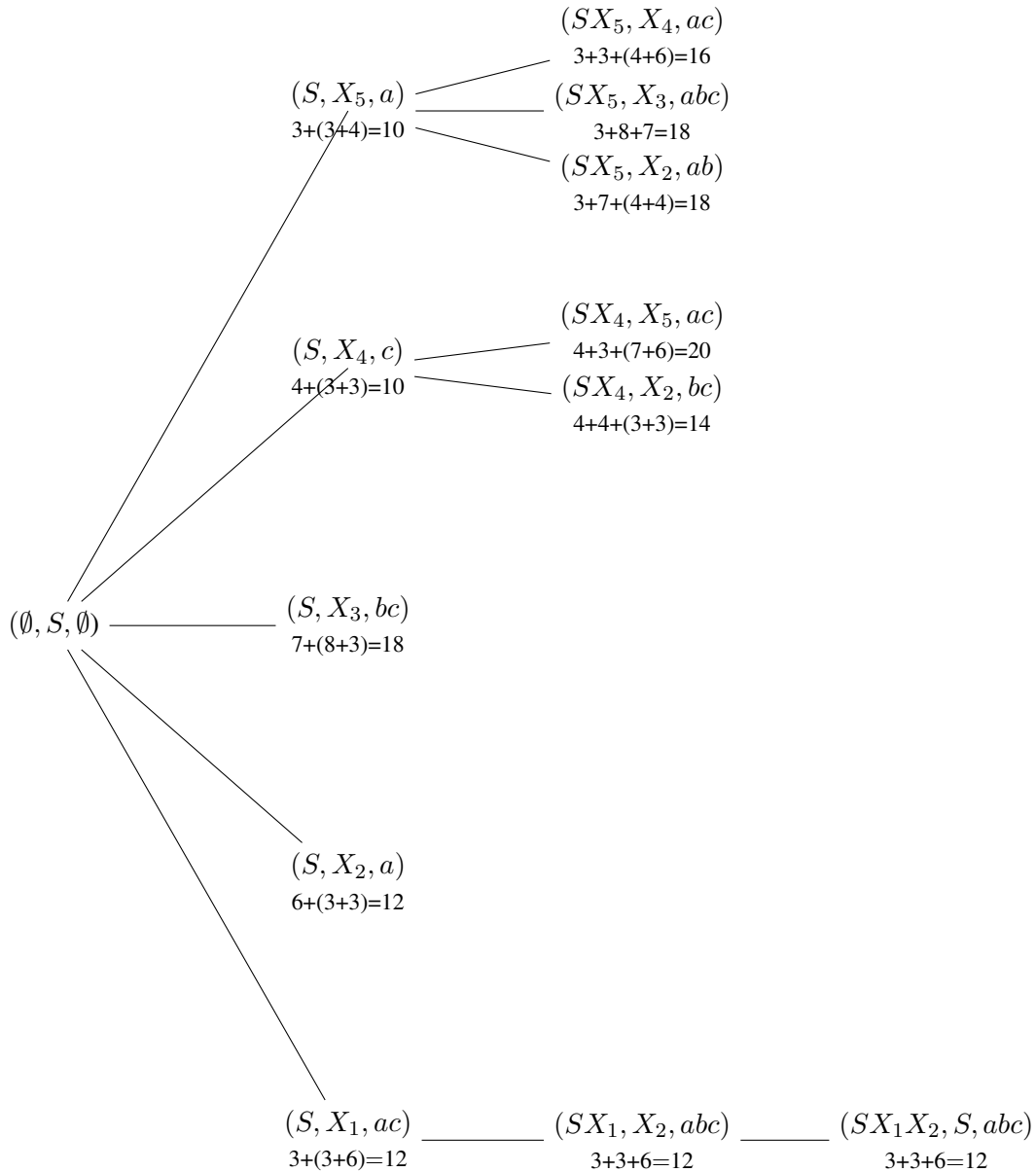
Sub-problem 1

One possible state representation could be $(cell\ history, current\ cell, symbols\ collected)$, where we keep track of the cells previously visited, the current cell the robot is in, and which symbols that have been collected so far. We can, however, also do without including the cell history in the state representation.

Sub-problem 2

One possible heuristic function could be the minimal cost incurred by visiting an admissible cell holding a symbol not yet collected and afterwards returning to S .

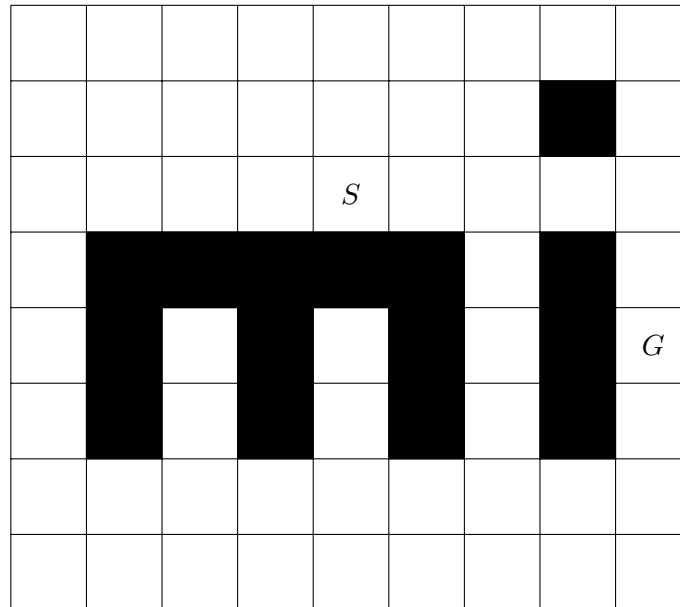
Sub-problem 3



Exercise 5 :

Consider the problem of finding a path from position S to G in the grid shown below. You can only move horizontally and vertically and only one step at a time; no step can be made into the shaded areas or outside

the grid. The cost of the path between two positions is the number of steps on the path.



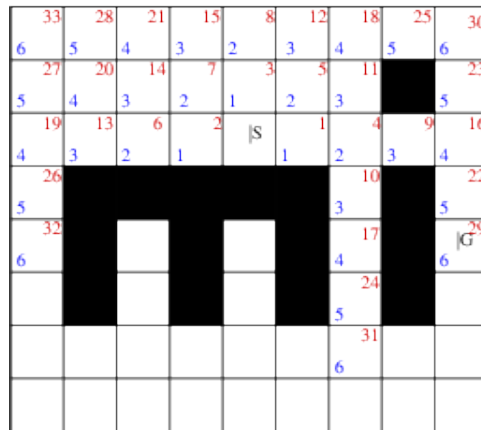
1. Number the positions (cells in the grid) in the order in which they are added to the frontier in a uniform-cost search for G starting at S . Assume that the ordering of the operators is *right*, *down*, *left*, and *up*, and that there is multiple path pruning. When multiple lowest-cost paths exists, choose the path first added to the frontier.
2. Define a heuristic function that underestimates the cost of the lowest-cost path.
3. Number the positions (cells in the grid) in the order in which they are added to the frontier according to an A^* search using your heuristic function. Resolve ambiguities using the operator ordering and the procedure above.

NB: When numbering the positions it may be helpful to also annotate the positions with the cost/function values calculated during the search.

Solution:

Answer 1

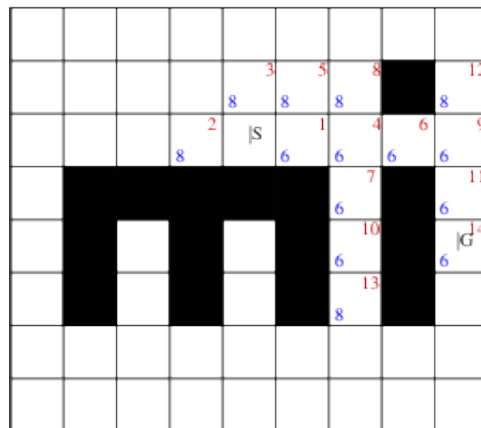
The red numbers specify the order in which the cells are added to the frontier. The blue numbers are the costs associated with the cells.



Answer 2

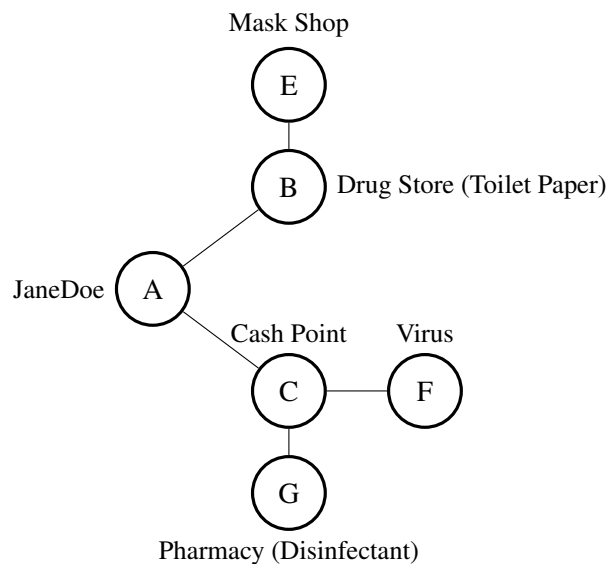
One possible heuristic function simply calculates the cost of the lowest-cost path ignoring the obstacles (shaded regions).

Answer 3



Exercise 1 :

Jane Doe's country was hit by a virus. She wants to fight the virus. In order to do so, she either needs disinfectant, or a mask and toilet paper. Currently, Jane Doe is at home at location *A*, ToiletPaper can be bought at the Drug Store at location *B*, a mask can be bought at location *E*, and disinfectant can be bought at the pharmacy at location *G*. Note that she first needs to withdraw money at the cashpoint at location *C*. The problem was formalized as (*P* not given explicitly):



$$I = \{at(A)\}$$

$$G = \{VirusDead\}$$

$$A = \{buyDisinfectant, buyMask, buyToiletPaper, withdrawMoney, \\ useToiletPaper, useDisinfectant\} \\ \cup \{walk(x, y) \mid x, y \in \{A, \dots, G\} \text{ are connected on the map}\}$$

walk(x, y) :
pre : { *at*(x) }
add : { *at*(y) }
del : { *at*(x) }

buyDisinfectant :
pre : { *at*(G), *has2Money* }
add : { *hasDisinfectant* }
del : { *has2Money* }

buyMask2 :
pre : { *at*(E), *has2Money* }
add : { *hasMask*, *has1Money* }
del : { *has2Money* }

buyMask1 :
pre : { *at*(E), *has1Money* }
add : { *hasMask* }
del : { *has1Money* }

buyToiletPaper2 :
pre : { *at*(B), *has2Money* }
add : { *hasToiletPaper*, *has1Money* }
del : { *has2Money* }

buyToiletPaper1 :
pre : { *at*(B), *has1Money* }
add : { *hasToiletPaper* }
del : { *has1Money* }

withdrawMoney :
pre : { *at*(C) }
add : { *has2Money* }
del : { *has1Money* }

useDisinfectant :
pre : { *at*(F), *hasDisinfectant* }
add : { *VirusDead* }
del : { *hasDisinfectant* }

useToiletPaper :
pre : { *at*(F), *hasToiletPaper*, *hasMask* }
add : { *VirusDead* }
del : { *hasToiletPaper* }

- a) Give an optimal plan for Π . What is the value of $h^*(I)$?
- b) Compute h^{FF} for the initial state. Write down the action and fact sets (A_i and F_i) for each iteration of the RPG algorithm. In the plan extraction, for each step t , write down the fact set G_t , which actions are selected for each fact in G_t , and the resulting fact set updates for each selected action. The order in which you select the actions that generate the facts is not relevant. **Try to avoid using the disinfectant during plan extraction.** What is the value of $h^{FF}(I)$?
- c) Run A* search on the problem using h^+ as the heuristic. If several nodes have the same $g + h$ value, rely on the heuristic and **choose the one with the smaller h value**. If there are several nodes with same g and same h value, break ties by using the alphabetical order on location names. In each search node, mention the facts that are true, the g and h values as well as the expansion order. In the search graph, mark duplicate nodes (e.g. by crossing them out).

Note: Use the following abbreviations.

$h2M := has2Money$, $hD := hasDisinfectant$, $VD := VirusDead$

Solution:

a) i) $walk(A, C) \rightarrow withdrawMoney \rightarrow walk(C, G) \rightarrow buyDisinfectant \rightarrow walk(G, C) \rightarrow walk(C, F) \rightarrow useDisinfectant$

ii) $h^*(I) = 7$

b) Forward search to compute the RPG:

- $F_0 = I = \{at(A)\}$
- $A_0 = \{walk(A, B), walk(A, C)\}$
 $F_1 = F_0 \cup \{at(B), at(C)\}$
- $A_1 = A_0 \cup \{walk(B, E), walk(B, A), walk(C, A), walk(C, F), walk(C, G), withdrawMoney\}$
 $F_2 = F_1 \cup \{at(E), at(F), at(G), has2Money\}$
- $A_2 = A_1 \cup \{walk(E, B), walk(F, C), walk(G, C), buyToiletPaper2, buyMask2, buyDisinfectant\}$
 $F_3 = F_2 \cup \{hasToiletPaper, hasMask, hasDisinfectant, has1Money\}$
- $A_3 = A_2 \cup \{useToiletPaper, useDisinfectant, buyToiletPaper1, buyMask1\}$
 $F_4 = F_3 \cup \{VirusDead\} \supseteq G \rightarrow stop$

Backward search to extract a plan:

Initially: $G_0, G_1, G_2, G_3 = \emptyset, G_4 = \{VirusDead\}$

Layer	Selected fact	Inserted Action	Marked facts
4	<i>VirusDead</i>	<i>useToiletPaper</i>	$G_4 = \{VirusDead\}$
3	<i>hasToiletPaper</i>	<i>buyToiletPaper2</i>	$G_3 = \{hasToiletPaper, hasMask\}$
	<i>hasMask</i>	<i>buyMask2</i>	
2	<i>has2Money</i>	<i>withdrawMoney</i>	$G_2 = \{has2Money, at(E), at(F)\}$
	<i>at(E)</i>	<i>walk(B, E)</i>	
	<i>at(F)</i>	<i>walk(C, F)</i>	
1	<i>at(B)</i>	<i>walk(A, B)</i>	$G_1 = \{at(B), at(C)\}$
	<i>at(C)</i>	<i>walk(A, C)</i>	

$h^{FF}(I) = 8$ since eight actions were selected during plan extraction.

c) The search graph is depicted in figure 1.

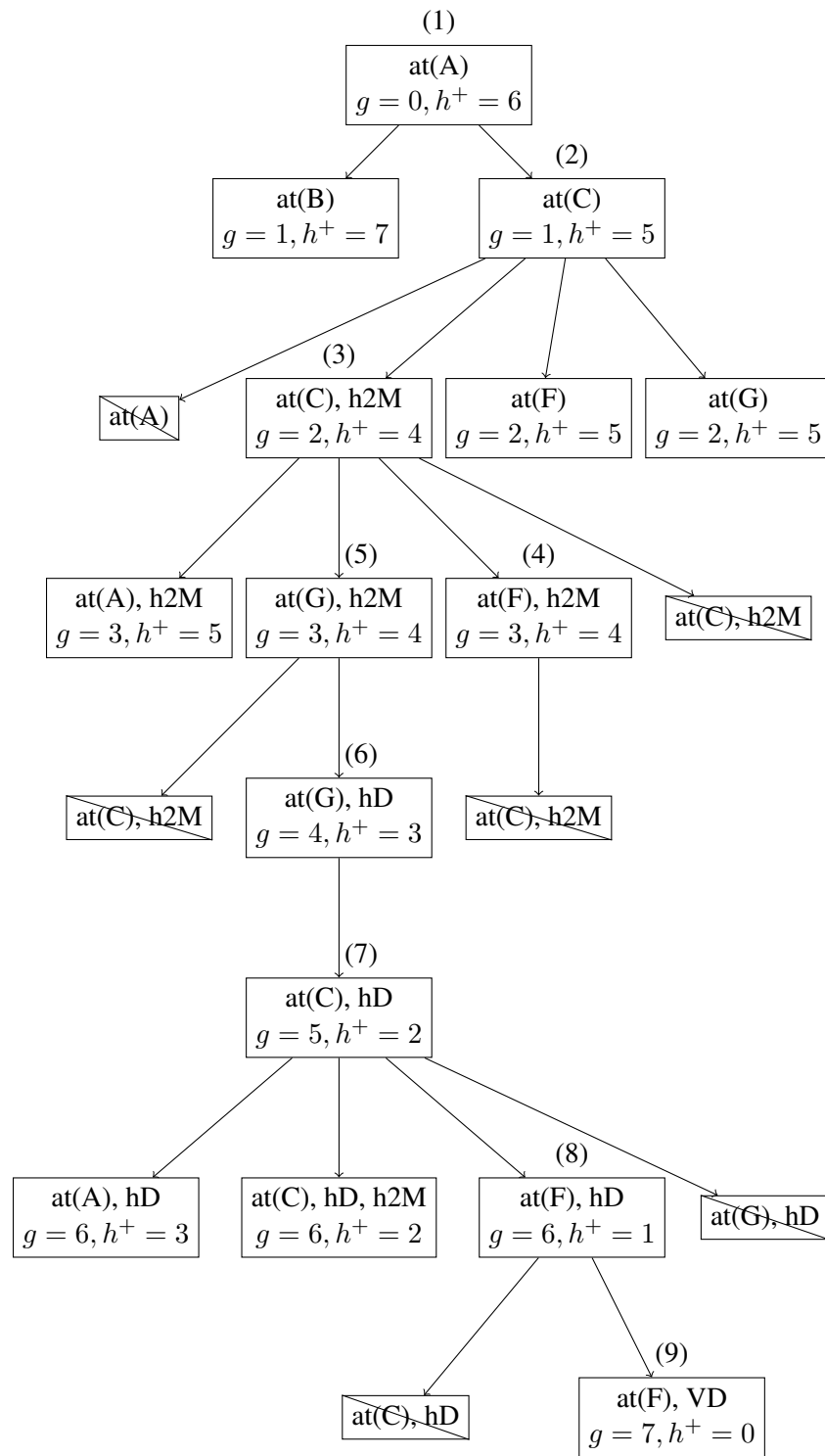


Figure 1: The search graph of part (b).

Exercise 2 :

The owl Steffi wants to deliver a letter containing the result of the student council elections from the CS student council to the AStA building. After delivering the results, she wants to go back to her nest in the student council. Steffi is currently located at the Mensa building. She can fly between any two locations.

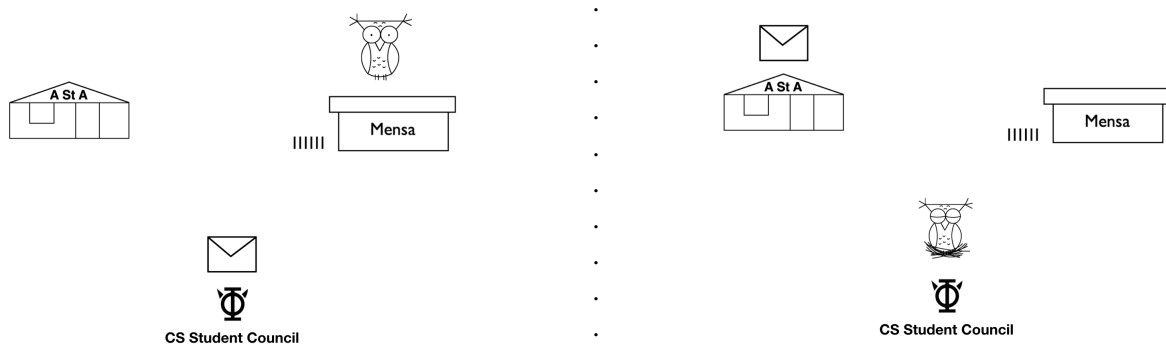
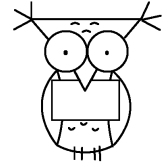


Figure 2: Initial state (left) and goal (right) of the planning task

- (i) Complete the formalization of the planning task. “Empty” stands for Steffi not carrying anything. Fill in all gaps indicated by a question mark (?). You may introduce new predicates if necessary.

$$P = \{at(Steffi, x), at(Letter, x), carrying(Letter), empty\}$$

$$\text{for } x \in \{Mensa, AStA, Council\}$$

$$I = \{at(Steffi, Mensa), at(Letter, Council), empty\}$$

$$G = \{?\}$$

$$A = \{fly(x, y) \mid (x, y) \in \{?\}\}$$

$$\cup \{pickup(i, x) \mid x \in \{?\}, i \in \{?\}\}$$

$$\cup \{drop(i, x) \mid x \in \{?\}, i \in \{?\}\}$$

- $fly(x, y) : \text{pre: } \{?\}, \text{add: } \{?\}, \text{del: } \{?\}$
- $pickup(i, x) : \text{pre: } \{?\}, \text{add: } \{?\}, \text{del: } \{?\}$
- $drop(i, x) : \text{pre: } \{?\}, \text{add: } \{?\}, \text{del: } \{?\}$

- (ii) Give an optimal plan for the task. What is the value of $h^*(I)$?

Solution:

$$(i) I = \{at(Steffi, Mensa), at(Letter, Council), empty\}$$

$$G = \{at(Letter, AStA), at(Steffi, Council)\}$$

$$\begin{aligned} A = & \{fly(x, y) \mid x, y \in \{Mensa, AStA, Council\}, x \neq y\} \\ & \cup \{pickup(i, x) \mid x \in \{Mensa, AStA, Council\}, i \in \{Letter\}\} \\ & \cup \{drop(i, x) \mid x \in \{Mensa, AStA, Council\}, i \in \{Letter\}\} \end{aligned}$$

- $fly(x, y) :$

$$pre : \{at(Steffi, x)\}$$

$$add : \{at(Steffi, y)\}$$

$$del : \{at(Steffi, x)\}$$

for all $x, y \in \{Mensa, AStA, Council\}, x \neq y$

- $pickup(i, x) :$

$$pre : \{at(i, x), at(Steffi, x), empty\}$$

$$add : \{carrying(i)\}$$

$$del : \{at(i, x), empty\}$$

for all $x \in \{Mensa, AStA, Council\}, i \in \{Letter\}$

- $drop(i, x) :$

$$pre : \{carrying(i), at(Steffi, x)\}$$

$$add : \{at(i, x), empty\}$$

$$del : \{carrying(i)\}$$

for all $x \in \{Mensa, AStA, Council\}, i \in \{Letter\}$

- (ii) $\pi = \langle fly(Mensa, Council), pickup(Letter, Council), fly(Council, AStA),$
 $drop(Letter, AStA), fly(AStA, Council) \rangle$
 $h^*(I) = 5$

Exercise 3 :

Consider the following planning task, in which we have only one letter, Steffi is already holding the letter and Steffi can only move between the AStA and the Student Council:

$$P = \{atS(C), atS(A), atL(C), atL(A), atL(S)\}$$

$$I = \{atS(C), atL(S)\}$$

$$G = \{atS(C), atL(A)\}$$

$$A = \{fly(x, y), pickup(x), drop(x)\}, \text{ where } x, y \in \{A, C\}; \text{ and } x \neq y \text{ for } fly(x, y)$$

$$\begin{aligned} fly(x, y) : \quad & pre : \{atS(x)\} \\ & add : \{atS(y)\} \\ & del : \{atS(x)\} \end{aligned}$$

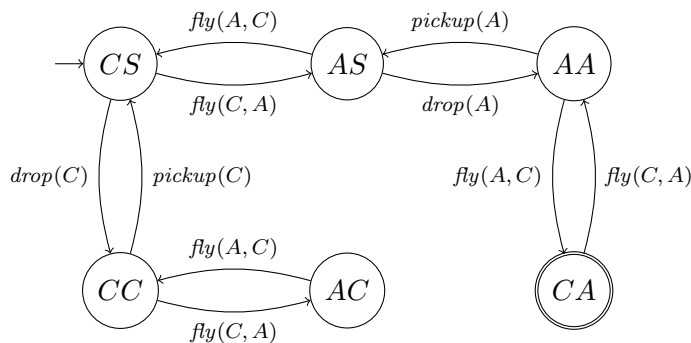
$$\begin{aligned} pickup(x) : \quad & pre : \{atL(x), atS(x)\} \\ & add : \{atL(S)\} \\ & del : \{atL(x)\} \end{aligned}$$

$$\begin{aligned} drop(x) : \quad & pre : \{atL(S), atS(x)\} \\ & add : \{atL(x)\} \\ & del : \{atL(S)\} \end{aligned}$$

- (i) How many states are there in this planning task?
- (ii) Draw the reachable part of the state space. Denote every state by the facts that are currently true, i.e., the current positions of Steffi and the Letter (for example, the initial state can be denoted by CS).
- (iii) How many reachable states are in this planning task?

Solution:

There are 2^5 states, out of which 6 are reachable.



Exercise 4 :

For all of the following statements, decide if they are correct. Briefly justify your answer (1–3 sentences).

1. In the delete relaxation, an optimal plan (if it exists) has length at most $|G|$.
2. Our implementation of h^{FF} is an admissible heuristic.

3. Let $\mathcal{R}$ be a relaxation mapping that drops all preconditions in a given planning task. Then the heuristic defined as $h^{\mathcal{R}} := h^* \circ \mathcal{R}$ is an admissible heuristic.
4. For every state it holds that if it is unsolvable in the original problem, it is unsolvable in the delete relaxation.
5. For every state it holds that if it is solvable in the original problem, it is solvable in the delete relaxation.
6. Let $\Pi = (P, A, I, G)$ be a STRIPS planning task with empty delete lists, i.e. $\text{del}_a = \emptyset$ for all action $a \in A$. The task $\Pi' = (P, A, I, G \setminus I)$ in which all facts that are initially true have been removed from the set of goal facts is solvable if and only if Π is solvable.

Solution:

1. False. This was true for the only-adds relaxation, but only because we did not have any preconditions. Consider the following counterexample in which the optimal plan in the delete relaxation has length $2 > |G| = 1$.

$$P = \{p, q\}$$

$$A = \{a_1, a_2\}$$

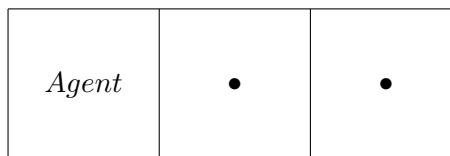
$$I = \{\}$$

$$G = \{q\}$$

$$\text{pre}_{a_1} = \{\}, \quad \text{add}_{a_1} = \{p\}, \quad \text{del}_{a_1} = \{\}$$

$$\text{pre}_{a_2} = \{p\}, \quad \text{add}_{a_2} = \{q\}, \quad \text{del}_{a_2} = \{\}$$

2. False. Our implementation gives us **some** plan for the delete-relaxation, not the shortest. The claim would only hold if h^{FF} always computed the shortest plan in the delete-relaxation, since $h^+(s) \leq h^*(s)$.
3. True. Any plan in the original problem would still be a plan in this relaxation, as all actions are applicable because of absent preconditions.
4. False. Consider this task in which our agent shall be at two locations (•) **at the same time**. He can move left or right if he stays inside the 3 squares.



Obviously this is impossible. However, in the delete relaxation the agent can move to the very right. After doing this, the agent is everywhere in the delete-relaxation, as our previous positions have never been erased. Therefore, $h^+(I) = 2$ for this example.

5. True. Any plan in the original problem is a plan in the delete relaxation.
6. True. Since we have no deletes, the goal facts that were initially true stay true forever. Therefore, any plan in this relaxation is also a plan in the original problem.

Exercise 5 (Optional):

Optional but recommended, e.g., as part of the extra exercise sessions

In this exercise, we consider the problem of collecting objects of different colors, while moving along a road map that takes the form of a directed tree of depth 3 (Figure 3). Each tree node might contain up to three objects having any of three different colors (red, green, blue); moving to that node immediately collects all the objects present.

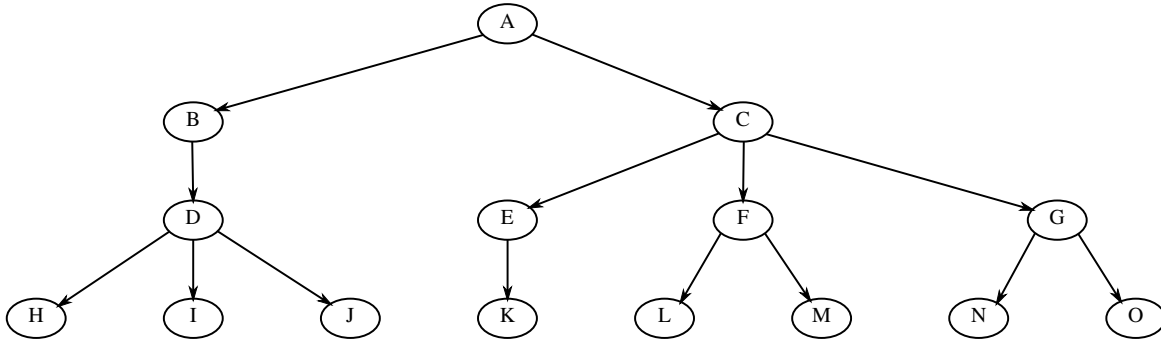


Figure 3: Empty road map

The corresponding STRIPS planning task $\Pi = (P, A, I, G)$ looks as follows:

$$\begin{aligned}
 P &= \{at(x) \mid x \in \{A, \dots, O\}\} \cup \{got(c) \mid c \in \{red, green, blue\}\} \\
 A &= \{move(x, y) \mid (x, y) \text{ is an edge in the tree}\} \\
 I &= \{at(A)\} \\
 G &= \{at(J), got(red), got(green), got(blue)\}
 \end{aligned}$$

The goal is to be at location J , while having the three colors.

Here, the action $move(x, y)$ is specified as follows:

- In case that the destination y does not contain any objects: $move(x, y) =$

$$\begin{aligned}
 pre &: \{at(x)\} \\
 add &: \{at(y)\} \\
 del &: \{at(x)\}
 \end{aligned}$$

for all x, y in the tree with an edge from x to y .

- In case that the destination y does contain objects (shown here for red and blue; other cases are defined accordingly): $move(x, y) =$

$$\begin{aligned}
 pre &: \{at(x)\} \\
 add &: \{at(y), got(red), got(blue)\} \\
 del &: \{at(x)\}
 \end{aligned}$$

for all x, y in the tree with an edge from x to y , where y contains both a red object and a blue object.

Note in the figure that the edges are directed, i.e., you can pass them only in one direction. Note further that any colors placed in A will not be taken (there is no move action going to A).

Your task is to give three different colorings of the road map. Such a coloring assigns each node of the road map up to three colors (it can have none, any of the three colors, any two of them, or all three). Then, you will use Greedy Best-First Search (GBFS) with the goal counting heuristic and lexicographic tie-breaking (so, if there are multiple nodes with the same h value, the one with a letter coming first in the alphabet will be selected for expansion). The goal is to specify a coloring so that GBFS returns a solution after expanding a specified number of states. For this note that if a goal state is to be expanded next, the search stops without generating any successors, so that we do not count this as an expansion.

For each of the parts (a), (b), and (c) do the following: (i) draw the road map with your coloring and (ii) draw the search space, showing all states generated during the search, including an edge from each expanded state to the child nodes generated when expanding it. Identify each state s by the name of the respective graph node (the node x for which “ $at(x)$ ” is true in s), as well as the subset of colors already collected in the state (those colors x for which “ $got(x)$ ” is true in s). Annotate each state with its heuristic value, as well as a number indicating the expansion order. (For generated states that are not expanded, use “–” as the expansion order number.)

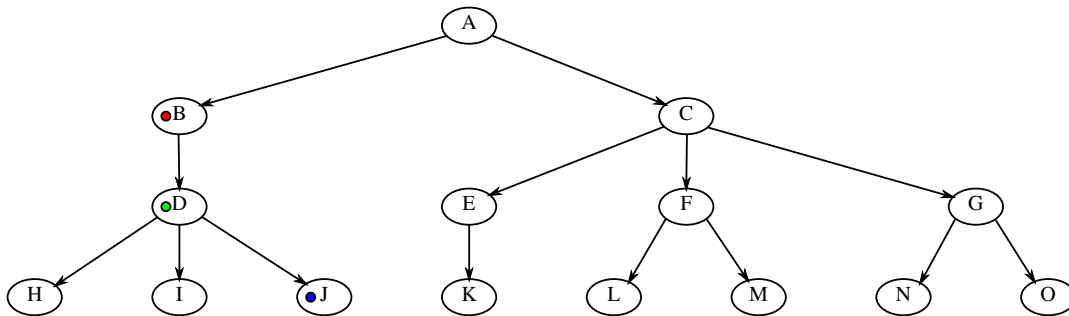
The goal counting heuristic is defined as follows: For state s , $h^{gc}(s)$ is the number of goal facts $p \in G$ that are not true in s . (In other words: $h^{gc}(s) = |G \setminus s|$.)

- (a) Color the graph and perform Greedy Best-First Search, so that it expands exactly three states.
- (b) Color the graph and perform Greedy Best-First Search, so that it expands at least six and at most nine states.
- (c) Color the graph and perform Greedy Best-First Search, so that it expands exactly twelve states.

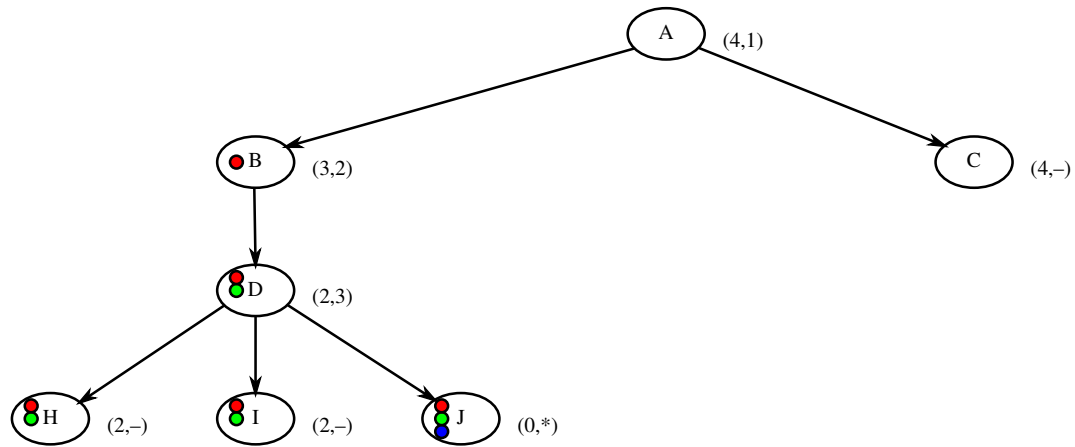
Solution:

All nodes in the following search space graphs are annotated with a pair of numbers (h, e) , where h is the heuristic value and e the number of that node in the expansion order.

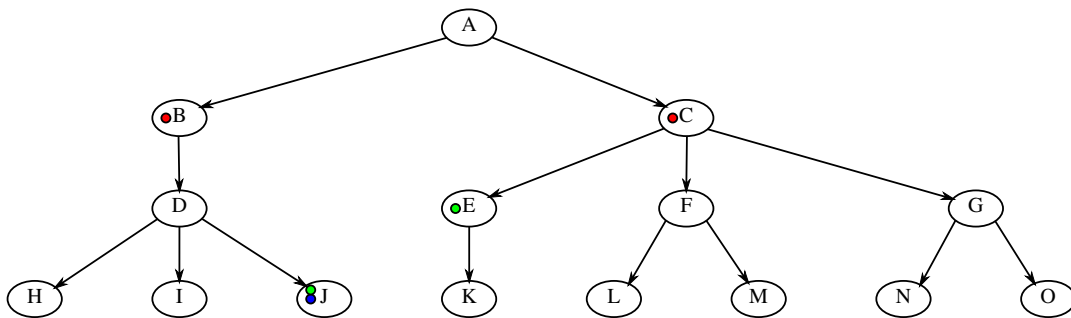
- (a) Example coloring giving three expansions:



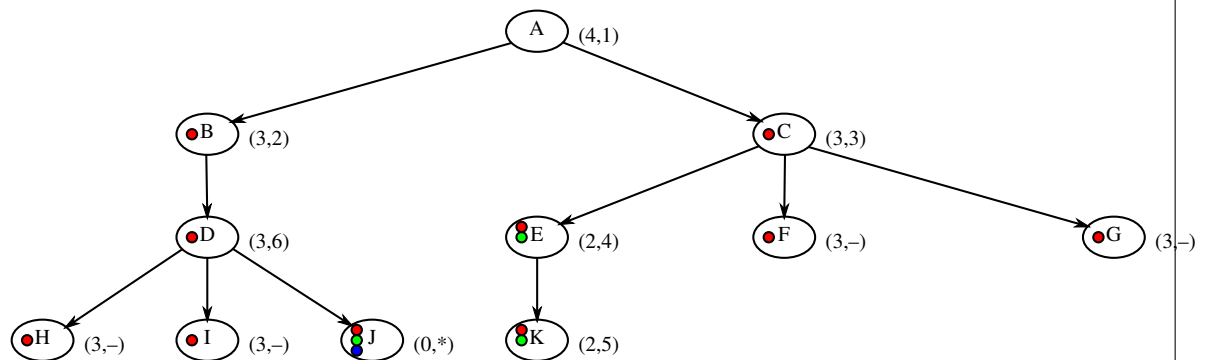
Corresponding search space of greedy best-first search using h^{gc} :



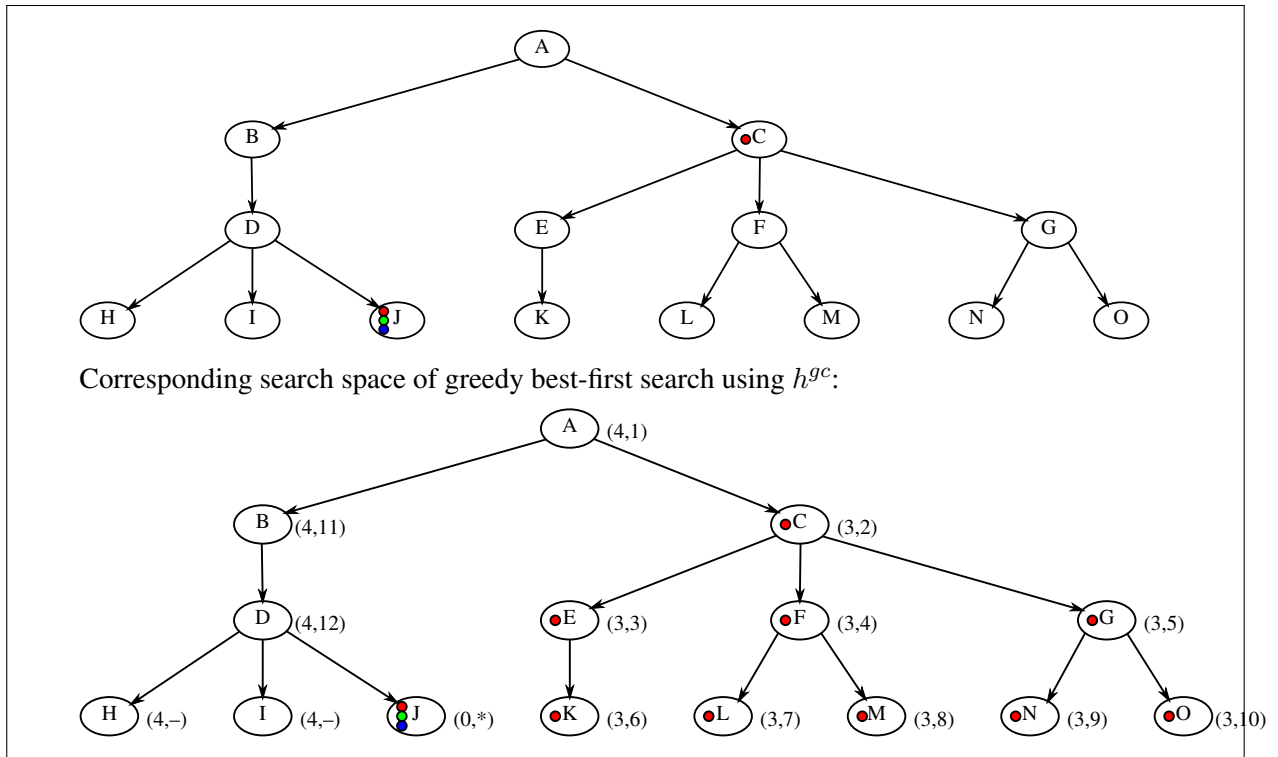
(b) Example coloring giving six expansions:



Corresponding search space of greedy best-first search using h^{gc} :



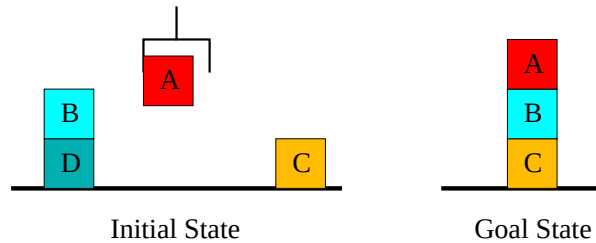
(c) Example coloring giving twelve expansions:



Exercise 6 (Optional):

Optional but recommended, e.g., as part of the extra exercise sessions

Consider the following Blocksworld task:



The problem is formalized as the following STRIPS planning problem $\Pi = (P, A, I, G)$:

$$\begin{aligned}
 P = & \{stacked(x, y) \mid x, y \in \{A, B, C, D\}\} \\
 & \cup \{ontable(x) \mid x \in \{A, B, C, D\}\} \\
 & \cup \{clear(x) \mid x \in \{A, B, C, D\}\} \\
 & \cup \{holding(x) \mid x \in \{A, B, C, D\}\} \\
 & \cup \{empty\}
 \end{aligned}$$

$$\begin{aligned}
 A = & \{stack(x, y), unstack(x, y) \mid x, y \in \{A, B, C, D\}, x \neq y\} \\
 & \cup \{putdown(x), pickup(x) \mid x \in \{A, B, C, D\}\}
 \end{aligned}$$

where

- $stack(x, y)$ for $x, y \in \{A, B, C, D\}, x \neq y$
 - $pre : \{holding(x), clear(y)\}$
 - $add : \{stacked(x, y), clear(x), empty\}$
 - $del : \{holding(x), clear(y)\}$
- $unstack(x, y)$ for $x, y \in \{A, B, C, D\}, x \neq y$
 - $pre : \{stacked(x, y), clear(x), empty\}$
 - $add : \{holding(x), clear(y)\}$
 - $del : \{stacked(x, y), clear(x), empty\}$
- $putdown(x)$ for $x \in \{A, B, C, D\}$
 - $pre : \{holding(x)\}$
 - $add : \{ontable(x), clear(x), empty\}$
 - $del : \{holding(x)\}$
- $pickup(x)$ for $x \in \{A, B, C, D\}$
 - $pre : \{ontable(x), clear(x), empty\}$
 - $add : \{holding(x)\}$

– $del : \{ontable(x), clear(x), empty\}$

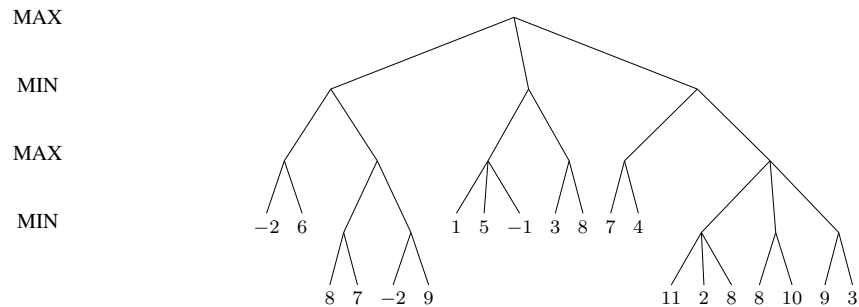
- a) Provide I and G , according to the figure (we do not consider $clear(A)$ part of the goal).
 - b) Provide an optimal plan, what is the value of $h^*(I)$?
 - c) Provide an optimal delete-relaxed plan, what is the value of $h^+(I)$?
-

Solution:

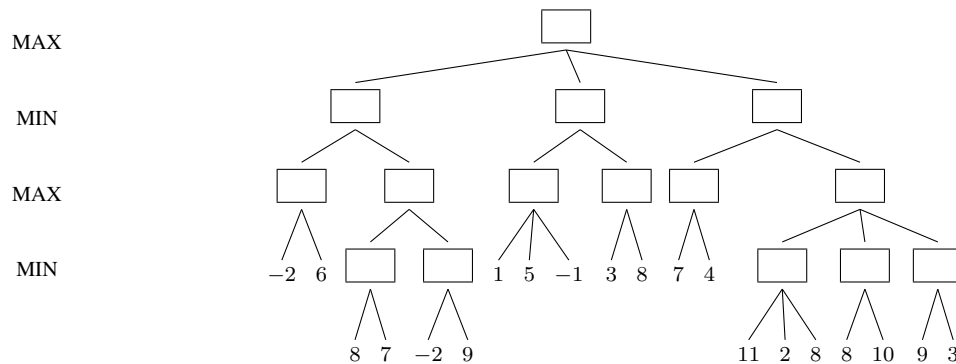
- a) $I = \{ontable(D), stacked(B, D), clear(B), ontable(C), clear(C), holding(A)\}$,
 $G = \{stacked(A, B), stacked(B, C), ontable(C)\}$ ($ontable(C)$ would be optional)
- b) Optimal plan: $\langle putdown(A), unstack(B, D), stack(B, C), pickup(A), stack(A, B) \rangle$. $h^*(I) = 5$
- c) Optimal relaxed plan: $\langle stack(A, B), unstack(B, D), stack(B, C) \rangle$. $h^+(I) = 3$

Exercise 1 :

Consider the following game tree corresponding to a two-player zero-sum game as specified in the lecture. As usual, **Max is to start in the initial state (i.e., the root of the tree)**. For the following algorithms, the expansion order is **from left to right, i.e., in each node the left-most branch is expanded first**.



1. In the following tree, perform Minimax search, i.e., annotate all internal nodes with the correct Minimax value.

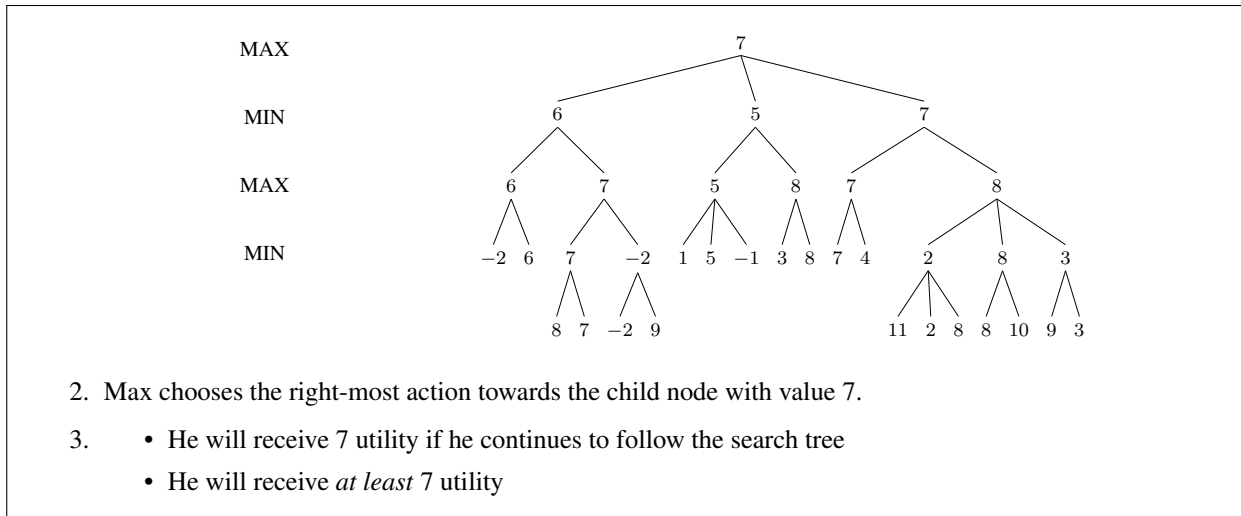


2. Which move does Max choose?
3. Consider, again, our game tree given above. Max chooses the action with the highest utility in the root of the tree. What can you say about the utility he will receive against:
 - An optimally playing opponent
 - A non-optimal playing opponent

Note: Max always plays optimally.

Solution:

1. The Minimax tree:



Exercise 2 :

Max and Minnie are playing a classic children's game called **Tic-tac-toe** in which players take turns marking the cells in a 3×3 grid. Max marks Xs and Minnie marks Os. Minnie wins with utility -1 if **any line (horizontal, vertical, or diagonal)** fills up with three Os, whereas Max wins with utility $+1$ if **any line** fills up with three Xs. If there are no empty cells left and no one has won so far, the game ends in a draw with utility 0 .

(a) Consider the state depicted below. Here, it is Max's turn to play.

x		o
x	o	
		o

Draw the **full** Minimax tree and annotate every node with its utility.

(b) Consider the evaluation function $f(x) :=$ the number of (horizontal) rows which contain at most one O. For example, in the initial state $f(x) = 3$.

Draw the Minimax tree with this evaluation function and a depth of 2. Annotate every node with its utility.

(c) Assuming a perfectly playing opponent, Minimax search without a depth limit will always guarantee a draw. However, it is not obvious whether this guarantee can still be made when imposing certain depth limits in combination with certain evaluation functions. As an example, consider the simple evaluation function g aimed to detect situations in which the opponent can win in one step (if it is their turn).

$$g(x) := \left\{ \begin{array}{ll} -1, & \text{if there is a line (horizontal, vertical, or diagonal) with} \\ & \text{two opponent marks and an empty field} \\ 0, & \text{otherwise} \end{array} \right\}$$

Now prove or disprove the following claim:

Running Minimax search with a depth limit of 3 and evaluation function g is sufficient to guarantee a draw against a perfectly playing opponent in a 3×3 Tic-tac-toe game. You may **not** assume that you are allowed to start the game.

Solution:

1. See Figure 1.

MAX

x		o
x	o	
		o

+1

MIN

x	x	o
x	o	
		o

-1

See Figure 2

x		o
x	o	x
		o

-1

See Figure 3

x		o
x	o	
x		o

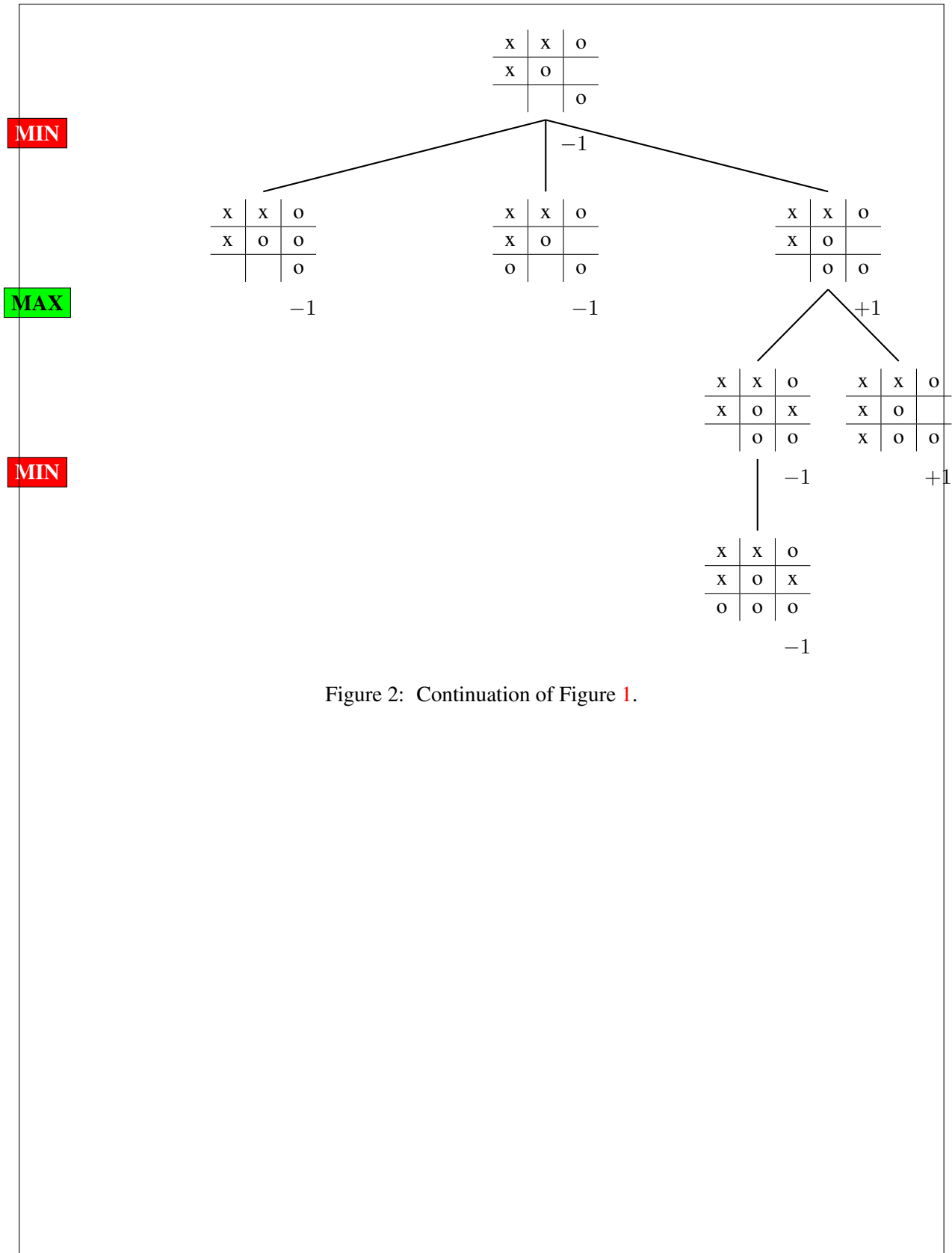
+1

x		o
x	o	
	x	o

-1

See Figure 4

Figure 1: Solution of Exercise 1 (a).



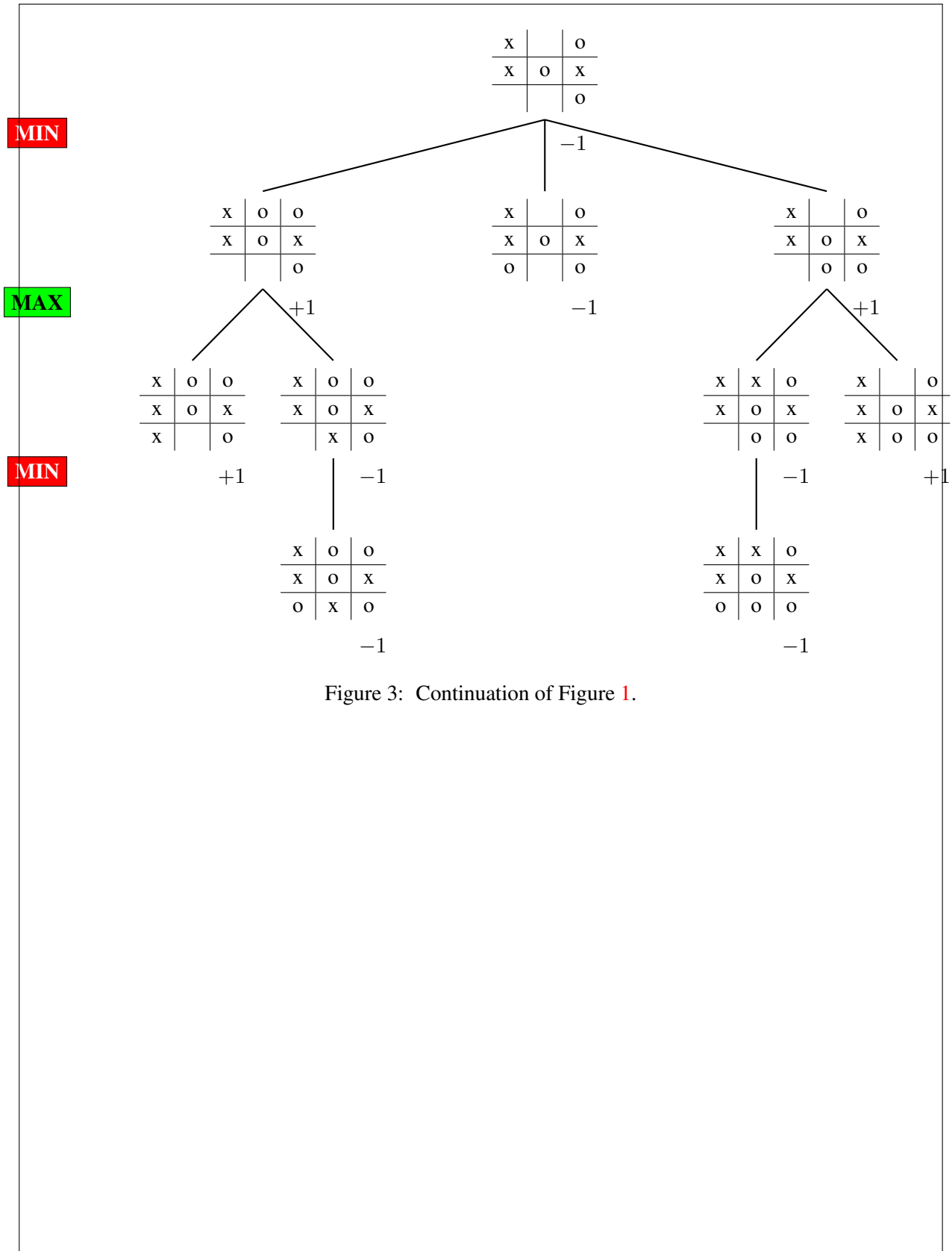


Figure 3: Continuation of Figure 1.

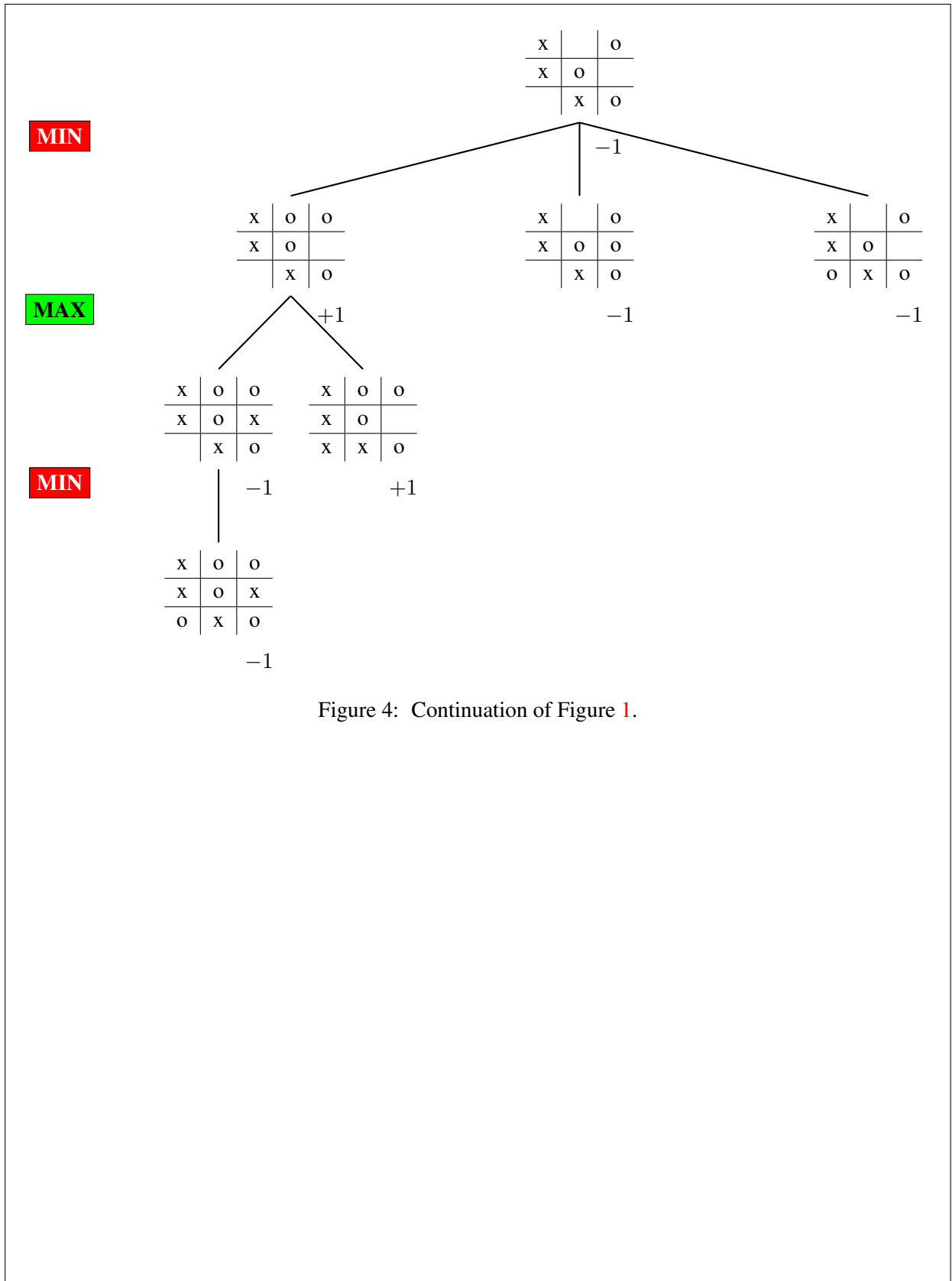


Figure 4: Continuation of Figure 1.

2. See Figure 5.

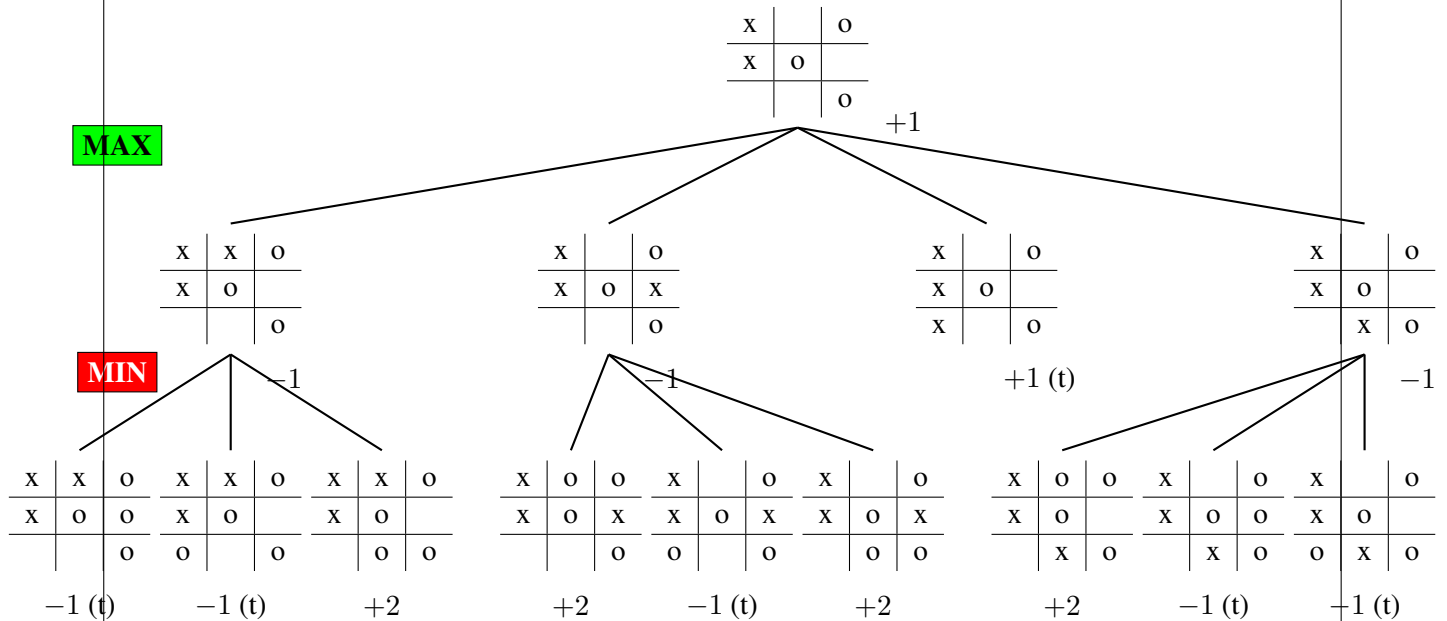
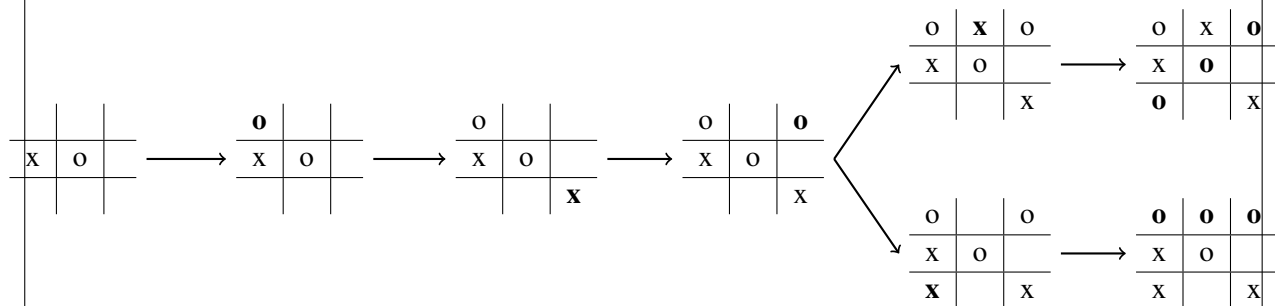


Figure 5: Solution of Exercise 1 (b). Nodes marked with (t) are terminal nodes.

3. The claim is false. We prove this on the basis of the following counterexample.

Let us assume we are playing as Max and our opponent Minnie has begun the game by placing an O in the centre. Up to symmetry, there are only two ways in which we can respond: We can place an X in a corner or we can place the X in the middle of a row or column. If we do not mark a corner, Minnie has a winning strategy, which is depicted below.



Thus, to avoid the risk of being defeated, Minimax search must recognize this failing move. In our case this means that the utility of placing an X in the corner must be strictly greater than the one for failing to do so. As we cannot win the game within four (half-)moves (because we need 3 own marks), the utility value for putting an X in the corner is ≤ 0 . Therefore the utility value for putting the X somewhere else must be -1 .

However, it is easy to see that Minimax search starting in

x		
	o	

 with a depth limit of 2

(because we have already made the first move) will produce a utility value of 0. This is because, no matter where Minnie puts her next O, there is always at most one line with 2 Os and an empty field after the next step. Thus Max can force a state in which g evaluates to 0 simply by putting an X in that respective line. s

Exercise 3 :

Please decide for each of the following statements whether it is true or false and justify your answer (1-3 sentences per statement).

1. A two-player zero-sum game has exactly three possible outcomes in terms of its utility function.
2. Full Minimax search always yields the best possible outcome in terms of its utility, no matter how the opponent plays.
3. Zero-sum games always have a Nash Equilibria
4. Zero-sum games always have a Nash Equilibria consisting of pure strategies
5. Non-Zero-sum games always have a Nash Equilibria

Solution:

1. False. In some two-player zero sum games there is a larger variety of possible outcomes (e.g. the payoffs in backgammon range from +192 to -192).
2. False. It yields the best possible outcome assuming the opponent plays perfectly.
3. True. All games have at least one Nash Equilibrium if we consider mixed strategies
4. False. Rock-Paper-Scissors is a zero-sum game, but it does not have a Nash Equilibria with pure strategies
5. True. All games have at least one Nash Equilibrium if we consider mixed strategies

Exercise 4 :

Consider the following game representation in normal form:

Barb	Andy		
	a_1	a_2	a_3
b_1	2 0	1 0	2 2
b_2	2 0	1 1	0 0
b_3	2 1	0 0	0 2
b_4	2 0	0 0	0 2
b_5	0 0	1 1	0 2
b_6	0 0	1 1	0 0

The matrix shows the utilities for Andy (red numbers) and Barb (green numbers) for each combinations of strategies they can choose (Andy has 3 strategies to choose from, Barb has 6).

- Determine at least two Nash equilibria consisting of pure strategies for this game.

- Show that there is no Nash equilibrium where Barb plays b_4 , and Andy plays any (possibly mixed) strategy.

Solution:

Part a

(a_1, b_3) and (a_2, b_5)

Part b

If Barb plays b_4 and Andy plays (p_1, p_2, p_3) , the expected utility for Barb is

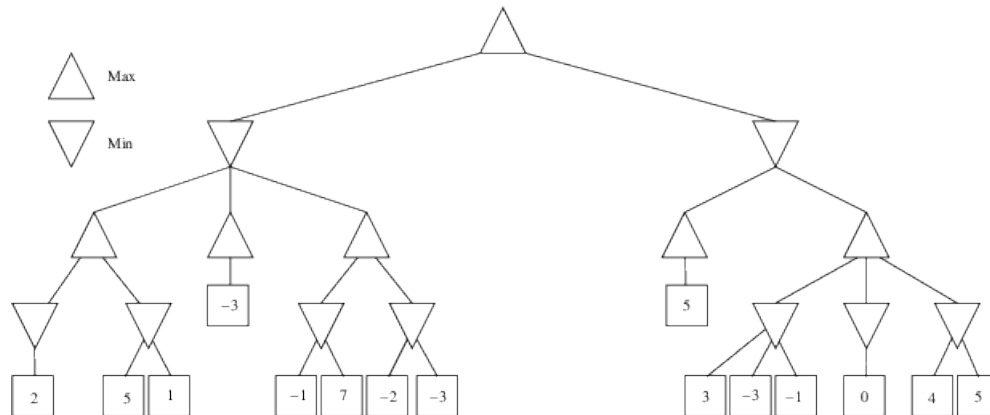
$$EU(\text{Barb}) = p_1 \cdot 0 + p_2 \cdot 0 + p_3 \cdot 2 = 2 \cdot p_3$$

However, if Barb switches to b_3 her expected utility becomes

$$EU(\text{Barb}) = p_1 \cdot 1 + p_2 \cdot 0 + p_3 \cdot 2 = 1 \cdot p_1 + 2 \cdot p_3.$$

Exercise 5 :

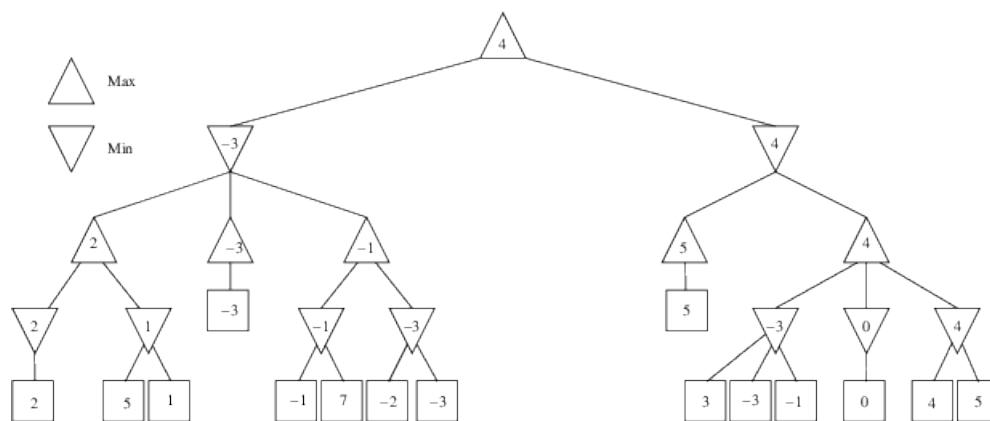
Consider the following game tree:



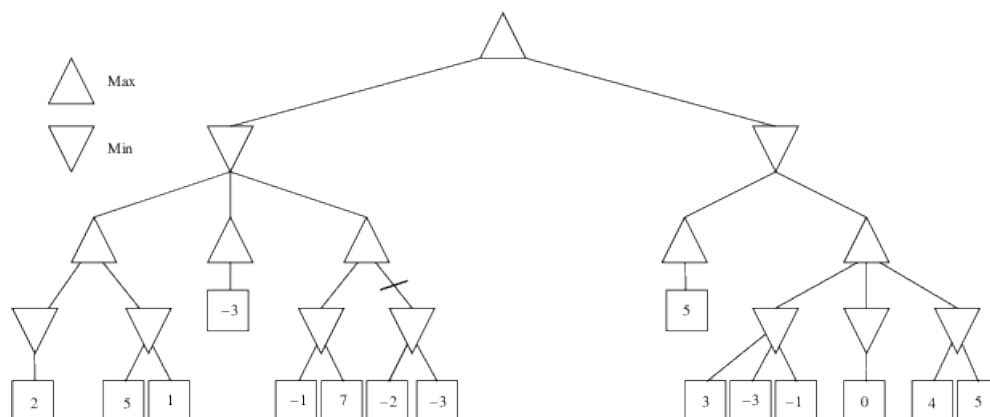
- Compute the utility values for all nodes.
- If the utility values are computed in a depth-first order that always considers branches in left-to-right order, which nodes can be pruned, i.e. for which nodes is it not required to compute the utility value in order to determine the optimal strategy for both players?
- For each node in the game tree, determine the ordering of the outgoing branches that is optimal in the following sense: if utility values are computed for nodes in that order, then a maximal number of nodes can be pruned in the utility computation.

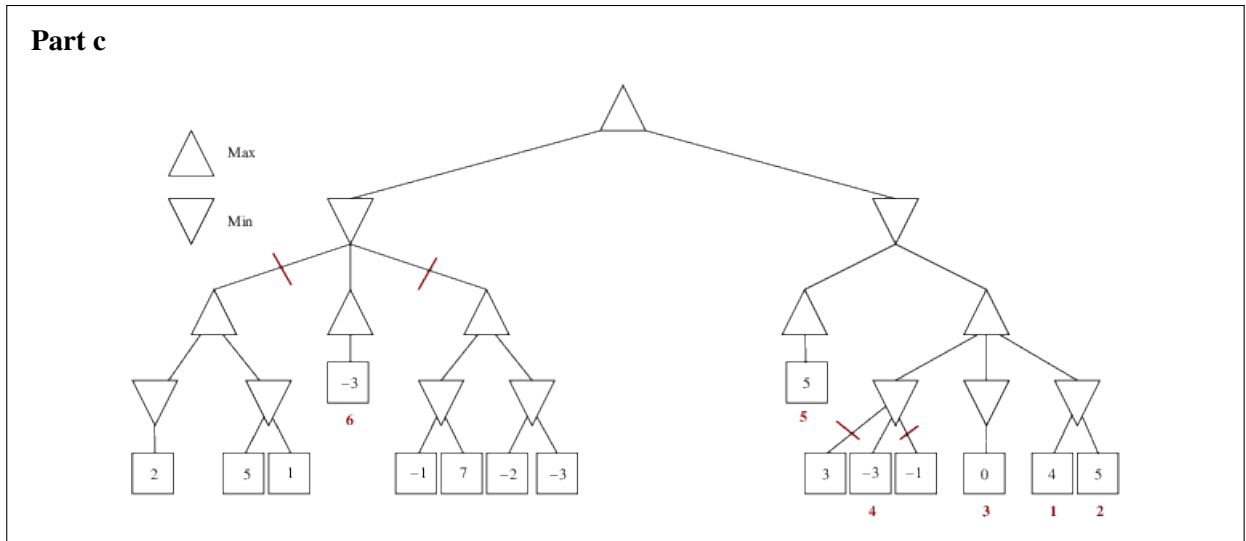
Solution:

Part a



Part b



Part c

Exercise 1 :

Consider the following variation of the example from the lecture:

	1	2	3
1			
2			
3			

The adventurer still may slip to the right with 20% probability, as explained in the lecture slides. The only difference is that the reward is 1 point for the gold, -20 points for the snake, and 0 everywhere else.

- Compute the first 3 iterations of value iteration with $\gamma = 0.9$
- Consider the value function after the 3th iteration. Does the optimal policy change with respect to the version in the lecture? Justify your answer.

Solution:

	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

Iteration 0:

	1	2	3
1	0	0	1
2	-0	0	-20
3	0	0	0

Iteration 1:

	1	2	3
1	0	0.9	1
2	0	-3.6	- 22.88
3	0	0	0

Iteration 2:

	1	2	3
1	0.81	0.9	1
2	-0.65	-3.47	- 23.40
3	0	0	0

Iteration 3:

The optimal policy will always remain in the bottom row and never attempt to get the gold. This is different as in the lecture, where the optimal policy was to get the gold going as far from the snake as possible. This makes sense, as here, the adventurer is more scared of the snake so it won't risk her life to get the gold.

Exercise 2 :

Consider the following variation of the example from the lecture:

	1	2	3
1			
2			
3			

But now the snake will move to an adjacent square every time. The following rules apply:

- If the adventurer is where the snake is, the snake will remain there. In that case, the adventurer cannot move with probability 0.5, else it moves where they wanted to.
- If the adventurer is next to the snake, then the snake will jump to the adventurer with probability 0.5 (and the adventurer cannot move) or remain where it is.

- If the adventurer is not next to the snake, then the snake will move at random (0.25 percent north, 0.25 percent south, and 0.5 remains where it is).
- As in the original problem, when the adventurer moves there is a small chance (20%) that she falls to the right.

Getting the gold provides the adventurer 10 points. Being in the same square as the snake provides -5 points. Everything else provides -0.1 points. Formalize this problem as an MDP. Provide a formal description of the following.

- Set of states
- Actions
- Reward function
- Transition probability function. In particular, describe the possible outcomes for the initial state where we apply the north action.
- Transition probability function. In particular, describe the possible outcomes for the following state where we apply the east action.

	1	2	3
1			
2			
3			

Solution:

- We one state for each position of the adventurer (x_a, y_a) and each position of the snake (x_s, y_s) . We describe each state as a 4-tuple (x_a, y_a, x_s, y_s) . Therefore, the set of states contains 81 elements: $\{(1, 1, 1, 1), (1, 1, 1, 2), \dots\}$
- The actions are as in the original problem: $\{N, E, W, S\}$
- If $(x_a, y_a) = (3, 1)$ then $R(s) = 10$. Otherwise, if $(x_a, y_a) = (x_s, y_s)$, then $R(s) = -5$. Otherwise $R(s) = -0.1$
- The possible outcomes are:
 - $(2, 3, 3, 1)$, with probability $0.2 \cdot 0.25 = 0.05$
 - $(2, 3, 3, 2)$, with probability $0.2 \cdot 0.5 = 0.1$
 - $(2, 3, 3, 3)$, with probability $0.2 \cdot 0.25 = 0.05$

- (1, 2, 3, 1), with probability $0.8 \cdot 0.25 = 0.2$
- (1, 2, 3, 2), with probability $0.8 \cdot 0.5 = 0.4$
- (1, 2, 3, 3), with probability $0.8 \cdot 0.25 = 0.2$

(v) The possible outcomes are:

- (2, 3, 2, 3), with probability 0.5
- (3, 3, 2, 2), with probability 0.5

Exercise 3 :

Consider the game of Tetris: [https://en.wikipedia.org/wiki/Tetris_\(NES_video_game\)](https://en.wikipedia.org/wiki/Tetris_(NES_video_game)).

Formalize this problem as an MDP. Provide a description of the following.

- Set of states
- Actions
- Reward function
- Transition probability function
- Initial state
- Set of terminal states

The description can be just a short text in English textual and not necessarily entirely formal.

Solution:

(i) We can describe a state in terms of the following variables:

- For each cell, a boolean variable indicating whether there is a block there or not.
- Which piece is falling
- The position of the piece that is falling

We have a state for each combination of the values of these variables.

Optionally, one can also model the “next-piece”, as well as the number of lines.

- There are nine actions: $\{\text{moveleft}, \text{don'tmove}, \text{moveright}\} \times \{\text{rotateleft}, \text{don'trotate}, \text{rotateright}\}$.
Optionally, one could also model push-down.
- The reward function corresponds to the score in the game. Every time lines are cleared, the player achieves the corresponding reward.
- The transition function is almost always deterministic. The state can be computed following the rules of the game, where the falling piece is moved according to the player's action. The transition function is not deterministic when deciding the next piece, which is chosen uniformly at random.

- (v) In the initial state the board is clear and no piece has been selected yet.
- (vi) The terminal states are those where the player “dies”, i.e., the next piece does not have any space to be put into. Optionally, one could put a line cap too.

Exercise 1 :

Consider the following variation of the example from the lecture, where we do not know neither the probabilistic transition function or the reward function:

	1	2	3
1			
2			

The coordinates are expressed as pairs (x, y) , where x corresponds to the column and y to the row, and the north west corner is $(1,1)$, as in the Figure.

Apply Q-learning on this example, with a learning rate $\alpha = 0.9$, and a discount factor $\gamma = 0.8$. Specifically:

- (i) Define what is the table that the agent uses for learning. Initialize it to 0.
- (ii) Consider that the agent performs the following sequence of actions. How the table is updated? For each of the steps indicate the new table and how the values are updated.
 - north, go to $(1, 1)$, receive reward of -1
 - east go to $(2, 1)$, receive reward of $+5$
 - east go to $(2, 2)$, receive reward of -2
 - west go to $(1, 2)$, receive reward of $+2$
- (iii) If the agent decides to act greedily, what action would the agent perform next? Justify your answer.

Solution:

0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0

- north, go to $(1, 1)$, receive reward of -1

0	0	0
0	0	0
0	0	0
-0.9	0	0
0	0	0
0	0	0

- east go to (2, 1), receive reward of +5

0	0	0
0	4.5	0
0	0	0
-0.9	0	0
0	0	0
0	0	0

- east go to (2, 2), receive reward of -2

0	0	0
0	4.5	-1.8
0	0	0
-0.9	0	0
0	0	0
0	0	0

- west go to (1, 2), receive reward of +2

0	0	0
0	4.5	-1.8
0	0	0
-0.9	0	0
0	1.8	0
0	0	0

- The agent will apply any of the actions that maximize $Q(s, a)$, so it will choose to move in any direction except north.

Exercise 2 :

Consider an environment with two states $S = \{s_1, s_2\}$ and two actions $A = \{a_1, a_2\}$. The initial state is s_1 , and there are no terminal states.

When we apply action a_1 , we move to state s_1 . In that case, we obtain a reward of $+1$ if we came from s_1 , or $+7$ if we come from s_2 .

When we apply action a_2 we move to s_2 . In that case, we obtain a reward of -2 if we come from s_1 , or $+2$ if we come from s_2 .

We assume that the discount factor is $\gamma = 0.95$ and the learning rate is $\alpha = 0.9$.

- (i) Specify the probabilistic transition function.
- (ii) Specify the reward function. Note: in this example the reward depends on the action and the state you are coming from, so it should be a function of the form $R(s, a)$.
- (iii) Trace through the first few steps of the Q-learning algorithm and with all Q values initially set to zero. Sample actions according to the exploitation principle, choosing always the best action according to the current Q-values, and if both actions tie prefer a_1 by default. Explain why it is necessary to force exploration through probabilistic choice of actions, in order to ensure convergence to the true Q values.
- (iv) Trace through the first few steps of the Q-learning algorithm with all Q values initially set to zero. Sample actions according to ϵ -greedy strategy, using the following “random” selection for each of the steps: random (a_2 is chosen), random (a_1 is chosen), greedy, random (a_2 is chosen).
- (v) Determine what is the optimal value function V^* ($\gamma = 0.95$).
Hint: consider that V^* is the solution to the Bellman Equation (see Bellman update in Chapter 12, slides 25/26).
Hint2: When you have an expression that maximizes over the expected reward of multiple actions you can guess what the optimal policy is. If you do, then you should at the end confirm that the values you got satisfy the Bellman Equation.
- (vi) Determine what is the optimal Q-value function Q^* ($\gamma = 0.95$). Hint: Explain how the optimal value and Q-value function are related, and use the values in the previous part.

Solution:

- (i) Probabilistic transition function is: $P(s_1 | s_1, a_1) = 1$. $P(s_2 | s_1, a_2) = 1$ $P(s_1 | s_2, a_1) = 1$
 $P(s_2 | s_2, a_2) = 1$

In this case the transition function is deterministic.

- (ii) The reward function is:

$$R(s_1, a_1) = 1$$

$$R(s_1, a_2) = -2$$

$$R(s_2, a_1) = +7$$

$$R(s_2, a_2) = +2$$

(iii) Starting at s_1 with the following Q-values

	s_1	s_2
a_1	0	0
a_2	0	0

We pick action a_1 . We reach s_1 , and update the Q-value:

	s_1	s_2
a_1	0.9	0
a_2	0	0

As, $Q(s_1, a_1) > Q(s_1, a_2)$, we will keep always selecting a_1 and never explore what happens if we execute a_2 , which could lead to better rewards.

	s_1	s_2
a_1	1.7595	0
a_2	0	0

(iv) Now, we run Q-learning according to the ϵ -greedy strategy with the randomness provided in the exercise:

	s_1	s_2
a_1	0	0
a_2	0	0

We pick action a_2 . We reach s_2 , and update the Q-value:

$$expectedval = R(s_1, a_2) + \gamma \max_a Q(s_2, a) = -2 + 0 = -2$$

$$Q'(s_1, a_2) = 0.1 \cdot Q(s_1, a_2) + 0.9 \cdot expectedval = -1.8$$

	s_1	s_2
a_1	0	0
a_2	-1.8	0

We pick action a_1 . We reach s_1 , and update the Q-value:

$$expectedval = R(s_2, a_1) + \gamma \max_a Q(s_1, a) = +7 + 0 = +7$$

$$Q'(s_2, a_1) = 0.1 \cdot Q(s_2, a_1) + 0.9 \cdot expectedval = 6.3$$

	s_1	s_2
a_1	0	6.3
a_2	-1.8	0

We greedily pick action a_1 . We reach s_1 , and update the Q-value:

$$expectedval = R(s_1, a_1) + \gamma \max_a Q(s_1, a) = +1 + 0$$

$$Q'(s_1, a_1) = 0.1 \cdot Q(s_1, a_1) + 0.9 \cdot expectedval = 0.9$$

	s_1	s_2
a_1	0.9	6.3
a_2	-1.8	0

We randomly pick action a_2 . We reach s_2 , and update the Q-value:

$$expectedval = R(s_1, a_2) + \gamma \max_a Q(s_2, a) = -2 + 0.95 \cdot 6.3 = 3.985$$

$$Q'(s_1, a_2) = 0.1 \cdot Q(s_1, a_2) + 0.9 \cdot expectedval = -0.18 + 3.5865 \approx 3.4$$

	s_1	s_2
a_1	0.9	6.3
a_2	3.4	0

The algorithm is already starting to converge towards the optimal policy. However, if it wasn't due to randomly picking a_2 , when the agent assigned it a reward of -1.8 , compared to the reward of $a_1(0.9)$, we would have stuck in a local minima again.

(v) V^* is the optimal value function, which satisfies the Bellman equation:

$$\forall s \in S, V^*(s) = \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma V^*(s'))$$

Replacing for our tiny example, with simplified reward function, we get

$$V^*(s_1) = \max(R(s_1, a_1) + \gamma V^*(s_1), R(s_1, a_2) + \gamma V^*(s_2))$$

$$V^*(s_2) = \max(R(s_2, a_1) + \gamma V^*(s_1), R(s_2, a_2) + \gamma V^*(s_2))$$

Note that $V^*(s_1)$ and $V^*(s_2)$ are the values that we want to compute. We'll call them x_1 and x_2 from now on to make it extra clear that this is just solving a system of equations.

Replacing the known values we get:

$$x_1 = \max(1 + 0.95x_1, -2 + 0.95x_2)$$

$$x_2 = \max(7 + 0.95x_1, 2 + 0.95x_2)$$

Intuitively, the cycle that provides maximum reward each 2 steps is $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_1} s_1$, with a total reward of 5 each 2 steps. By fixing these actions we get: (note: to make this formal, one could compute the value for each of the following equations).

$$x_1 = -2 + 0.95x_2$$

$$x_2 = 7 + 0.95x_1$$

Solving via substitution:

$$x_1 = -2 + 0.95(7 + 0.95x_1) \approx 4.65 + 0.9x_1$$

$$x_1 = V^*(s_1) = 46.5$$

$$x_2 = V^*(s_2) = 51.175$$

We can confirm that these values satisfy the Bellmann equation:

$$46.5 = \max(1 + 0.95 \cdot 46.5, -2 + 0.95 \cdot 51.175) = \max(45.175, 46.6) \approx 46.5$$

(we get an approximation due to rounding in some operations)

$$V^*(s_2) = \max(7 + 0.95 \cdot 46.5, 2 + 0.95 \cdot 51.175) = \max(51.175, 50.6) = 51.175$$

(vi) $Q^*(s, a)$ is the optimal value starting from s and choosing a . Therefore, in this example:

$$Q^*(s_1, a_1) = R(s_1, a_1) + \gamma \cdot V^*(s_1) = 1 + 0.95 \cdot 46.5 = 45.175$$

$$Q^*(s_1, a_2) = R(s_1, a_2) + \gamma \cdot V^*(s_2) = -2 + 0.95 \cdot 51.175 = 46.5$$

$$Q^*(s_2, a_1) = R(s_2, a_1) + \gamma \cdot V^*(s_1) = 7 + 0.95 \cdot 46.5 = 51.175$$

$$Q^*(s_2, a_2) = R(s_2, a_2) + \gamma \cdot V^*(s_2) = 2 + 0.95 \cdot 51.175 = 50.6$$

Q^*	s_1	s_2
a_1	45.175	46.5
a_2	51.175	50.6

Note also that these are exactly the values that we computed when we were confirming that V^* satisfies the Bellmann Equation. Basically, we were confirming if our choice of policy was indeed the correct one or choosing some other action could yield better expected rewards.

Exercise 3 :

Please decide for each of the following statements whether it is true or false and justify your answer (1-3 sentences per statement).

1. It is possible to learn the optimal policy for an MDP by interacting with the environment, even if we do not know a priori the transition probabilities or the reward function.
2. Q-learning is guaranteed to converge to the optimal Q-value function independently of the strategy to select actions at each step.

Solution:

1. True, by applying Q-learning for example (one needs to be careful of having a good choice of action selection and update the learning rate).
2. False. For example, if we always select the same action, we cannot possibly learn anything about what will be the reward or states that could be reached by applying different actions.